

2025年度 コンピュータシステム 全説明スライド 画像圧縮 後半

2025/9/26(金)

担当講師 富岡莉生

2025年度 コンピュータシステム 第7回：データ構造

計算手順の書きやすさや効率化は、与えるデータの形で変わる

1. 情報をまとめて扱うための構造
 - 配列, 集合, クラス
2. 柔軟にデータ構造や数を書き換えるための構造
 - キュー, スタック, リスト, 辞書
3. 計算手順を簡易化, 効率化するための構造
 - ツリー, グラフ, リレーショナルデータベース

計算手順の書きやすさや効率化は、与えるデータの形で変わる

1. 情報をまとめて扱うための構造

- 配列, 集合, クラス

2. 柔軟にデータ構造や数を書き換えるための構造

- キュー, スタック, リスト, 辞書

3. 計算手順を簡易化, 効率化するための構造

- ツリー, グラフ, リレーショナルデータベース

変数を並べて一つにまとめたもの

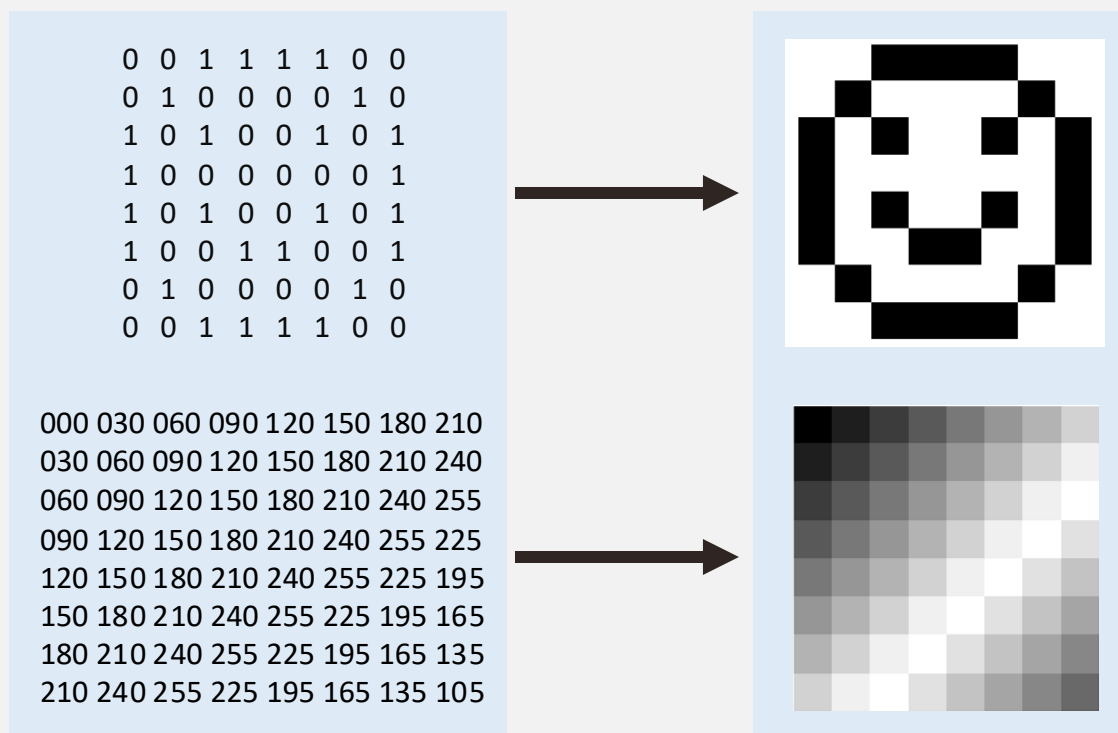
例) 取扱商品一覧

```
Fruits    = ["apple", "banana", "orange"]  
vegetable = ["tomato", "paprika", "okra", "avocado"]  
meat      = ["beef", "pork", "chicken", "mutton"]
```

機能

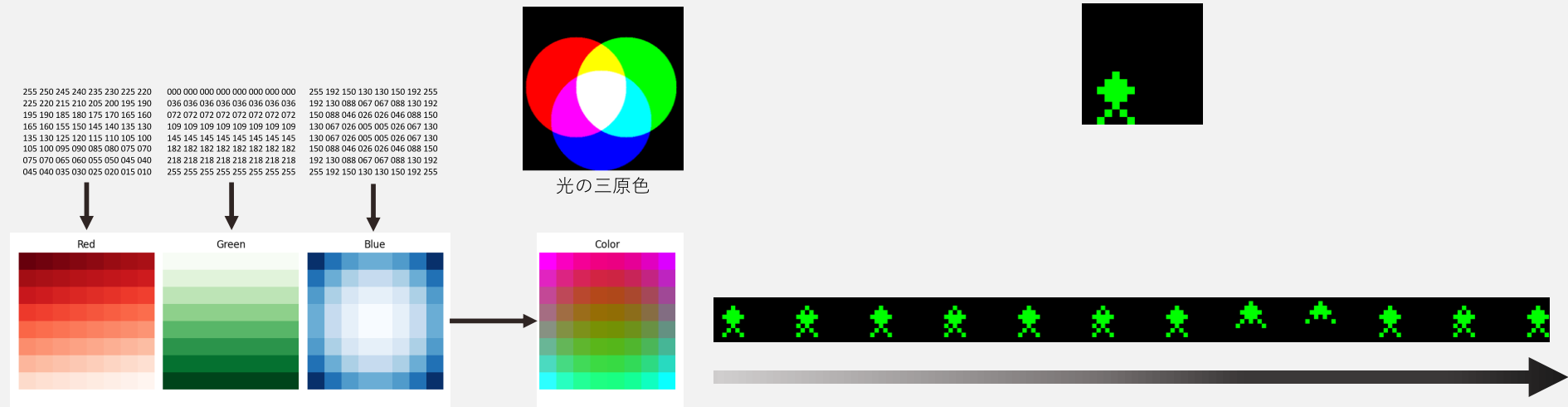
- 同じ型の変数を並べて[]に入れて定義する
- 場所を指定して情報を取り出す
例) fruits[0]は"apple", fruits[1]は"banana"
- 場所を指定して情報を変更する
例) fruits[2] = "grape"
→ fruits = ["apple", "banana", "grape"]
- 関数でサイズ（要素数）を確認する
例) number_of_fruits = size(fruits)
→ 3

変数を二次元状（行と列）にまとめたもの 例）モノクロ画像



画像をたくさんの数値が縦横に並んだ情報として出力する
(第二回：計算機で画像を表示する仕組み)

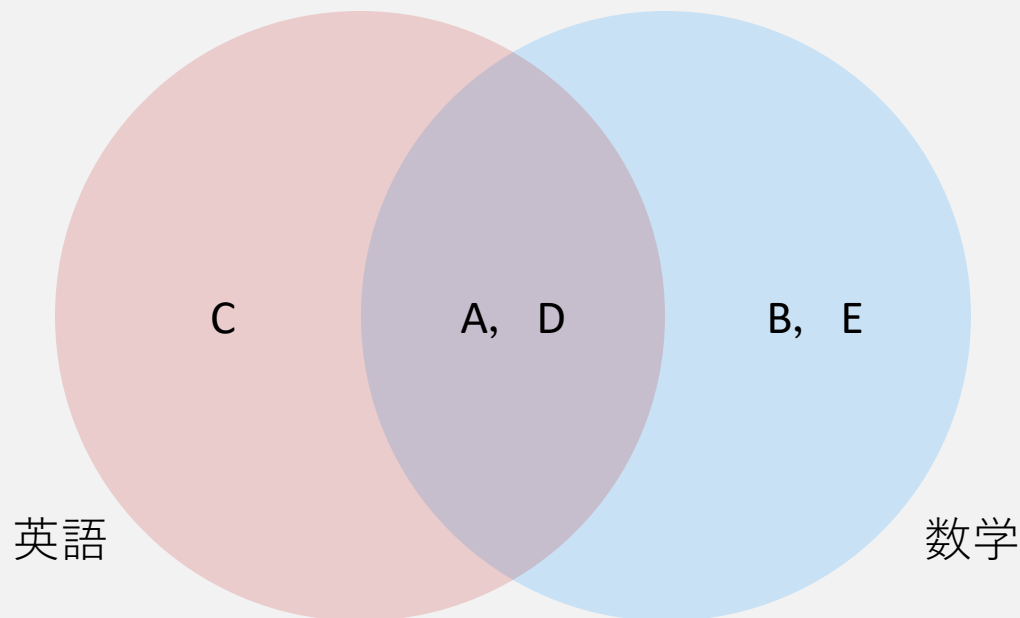
変数を3次元以上にまとめたもの



場所や順序，重複のない要素の集まり

例) 授業履修者の集合

- 英語履修者 = {Aさん, Cさん, Dさん}
- 数学履修者 = {Aさん, Bさん, Dさん, Eさん}
- 英語履修者 かつ 数学履修者 (AND)
→ {Aさん, Dさん}
- 英語履修者 または 数学履修者 (OR)
→ {Aさん, Bさん, Cさん, Dさん, Eさん} # 重複しない



授業の履修者名簿を集合で定義

```
class_a = {"Alice", "Bob", "Charlie", "David"}
```

```
class_b = {"Charlie", "David", "Eve", "Frank"}
```

A または B : どちらかの授業を履修している学生

```
union = class_a | class_b
```

A かつ B : 両方の授業を履修している学生

```
intersection = class_a & class_b
```

A - B : class_aだけ履修している学生

```
only_a = class_a - class_b
```

```
# class_a の履修者
a1 = "Alice"
a2 = "Bob"
a3 = "Charlie"
a4 = "David"

# class_b の履修者
b1 = "Charlie"
b2 = "David"
b3 = "Eve"
b4 = "Frank"
u1 = a1 # Alice
u2 = a2 # Bob
u3 = a3 # Charlie
u4 = a4 # David
```

```
# bの値をaと照合して、重複がなければ追加
if b1 != a1 and b1 != a2 and b1 != a3 and b1 != a4:
    u5 = b1
else:
    u5 = None

if b2 != a1 and b2 != a2 and b2 != a3 and b2 != a4:
    u6 = b2
else:
    u6 = None

if b3 != a1 and b3 != a2 and b3 != a3 and b3 != a4:
    u7 = b3
else:
    u7 = None

if b4 != a1 and b4 != a2 and b4 != a3 and b4 != a4:
    u8 = b4
else:
    u8 = None
```

変数と条件分岐のみで「クラスa履修者 かつ クラスb履修者」を導出する計算手順

いくつかのデータと関数（処理手順）を一つにまとめたもの

例）ゲームキャラクター

ゲームキャラクターは次の変数を持つ

- 攻撃力 = 100
- 防御力 = 50
- 体力（HP） = 200
- 回避率 = 0.1
- わざ = [つつく, ける, 威嚇する, 叩く]

ゲームキャラクターは次の関数を持つ

攻撃：敵の体力にどれくらいのマイナス値を与えるかを計算する関数

数値型 威力 = 攻撃力 × 「わざ」の威力

回避：敵の攻撃を受けたときに攻撃を無効化できるかどうか判定する関数

論理型 回避成功 = 乱数 < 回避率

防御：敵の攻撃を受けたときにマイナス値をどれだけ軽減できるかを計算する関数

数値型 被ダメージ = 威力 - 防御力

回復：自分の体力を増やす計算手順

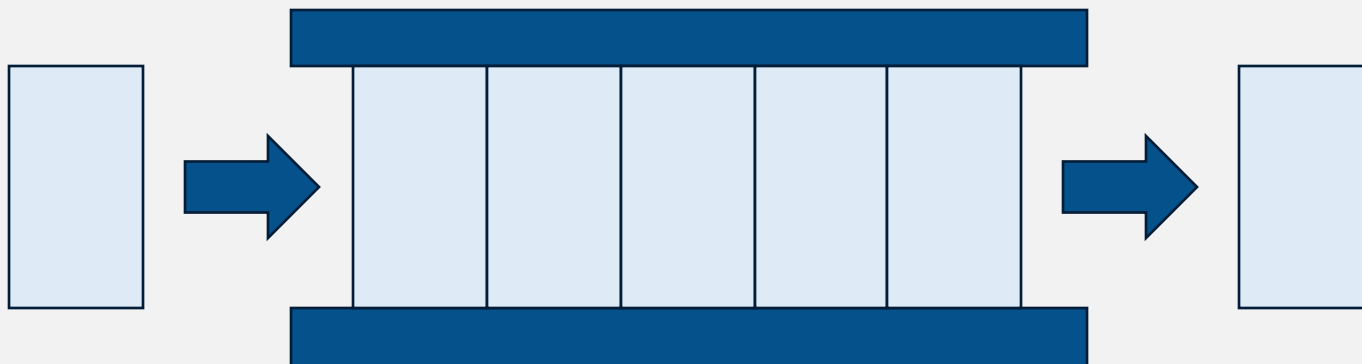
数値型 体力 = 体力 + 回復量

計算手順の書きやすさや効率化は、与えるデータの形で変わる

1. 情報をまとめて扱うための構造
 - 配列, 集合, クラス
2. 柔軟にデータ構造や数を書き換えるための構造
 - キュー, スタック, リスト, 辞書
3. 計算手順を簡易化, 効率化するための構造
 - ツリー, グラフ, リレーショナルデータベース

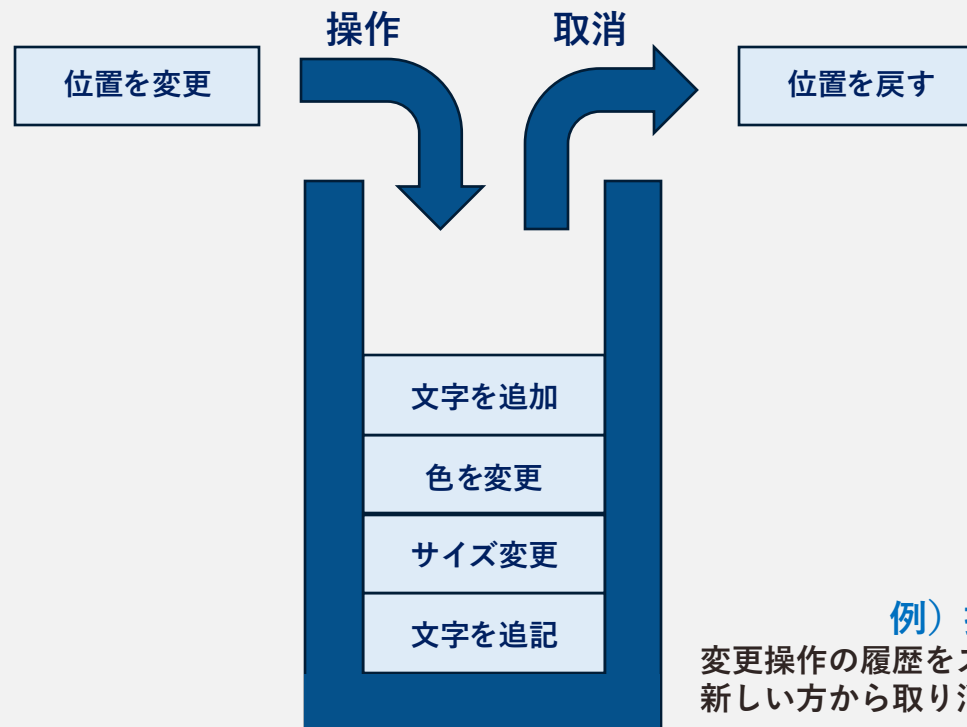
先入れ先出し（FIFO: First In, First Out）

- 比喻）レジでは先に待っている人から対応する
- 例1）YouTubeのキュー
- 例2）カラオケの予約リスト
- 例3）病院/レストランの順番待ち表



後入れ先出し（LIFO: Last In, First Out）

- 比喻）皿洗いでは，上に積まれた新しい皿から対応する
- 例1）操作の取り消し
- 例2）ブラウザの「戻る」機能



自由な位置にデータを挿入/削除できる構造

- 前後の要素のリンクが記述された構造
 - リnkを編集することで順序を編集できる
- 比喻) 新しく入手したトランプを、手札の数字順の位置に挿入する
- 例1) 音楽再生リストの曲順編集
- 例2) PowerPointでのスライド追加/挿入/削除/入れ替え操作

3 この授業について 3

コンピュータシステム
情報社会において情報技術 (IT) を活用できるようにするために
そのキーテクノロジーであるコンピュータの仕組みを学ぶ

授業目的

1. コンピュータの仕組みを理解する
2. コンピュータの仕組みに基づいて現象を正しく把握できるようにする

3 授業内容 4

授業内容

- I. 計算機システムを学ぶ意味を見つける
 - 1. 計算機とプログラムによる自動化
 - 2. 計算機、インターネット、情報システム
 - 3. 計算機やデータ、AIにできることを知る
 - 4. 計算機の基本原理
 - 5. 演習 (データ構造とアルゴリズム)
- II. 計算機やデータ、AIにできることを知る
 - 6. プログラムの基本
 - 7. データ構造
 - 8. アルゴリズム
 - 9. 演習 (データ構造とアルゴリズム)
- III. 問題を「計算機の使える形」に整理する
 - 10. 問題解決のためのモデル：PERT図
 - 11. 問題解決のためのモデル：決定表
 - 12. 演習 (PERT図、決定表)
- IV. 計算機の考え方を体感する
 - 13. 問題解決のためのモデル：PERT図
 - 14. 問題解決のためのモデル：決定表
 - 15. まとめ + 演習 (PERT図、決定表)
- V. 計算機のハードウェア構成を知る
 - 10. 計算機の中の情報表現
 - 11. 演算とハードウェア構成
- VI. 計算機特有の数値表現に慣れる
 - 12. 演習 (基数と論理演算)
- VII. 計算論的思考

5 授業の流れ 5

1. 日本経済や学部の教育方針を知り、現状や将来像に基づいて自分が計算機を学ぶ意味を見つける
2. 計算機やデータ、AIにできることを知り、目的達成に向けた人間と計算機、AIの効果的な分業を想像できるようにする
3. ITシステムの構造や仕組みを学ぶ

スライド 4 / 32 日本語 アクセシビリティ: 検討が必要です

授業内容 4

4

I. 計算機システムを学ぶ意味を見つける

1. 計算機とプログラムによる自動化
2. 計算機、インターネット、情報システム

II. 計算機やデータ、AIにできることを知る

3. 計算機の基本原理
4. 計算機と統計

III. 問題を「計算機の使える形」に整理する

5. 情報システムのデザイン

IV. 計算機の考え方を体感する

6. プログラムの基本
7. データ構造
8. アルゴリズム
9. 演習 (データ構造とアルゴリズム)

V. 計算機のハードウェア構成を知る

10. 計算機の中の情報表現
11. 演算とハードウェア構成

VI. 計算機特有の数値表現に慣れる

12. 演習 (基数と論理演算)

VII. 計算論的思考

13. 問題解決のためのモデル：PERT図
14. 問題解決のためのモデル：決定表
15. まとめ + 演習 (PERT図、決定表)

ノートを入力

スライド 4 / 32 日本語 アクセシビリティ: 検討が必要です

値に独自の名前（キー）をつけて管理できる構造

比喻）辞書（書籍）は、**単語名からそれが持つ情報を見つけ出せる**

- ・ 辞書（データ構造）では、名前を引いて情報を見つけ出せる

例1) アプリの内部設定

- ・ 設定項目に対応する値として、ユーザーが選んだ設定が保存されている

例2) 学生名簿

- ・ 学籍番号から、“所属”、“学年”、“履修科目”などの情報を参照できる

例3) 音楽ストリーミングサービス

- ・ 音楽IDごとに、曲名、アーティスト名、アルバム情報などを参照できる

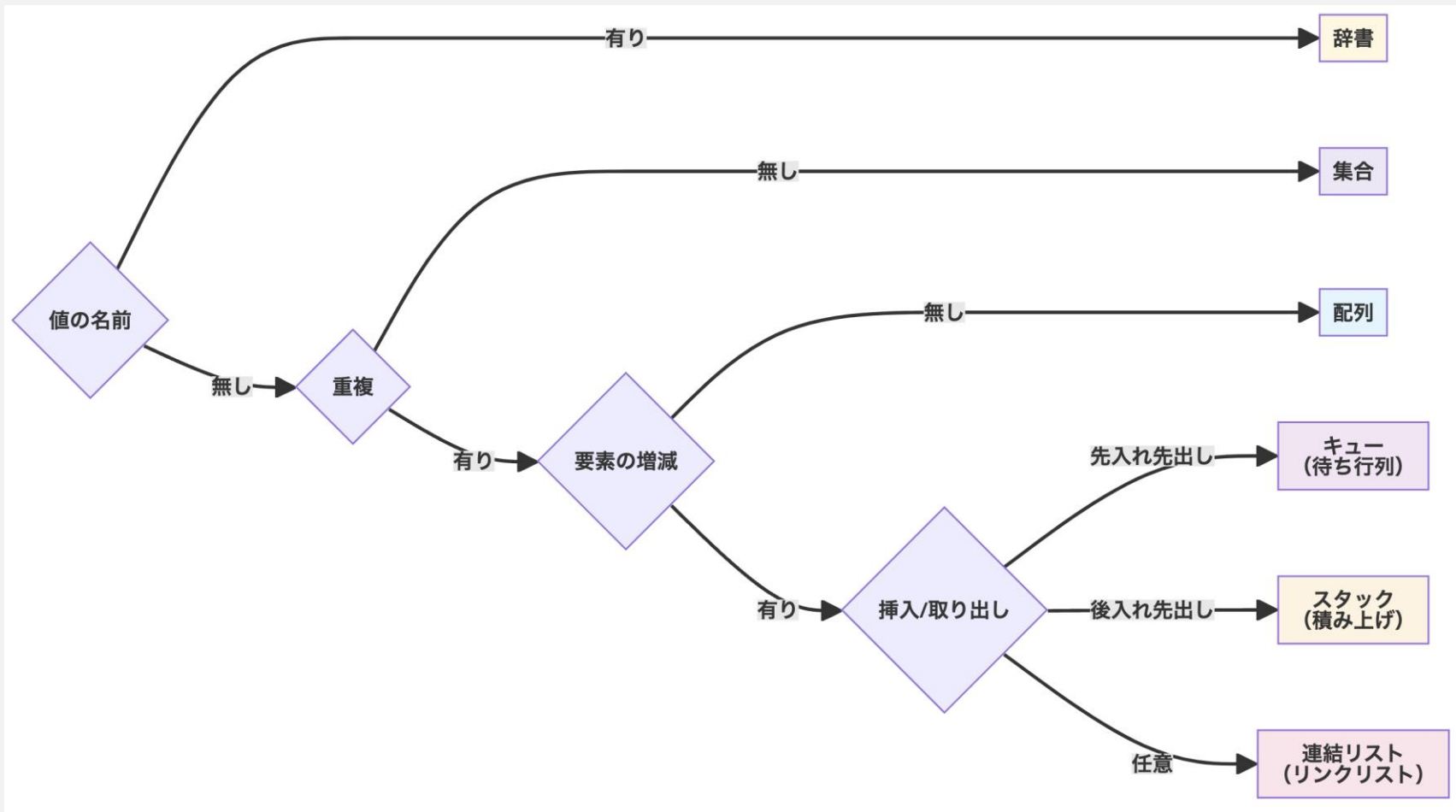
```
個人設定 = {  
  "テーマ": "ダーク",  
  "通知": "オン",  
  "言語": "日本語",  
  "文字サイズ": "中"  
}
```

使用例
個人設定[テーマ] → ダーク
個人設定[言語] → 日本語

```
学生名簿 = {  
  "2023001": {  
    "氏名": "佐藤翔",  
    "所属": "情報学部",  
    "学年": 2,  
    "専攻": "情報科学"  
  },  
  "2023002": {  
    "氏名": "鈴木楓",  
    "所属": "経済学部",  
    "学年": 3,  
    "専攻": "経済政策"  
  }  
}
```

使用例
学生名簿[2023002][所属] → 経済学部

大まかに※これまで紹介してきたデータ構造は以下のように整理できる



※現実には、本講義で紹介していないデータ構造も候補となり、分岐はより複雑で異なる形になります

計算手順の書きやすさや効率化は、与えるデータの形で変わる

1. 情報をまとめて扱うための構造
 - 配列, 集合, クラス
2. 柔軟にデータ構造や数を書き換えるための構造
 - キュー, スタック, リスト, 辞書
3. 計算手順を簡易化, 効率化するための構造
 - ツリー, グラフ, リレーショナルデータベース

階層関係を枝分かれによって表現した構造

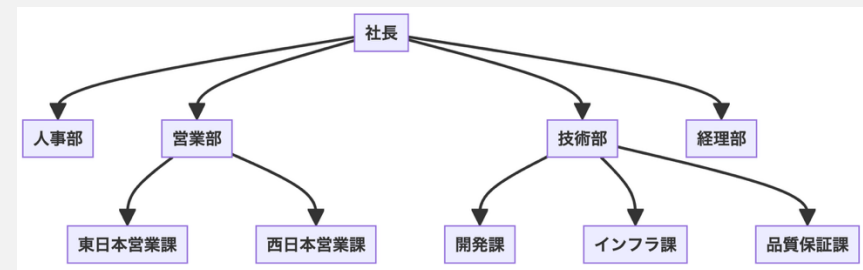
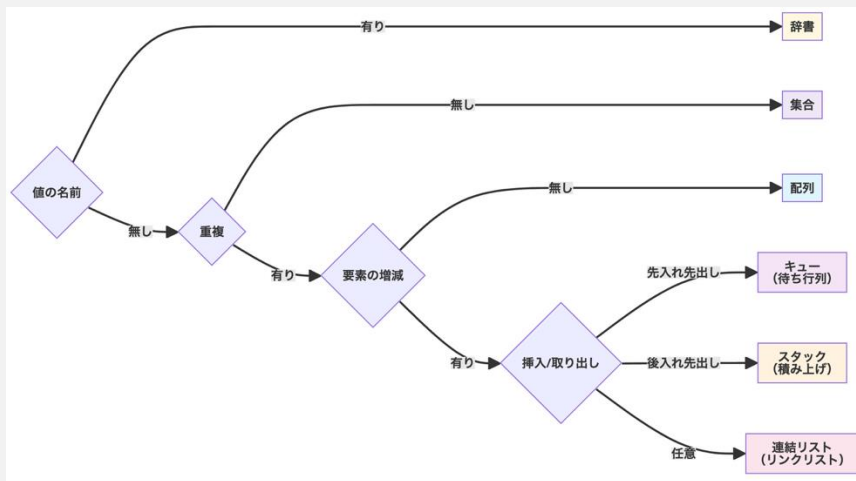
比喻）一つの根から枝分かれをして，末端には葉（情報）がある

→条件分岐（Yes/No）を繰り返すだけで，素早く必要な情報を探し出せる

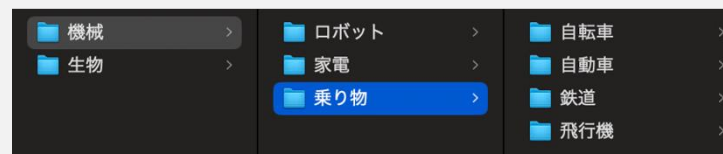
例1）決定木：起点から分岐を続けて，結論に至る

例2）組織図：最高責任者から分岐を続けて，各部署に到達する

例3）ファイルシステム：ファイルは一つのフォルダに必ず属する



...



...

上位

下位

根（ルートディレクトリ，`C:\`など）

枝（フォルダ，ディレクトリ）

葉（各ファイル，`report.pdf`など）

要素とその関係を、「点」とそれを結ぶ「線」で図示するような構造

例1) 路線図：駅（点）が路線で繋がれている関係

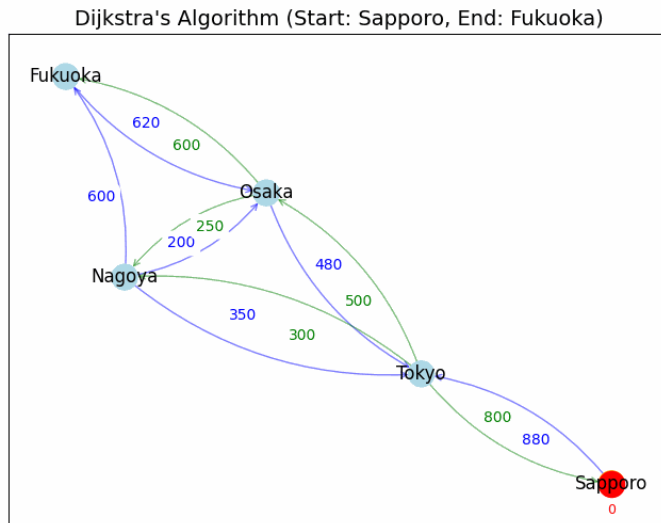
→ある駅から別の駅までの経路情報はグラフで表現できる

例2) ネットワーク：計算機（点）が通信経路（線）で結ばれている関係

→計算機が情報を送るための通信経路はグラフで表現できる

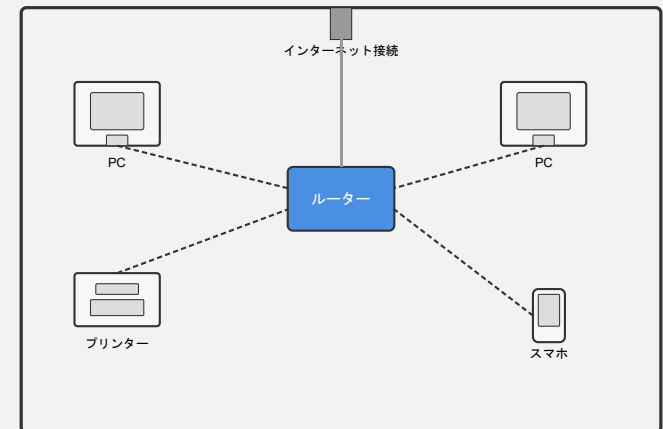
例3) SNS：ユーザー（点）同士は関係（線）で結ばれている

例4) 木構造もグラフの一種



tomiokario/route_search

https://github.com/tomiokario/route_search?tab=readme-ov-file



CC0: hashcc/railmaps: Railway maps of world's famous cities.

<https://github.com/hashcc/railmaps>

相互に参照可能な複数の表に情報を保存する構造

例) 店舗の持つ情報を管理するデータベース (Excel, Notion, SQLなど)

顧客ID (customer_id)	氏名 (name)	住所 (address)
1	山田 太郎	東京都新宿区
2	鈴木 花子	大阪府大阪市
3	佐藤 次郎	北海道札幌市

顧客リスト

商品ID (product_id)	商品名 (product_name)	単価 (price)
A01	ノートパソコン	120000
A02	マウス	2000
A03	キーボード	5000

商品リスト

日付 (date)	顧客ID (customer_id)	商品ID (product_id)	個数 (quantity)
2025-05-01	1	A01	1
2025-05-02	2	A02	2
2025-05-02	1	A03	1

購入リスト

例) 顧客1の購入金額を求める計算手順

1. 購入リストから顧客1の履歴を抽出
2. 商品IDから単価を取得
3. 全履歴で「価格 = 単価 × 個数」を計算
4. 全履歴の「価格」を合計すれば
顧客1の購入金額が導出される

リレーショナルデータベースは、「複数の表」を作成して、「互いに参照する」という一見すると複雑な構造を持つ

当然、エクセルファイルのように「一つの表」に全ての情報を入れて、そこから情報を取り出すという単純な実装もできる。

しかしながら、この実装では非効率な場合がある。

例えば、SNSを作る場合を考える。

エクセルの各行に投稿情報として、「投稿日時、投稿者ID、投稿者名、アイコン画像、投稿内容」を保持して、それをタイムラインに並べることで、簡易的なSNSが実装できそうである。

しかし、仮にあるユーザーが100件の投稿をした場合、そのエクセルには投稿者のアイコンが重複して100個保存されることになり、例えばストレージを圧迫するなどの問題が生じる。

このような場合、例えば次のような解決策がある。

1. 「投稿履歴」という表に「投稿日時、投稿者ID、投稿内容」だけを保持する
2. 「ユーザー」という別の表で「投稿者ID、投稿者名、アイコン画像」を保存する
3. タイムライン作成時には、「投稿履歴」の投稿者IDからユーザー情報を適宜取り出しながら並べる

これはリレーショナルデータベースの考え方である。

計算手順の書きやすさや効率化は、与えるデータの形で変わる

1. 情報をまとめて扱うための構造
 - 配列, 集合, クラス
2. 柔軟にデータ構造や数を書き換えるための構造
 - キュー, スタック, リスト, 辞書
3. 計算手順を簡易化, 効率化するための構造
 - ツリー, グラフ, リレーショナルデータベース

2025年度 コンピュータシステム 第8回：アルゴリズム

アルゴリズムを使うとき同じ機能でも様々な方式から選ぶ必要がある

1. 状況に応じたアルゴリズム選定の重要性
 - 同じ機能でも処理速度が異なるアルゴリズムが存在する
 - 計算手順の少なさと考えやすさのトレードオフ
 - アルゴリズムの速さを見積もる数式
2. 「考えやすさ vs 速さ」の具体例
 - 並び替えアルゴリズムの比較
 - 考えやすいアルゴリズム（挿入ソート，選択ソート）
 - 大きなデータを少ない手順で捌けるアルゴリズム（クイックソート）
3. 状況に応じたアルゴリズム選定
 - メモリ使用量の違い（空間計算量）
 - 問題条件の違い（ソートの安定性）

アルゴリズムを使うとき同じ機能でも様々な方式から選ぶ必要がある

1. 状況に応じたアルゴリズム選定の重要性

- 同じ機能でも処理速度が異なるアルゴリズムが存在する
- 計算手順の少なさと考えやすさのトレードオフ
- アルゴリズムの速さを見積もる数式

2. 「考えやすさ vs 速さ」の具体例

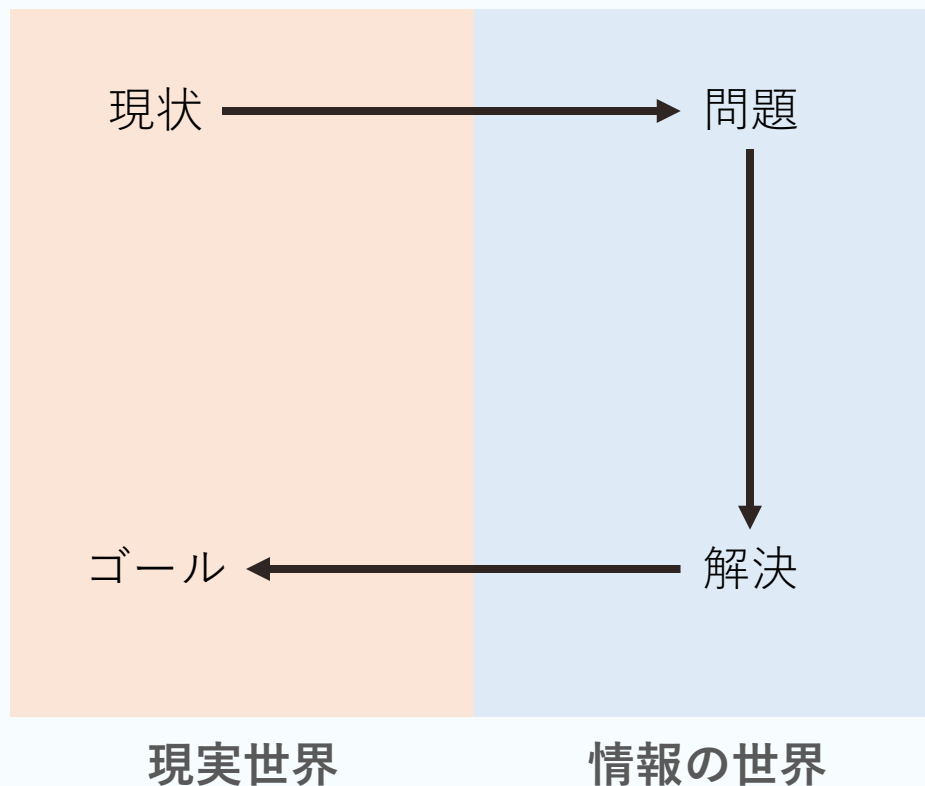
- 並び替えアルゴリズムの比較
- 考えやすいアルゴリズム（挿入ソート，選択ソート）
- 大きなデータを少ない手順で捌けるアルゴリズム（クイックソート）

3. 状況に応じたアルゴリズム選定

- メモリ使用量の違い（空間計算量）
- 問題条件の違い（ソートの安定性）

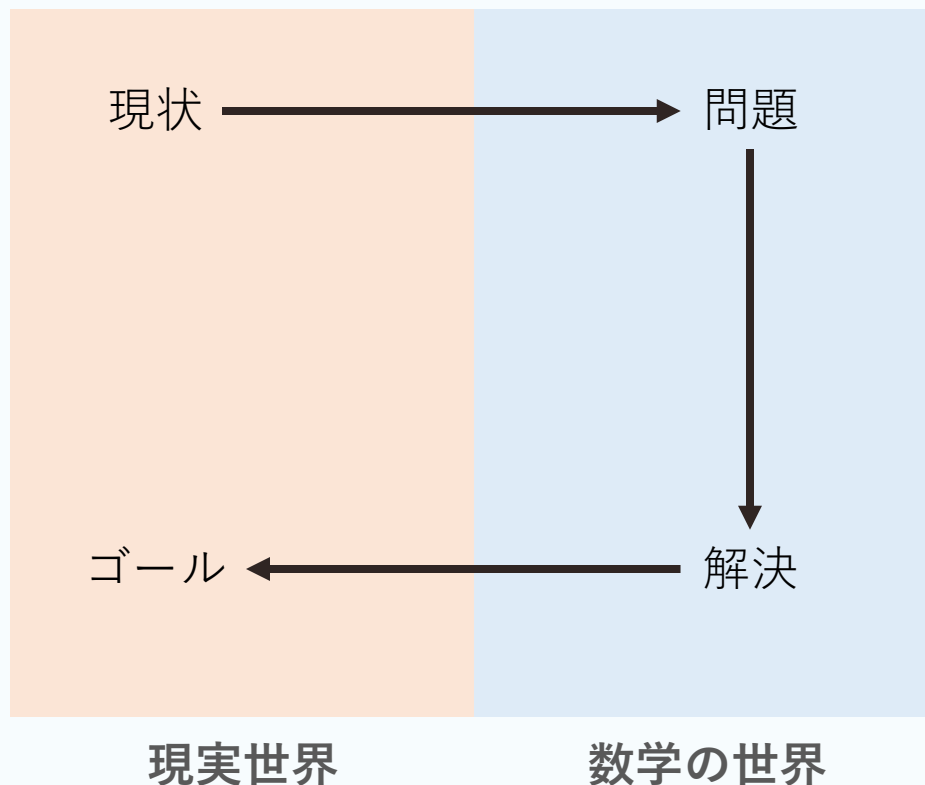
アルゴリズムを使って問題を解決するため

1. 既存アルゴリズムをそのまま使って問題を解決する
2. 頻出アルゴリズムを組み合わせて、状況に応じたアルゴリズムを作る
3. 革新的なアルゴリズムを開発する
 - これまで解決不能だった問題が解ける（高度な専門家の仕事）



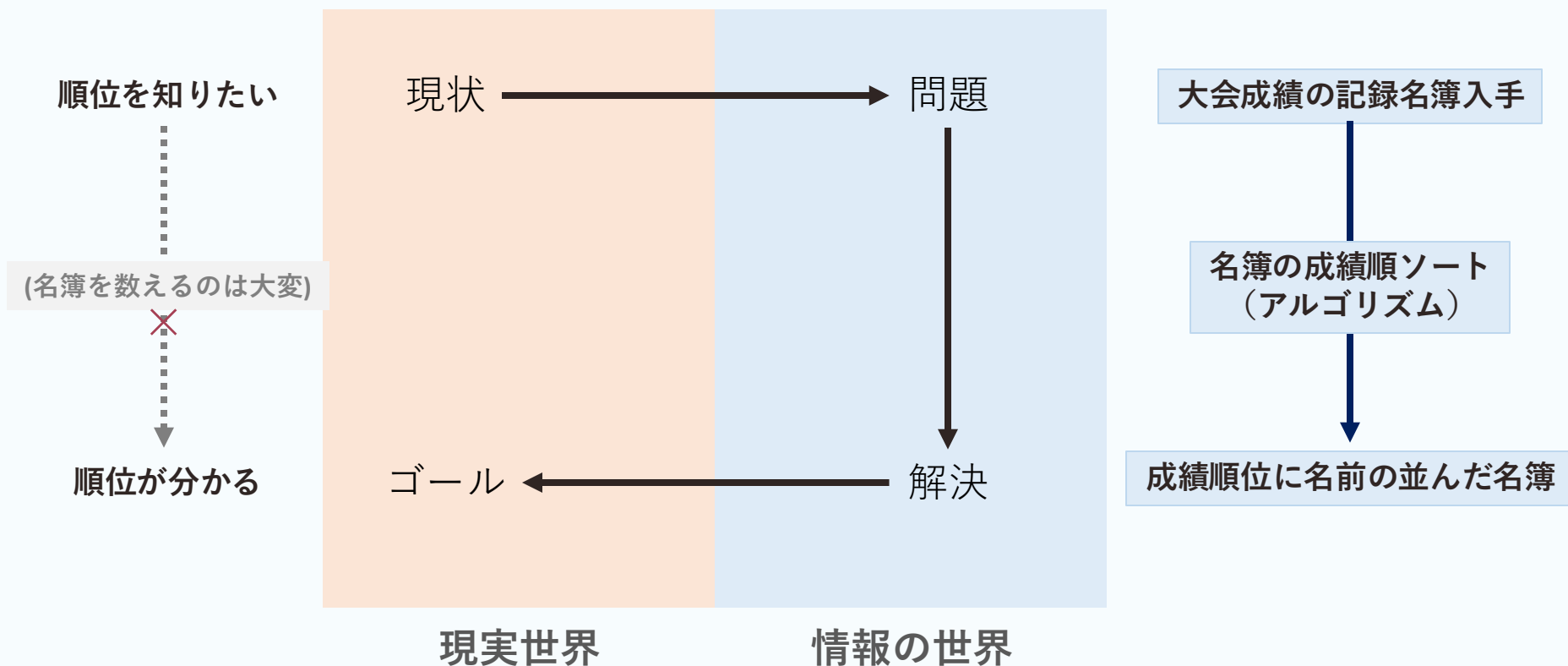
数学で現実世界の問題を解決するため

1. 頻出手順（公式）をそのまま使って問題を解決する
2. 公式を組み合わせて、目の前の問題の解法を見つける
3. 革新的な数学理論を開発する
 - これまで解決不能だった問題が解ける（高度な専門家の仕事）



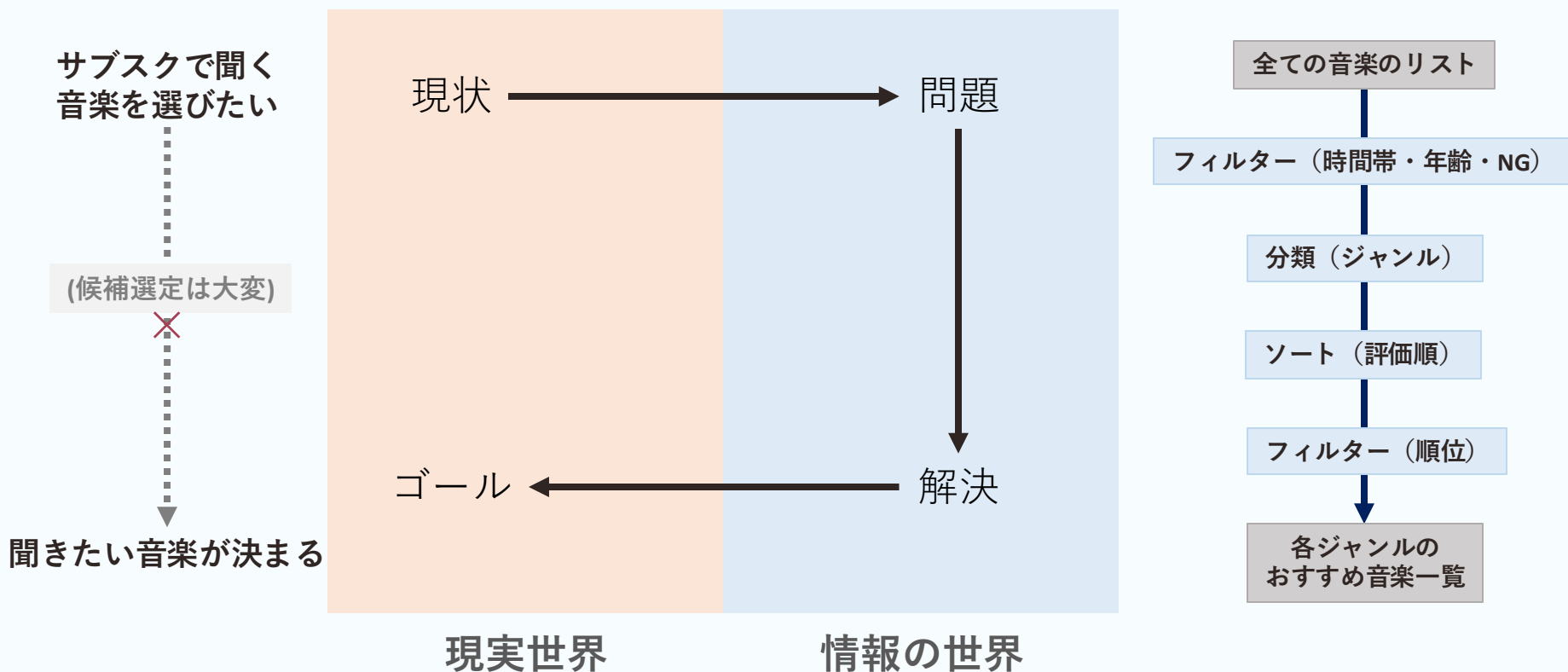
アルゴリズムを使って問題を解決するため

1. 既存アルゴリズムをそのまま使って問題を解決する
2. 頻出アルゴリズムを組み合わせて、状況に応じたアルゴリズムを作る
3. 革新的なアルゴリズムを開発する
 - これまで解決不能だった問題が解ける（高度な専門家の仕事）



アルゴリズムを使って問題を解決するため

1. 既存アルゴリズムをそのまま使って問題を解決する
2. 頻出アルゴリズムを組み合わせて、状況に応じたアルゴリズムを作る
3. 革新的なアルゴリズムを開発する
 - これまで解決不能だった問題が解ける（高度な専門家の仕事）



信頼性

正しく動作することは最も重要なこと

例1) 銀行システム：残高を正しく計算する

例2) 自動車の燃料噴射装置：適切な量を噴射する

その他，飛行機の墜落やOSの異常停止は望ましくない

計算速度

速さが信頼性を決めることもある

例1) 飛行機の障害物警告プログラム：接近前に警告しなければならない

例2) スマホアプリ：ユーザーが不快を感じる前に起動することが望ましい

例3) データベース：リクエストが届くより速く処理を終えなければ溢れる

例4) 生物学研究：系統推論の計算時間がかかりすぎると研究にならない

アルゴリズムの処理効率と分かりやすさの両立は難しい

計算複雑性 (computational complexity)

計算の手順としての複雑さ (≡ 処理の遅さ, 計算時間)

概念的複雑性 (conceptual complexity)

計算を考える上での複雑さ (≡ 実装の難解さ, 認知負荷)

トレードオフ

一方を重視すればもう一方が損なわれる関係性



理解や実装がしやすく速いアルゴリズムはなかなかない※

→ 開発・運用コストを考えると要求に対して十分な速さのアルゴリズムを選ぶことが重要

※正確には、「あるならそれを使えば良いが、現実としてトレードオフに直面し、意思決定を求められる場面は多い」

「実際に実行して時間を測る」方法は簡単だが万能ではない

- プログラムを動かす計算機に依存する（例：開発者のPC vs 格安スマホ）
- 実装するプログラミング言語に依存する
- プログラムの書き方に依存する
- 入力データの数に依存する



計算にかかる時間を事前に簡単に見積もりたい

条件を一般化して、おおよその計算時間が簡単に分かる式を作る

例1) 計算機の抽象化 (ランダムアクセス機械)

- 変数作成, 比較, 演算, メモリ参照などの基本操作を一定時間で行う装置を仮定する

例2) 計算時間の抽象化 (ステップ数)

- アルゴリズムに含まれる基本操作の回数 (ステップ数) を数える
- ステップ数を一般的な処理時間の見積もりとして用いる
 - CPUの処理性能などを代入すればおおよその時間がわかる

例3) 入力データの変数化

- 入力データサイズを変数 (nなど) として, 一般的に取り扱う
 - 条件が変化した場合でも, 変数の値を変えればステップ数がわかる

全ての条件を網羅的に検証する必要はあまりない

一般にはおおまかに次の三つが想定される

- 最良計算時間：想定される入力のうち、最も短い処理時間
- 最悪計算時間：想定される入力のうち、最も長い処理時間
- 平均計算時間：想定される全ての入力の平均的な処理時間

例) 二つの鍵がついた玄関ドアを開錠する

- 鍵の確認、鍵を回す操作を1ステップとする

1. 鍵Aを確認する
2. 鍵Aが閉まっていたら、鍵Aを回して開ける
3. 鍵Bを確認する
4. 鍵Bが閉まっていたら、鍵Bを回して開ける



※ChatGPTで生成

鍵A	鍵B	ステップ数
開	開	2 (確認2回 + 回転0回, 最良計算時間)
開	閉	3 (確認2回 + 回転1回)
閉	開	3 (確認2回 + 回転1回)
閉	閉	4 (確認2回 + 回転2回, 最悪計算時間)

最良計算時間：2ステップ

最悪計算時間：4ステップ

平均計算時間： $(2+3+3+4)/4 = 3$ ステップ

ほとんどのアルゴリズムは小さい入力サイズなら十分高速

見積もりが必要になるのは、入力サイズが非常に大きい場合のみ

例) 10万人の顧客データベースの操作, 100万本の光線を用いた描画

データが増えるとどれくらいのペースで計算時間が増加するかを大まかに見積もりたい

ステップ数の数式のうち最も増加速度の速い項のみを抽出すれば見積もりに十分

例1) 定数より変数の方が, データサイズ n が増えたときの影響は大きい

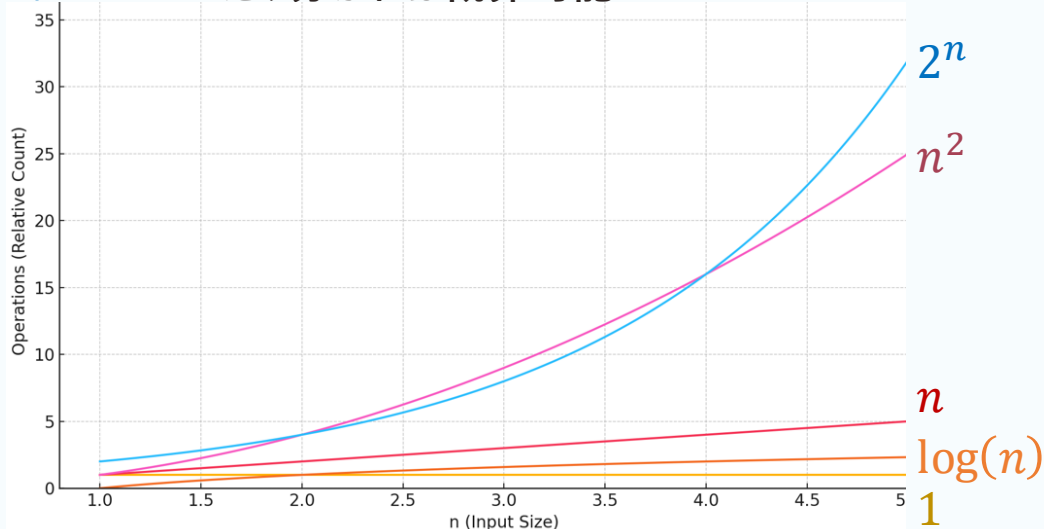
$n+5 \rightarrow n$ だけ分かれば概算可能

例2) 指数が小さいより大きい方が, 影響は大きい

$3n^5 + n^3 + 8 \rightarrow n^2$ だけ分かれば概算可能

例3) 冪乗関数より指数関数の方が, 影響は大きい

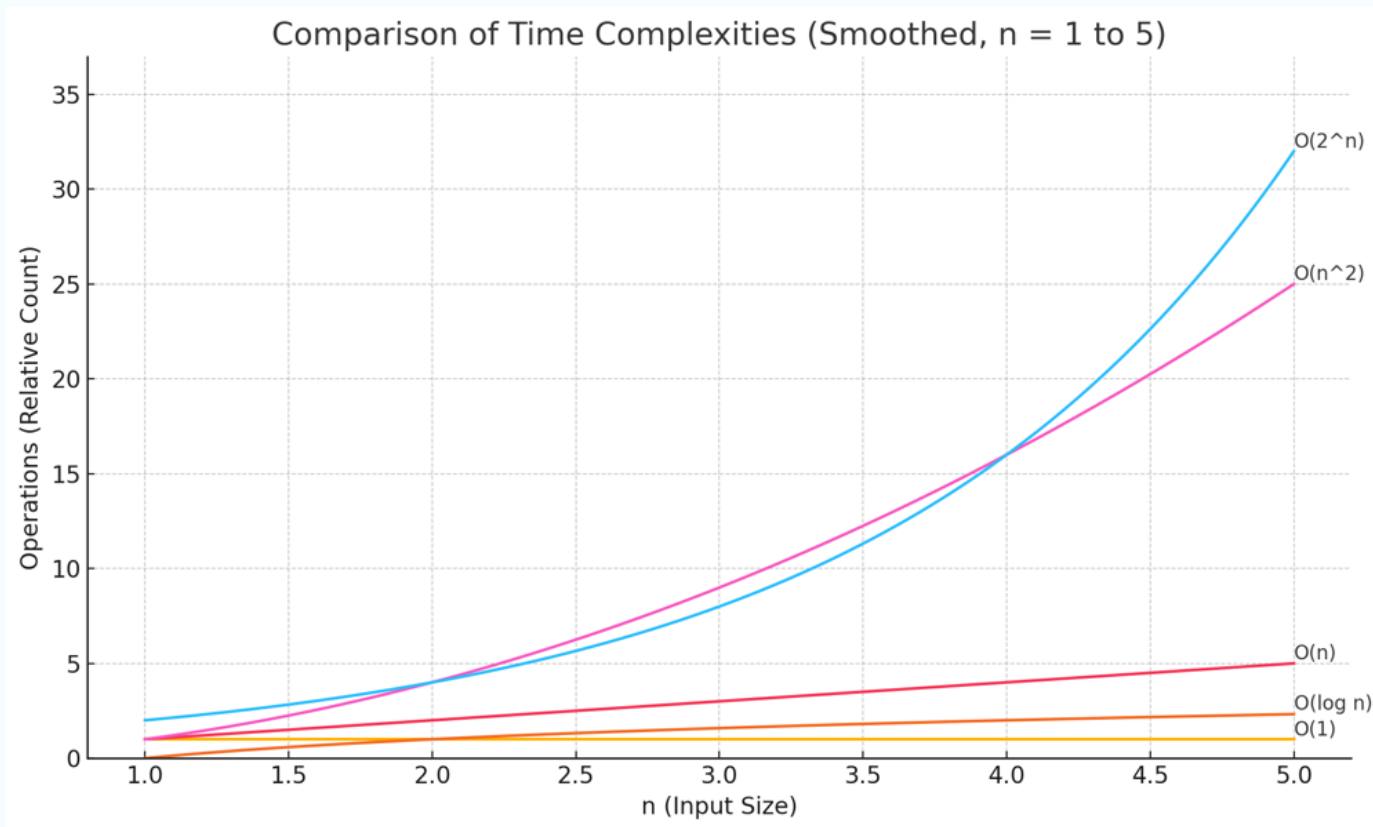
$2^n + n^3 \rightarrow 2^n$ だけ分かれば概算可能



簡略化の工夫の施された「計算時間の目安」を表現する方法

「 $f(n)$ の増加速度は n^2 と同程度だ」と言いたいとき、日常的な場面では※、
「 $f(n)$ は $O(n^2)$ である」とよく言われる

例) 計算時間が $3n^2 + n + 8$ のアルゴリズムは $O(n^2)$ である



※厳密には、「 $f(n)$ の増加速度は n^2 よりも小さい」（上界）という意味だが、本授業ではそこまで踏み込まない

代表的なアルゴリズムの速さの分類

指数計算時間：指数関数的に時間が増加する（ほとんどの場面で問題）

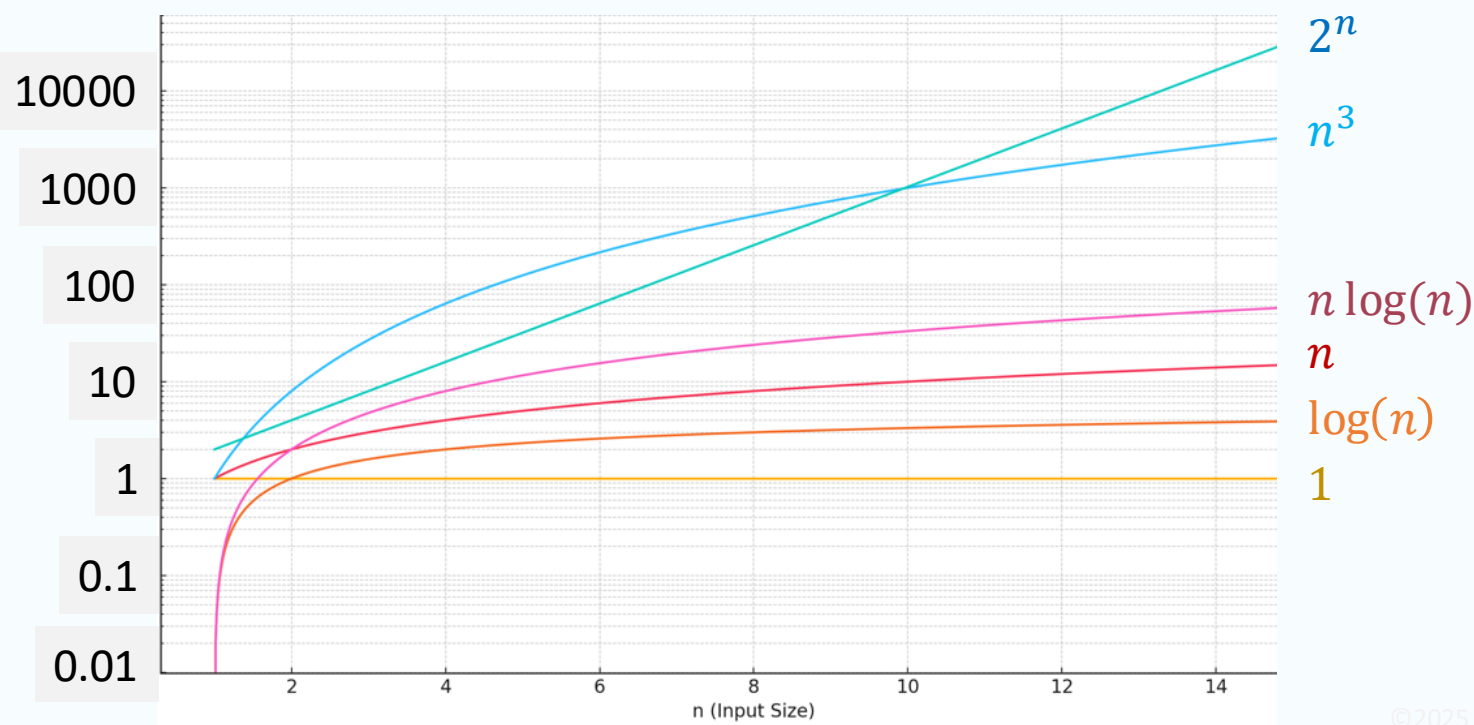
多項式計算時間：冪乗関数的に時間が増える（遅いが使用場面も多い）

対数線形計算時間：比例より大きく計算手順が増える $O(n \log(n))$

線形計算時間：データ数に概ね比例して時間が増える

対数計算時間：データが増えてもほとんど時間は増えない $O(\log(n))$

定数計算時間：データが増えても時間は変わらない



問題の規模に応じて，アルゴリズムがどれくらいの速度で動くのかの体感を持つことは重要

計算複雑性(処理効率)，概念的複雑性(開発・拡張コスト)，計算機性能(速度)のバランスを検討できる

- 例1) 毎秒膨大なリクエストが届くサービスなら，それに見合う効率のアルゴリズムと計算機が必要
- 例2) 操縦支援や株取引のように一瞬の遅れが命取りなら，入力や効率の事前見積りは必須
- 例3) 人間がリアルタイムで操作するアプリなら，快適な操作感を実現する程度の効率が必要

漸近記法に基づく見積りと体感が結びつけば，問題に対して最適なマシンやシステムを発注したり，それに関して専門家に相談したりしやすくなる

(現実の計算機の処理速度についての詳細は第10回，11回で取り扱う)

アルゴリズムを使うとき同じ機能でも様々な方式から選ぶ必要がある

1. 状況に応じたアルゴリズム選定の重要性
 - 同じ機能でも処理速度が異なるアルゴリズムが存在する
 - 計算手順の少なさと考えやすさのトレードオフ
 - アルゴリズムの速さを見積もる数式
2. 「考えやすさ vs 速さ」の具体例
 - 並び替えアルゴリズムの比較
 - 考えやすいアルゴリズム（挿入ソート，選択ソート）
 - 大きなデータを少ない手順で捌けるアルゴリズム（クイックソート）
3. 状況に応じたアルゴリズム選定
 - メモリ使用量の違い（空間計算量）
 - 問題条件の違い（ソートの安定性）

配列を順番通りに並び替えるアルゴリズム

素朴な発想で理解/実装しやすい（概念的複雑性が低い）

- バブルソート
- 選択ソート

大きなデータを少ない手順で捌ける（計算複雑性が低い）

- クイックソート

配列を順番通りに並び替えるアルゴリズム

素朴な発想で理解/実装しやすい（概念的複雑性が低い）

- バブルソート
- 選択ソート

大きなデータを少ない手順で捌ける（計算複雑性が低い）

- クイックソート

1つずつ要素を取り出し、適切な位置に挿入していく

- 山札から一枚ずつ引いて、手札の適切な位置に挿入して並べる方法と同じ

手札

4

4	8
---	---

2	4	8
---	---	---

山札

4	8	2	3	7	10	1
---	---	---	---	---	----	---

8	2	3	7	10	1
---	---	---	---	----	---

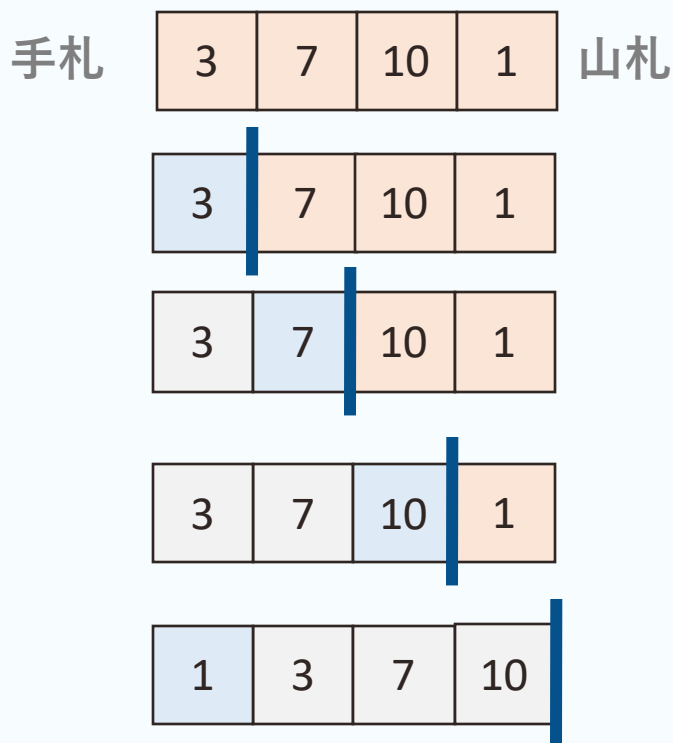
2	3	7	10	1
---	---	---	----	---

3	7	10	1
---	---	----	---

最良計算量： $O(n)$
平均計算量： $O(n^2)$
最悪計算量： $O(n^2)$

1つずつ要素を取り出し、適切な位置に挿入していく

山札から一枚ずつ引いて、手札の適切な位置に挿入して並べる方法と同じ



最良計算量： $O(n)$

平均計算量： $O(n^2)$

最悪計算量： $O(n^2)$

- 配列から最小値を探し、先頭要素と入れ替える

最小要素を探す

4	8	2	3	7	10	1
---	---	---	---	---	----	---

先頭と入れ替える

1	8	2	3	7	10	4
---	---	---	---	---	----	---

最小要素を探す

1	8	2	3	7	10	4
---	---	---	---	---	----	---

先頭と入れ替える

1	2	8	3	7	10	4
---	---	---	---	---	----	---

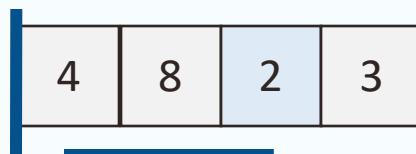
最良計算量： $O(n^2)$

平均計算量： $O(n^2)$

最悪計算量： $O(n^2)$

配列から最小値を探し，先頭要素と入れ替える

最小要素を探す



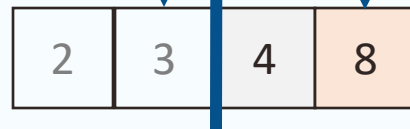
先頭と入れ替える



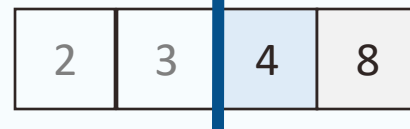
最小要素を探す



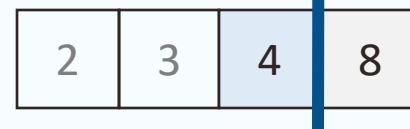
先頭と入れ替える



最小要素を探す



先頭と入れ替える



最良計算量： $O(n^2)$

平均計算量： $O(n^2)$

最悪計算量： $O(n^2)$

配列を順番通りに並び替えるアルゴリズム

素朴な発想で理解/実装しやすい（概念的複雑性が低い）

- バブルソート
- 選択ソート

大きなデータを少ない手順で捌ける（計算複雑性が低い）

- クイックソート

基準を決めて、それ以上/以下に分ける操作を繰り返す

基準を決める

4	8	2	3	7	10	1
---	---	---	---	---	----	---

基準

最良計算量： $O(n \log n)$

平均計算量： $O(n \log n)$

最悪計算量： $O(n^2)$

基準以上/以下に分ける

2	1	3	4	8	7	10
---	---	---	---	---	---	----

基準以下

基準以上

各グループに基準を決める

2	1	3	4	8	7	10
---	---	---	---	---	---	----

基準

基準

基準以上/以下に分ける

1	2	3	4	7	8	10
---	---	---	---	---	---	----

基準以上

基準以下

基準以上

単体になったら終了
2つ以上なら基準を決める

1

2

3

4

7

8

10

1

2

3

4

7

8

10

全て1つの要素に分離されたら
並び替えが完了している

1

2

3

4

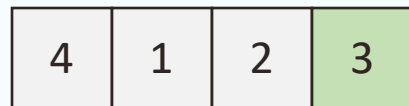
7

8

10

基準を決めて、それ以上/以下に分ける操作を繰り返す

基準を決める



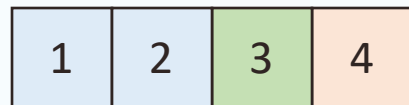
基準

最良計算量 : $O(n \log n)$

平均計算量 : $O(n \log n)$

最悪計算量 : $O(n^2)$

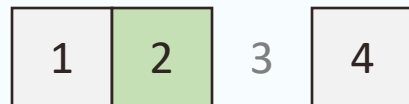
基準以上/以下に分ける



基準以下

基準以上

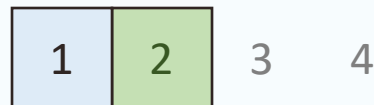
各グループに基準を決める



基準

要素が一つになったら探索終了

基準以上/以下に分ける



基準以下

全て1つの要素に分離されたら
並び替えが完了している

1 2 3 4

アルゴリズムを使うとき同じ機能でも様々な方式から選ぶ必要がある

1. 状況に応じたアルゴリズム選定の重要性
 - 同じ機能でも処理速度が異なるアルゴリズムが存在する
 - 計算手順の少なさと考えやすさのトレードオフ
 - アルゴリズムの速さを見積もる数式
2. 「考えやすさ vs 速さ」の具体例
 - 並び替えアルゴリズムの比較
 - 考えやすいアルゴリズム（挿入ソート，選択ソート）
 - 大きなデータを少ない手順で捌けるアルゴリズム（クイックソート）
3. 状況に応じたアルゴリズム選定
 - メモリ使用量の違い（空間計算量）
 - 問題条件の違い（ソートの安定性）

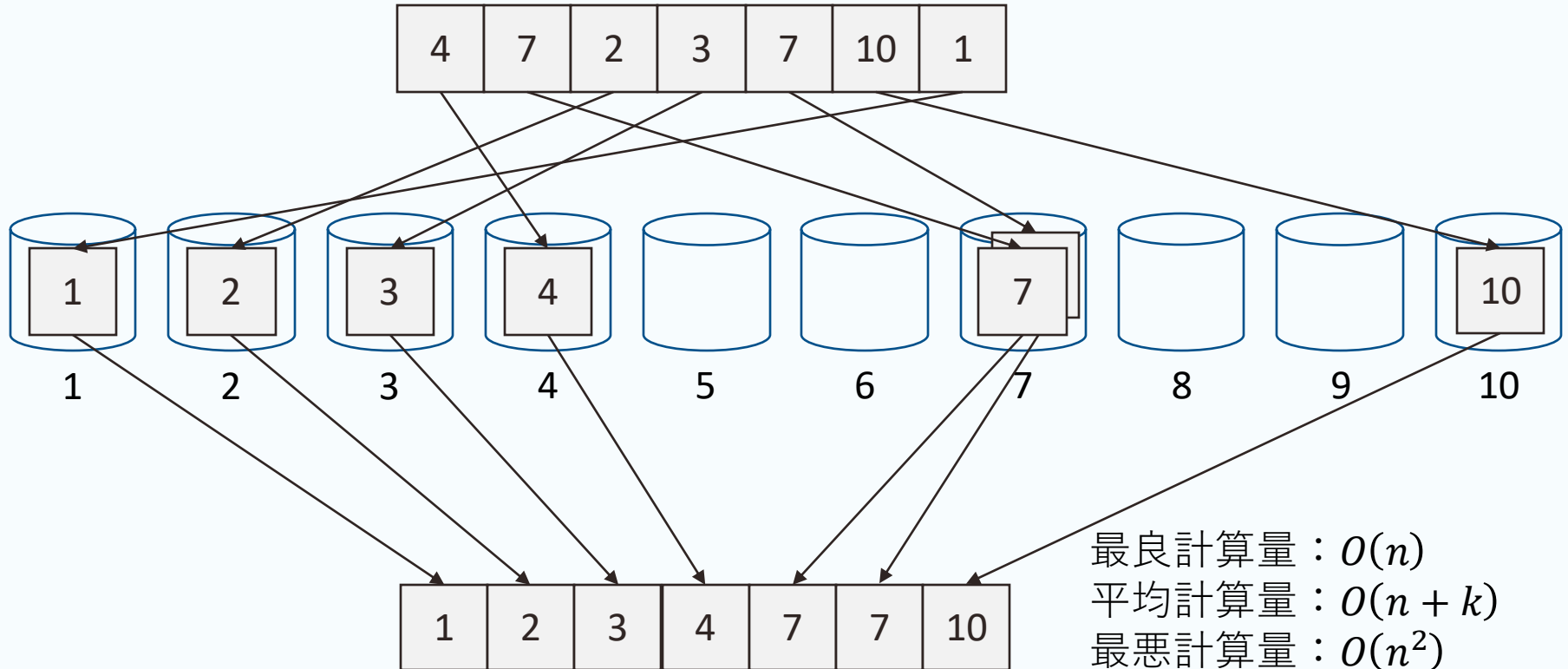
メモリ消費の少なさは重要な選定要件になりうる

- 計算機のメモリ（記憶容量）は有限
- 手順の少なさや理解しやすさと同様に，メモリ消費の少なさも重要

候補数分のバケットを用意して、各値をそれに入れて、整列する

- 要素を1回ずつ操作すれば良いため非常に高速
- 候補数が少ない場合にのみ有効な特殊なアルゴリズム

例) 小数や整数のように候補が無数にある場合、無数のバケットが必要



最良計算量 : $O(n)$
平均計算量 : $O(n + k)$
最悪計算量 : $O(n^2)$
最悪空間計算量 : $O(n \cdot k)$
※ k はバケットの数

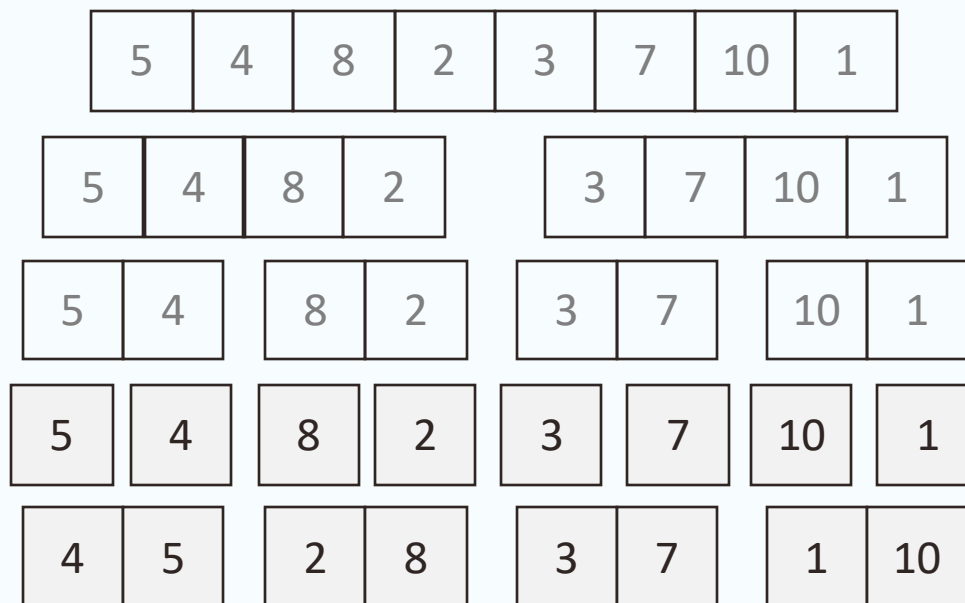
同じ値の要素があるとき、並び替え前の順序を維持する性質

- 二つ以上の項目で対象を比較したいときに有用

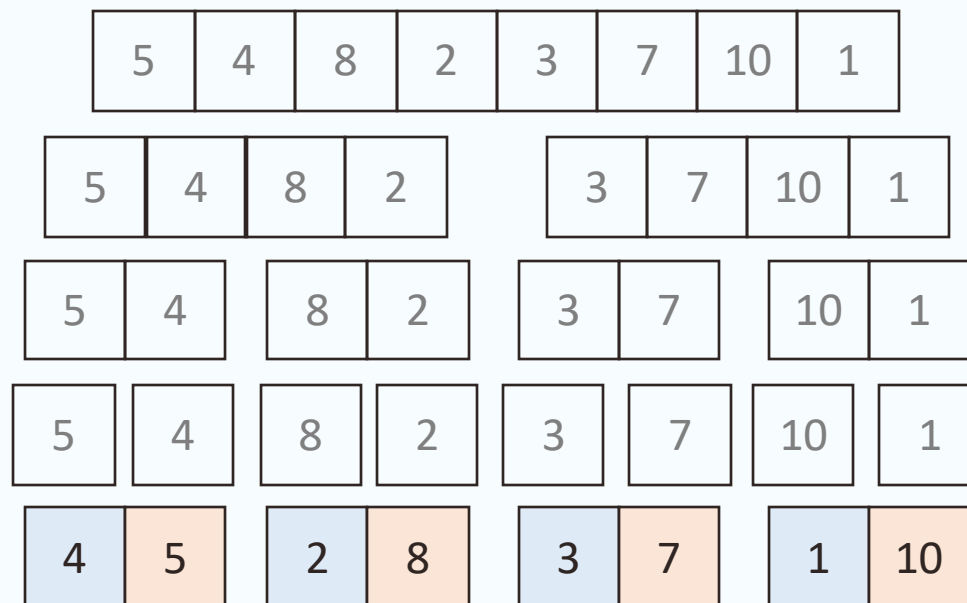
例1) テストの成績が同じなら、内申点を比較して合格者を決めたい

- (1)名簿を内申点順にソートしてから、(2)テストの成績順にソートする
 - 安定ソートであれば、成績が同じ受験者は内申点順に並ぶ
 - 不安定ソートなら、成績が同じ受験者は内申点に関係ない順序で並ぶ
- クイックソートは速いが不安定ソート
 - 安定性と速さが必要な場合、マージソートなどが用いられることがある

(1)データを半分に分割して，(2)決まった規則で統合していく

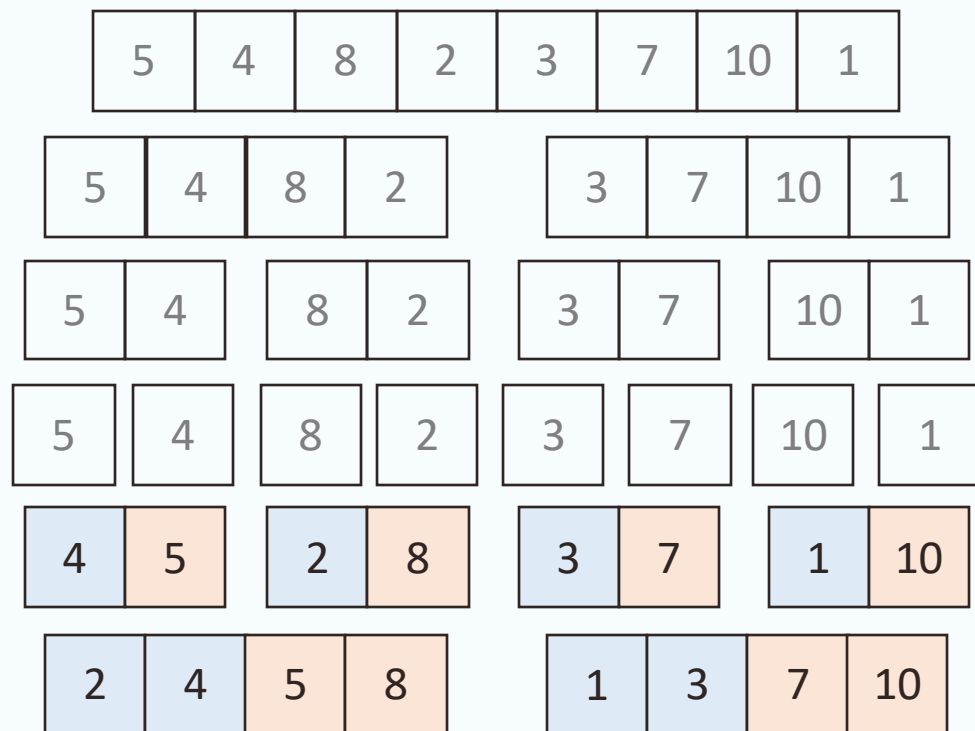


(1)データを半分に分割して，(2)決まった規則で統合していく



グループ内で
順番をつける

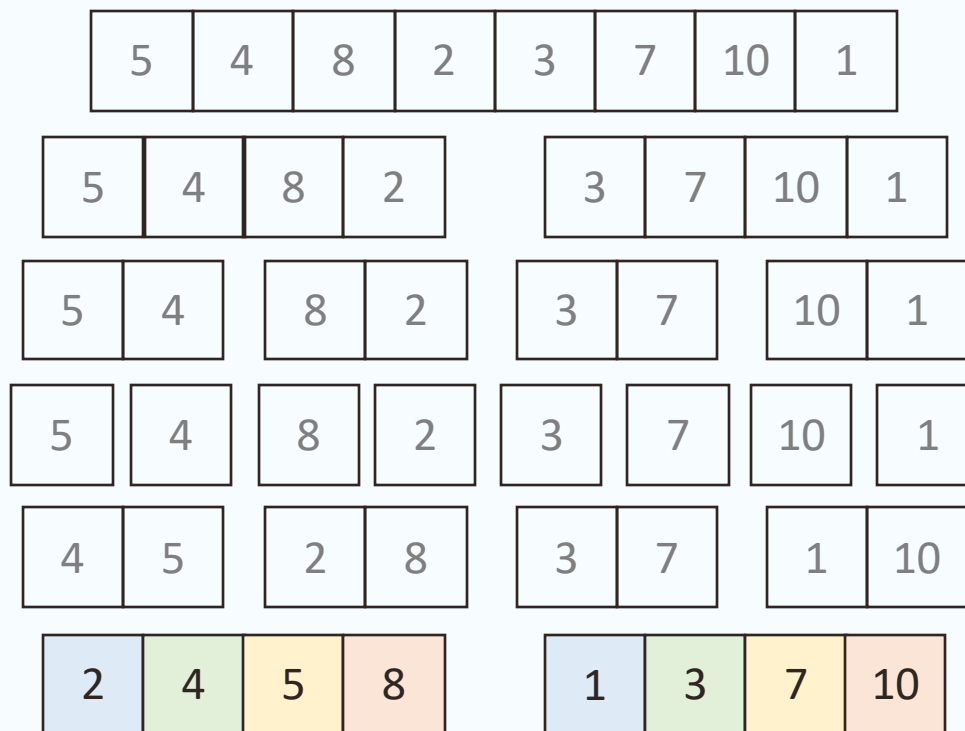
(1)データを半分に分割して，(2)決まった規則で統合していく



①青色同士を比較

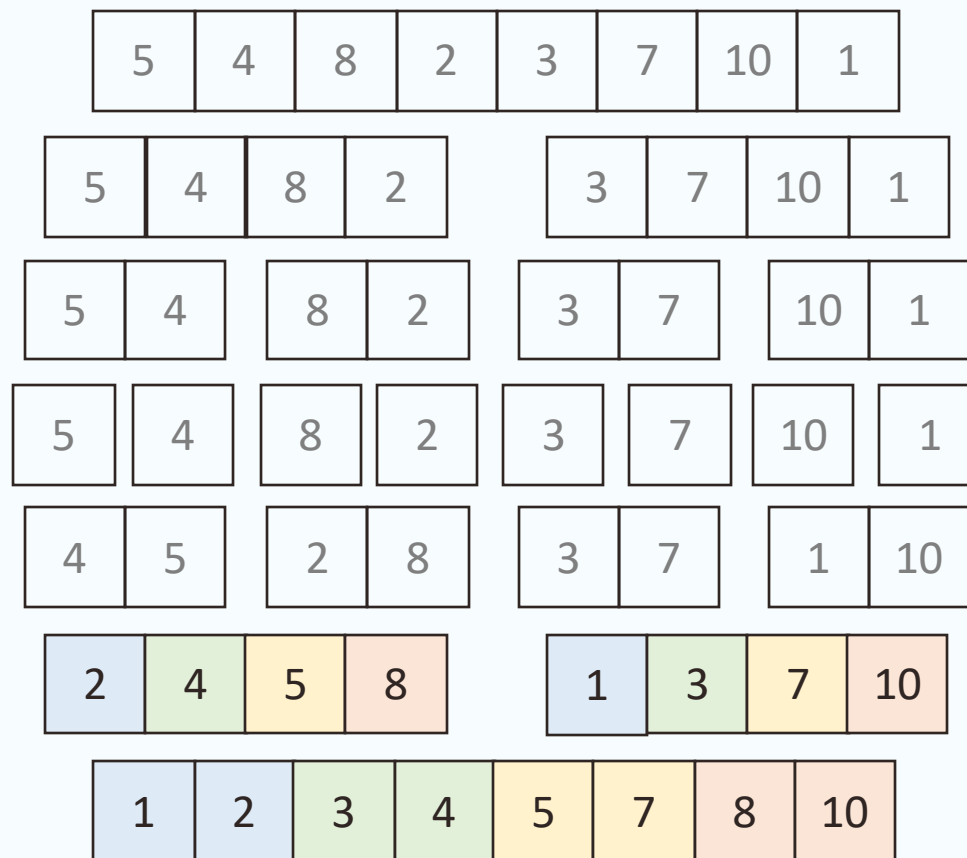
②橙色同士を比較

(1)データを半分に分割して，(2)決まった規則で統合していく



グループ内で
順番をつける

(1)データを半分に分割して，(2)決まった規則で統合していく



- ①青色同士を比較
- ②緑色同士を比較
- ③黄色同士を比較
- ④橙色同士を比較

アルゴリズムを使うとき同じ機能でも様々な方式から選ぶ必要がある

1. 状況に応じたアルゴリズム選定の重要性
 - 同じ機能でも処理速度が異なるアルゴリズムが存在する
 - 計算手順の少なさと考えやすさのトレードオフ
 - アルゴリズムの速さを見積もる数式
2. 「考えやすさ vs 速さ」の具体例
 - 並び替えアルゴリズムの比較
 - 考えやすいアルゴリズム（挿入ソート，選択ソート）
 - 大きなデータを少ない手順で捌けるアルゴリズム（クイックソート）
3. 状況に応じたアルゴリズム選定
 - メモリ使用量の違い（空間計算量）
 - 問題条件の違い（ソートの安定性）

2025年度 コンピュータシステム

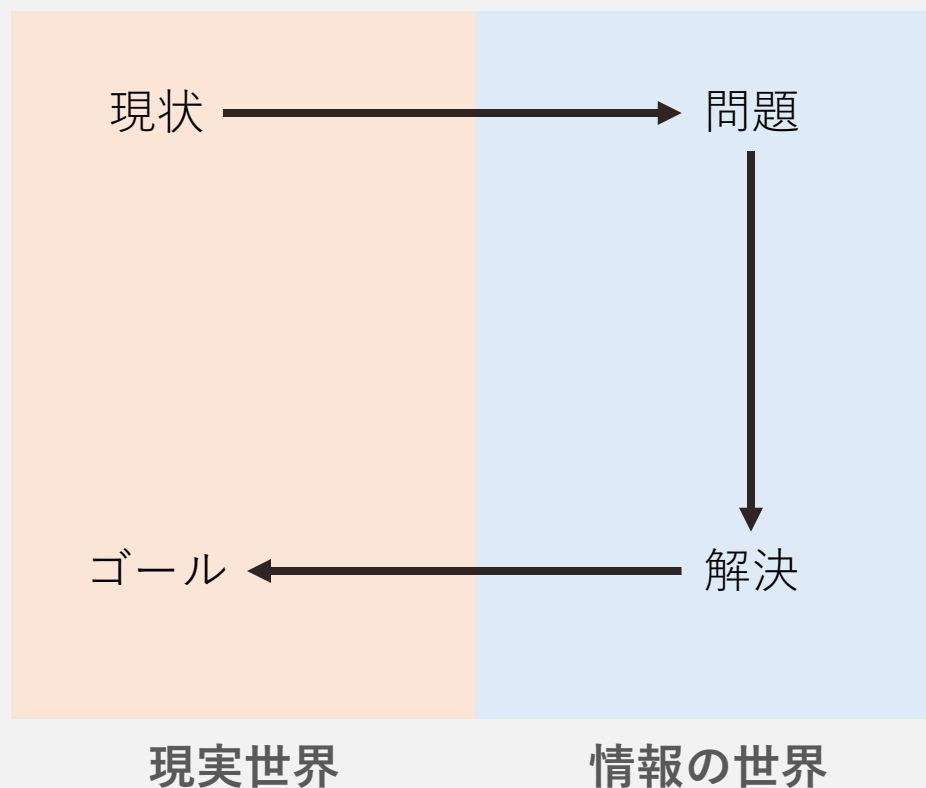
第9回：演習（データ構造とアルゴリズム）

アルゴリズムを使うとき同じ機能でも様々な方式から選ぶ必要がある

1. 既知のアルゴリズムを利用する問題
 - 授業で紹介したアルゴリズムを扱う練習
 - 例示されたデータに対する実際の動作を自分で再現する
 - アルゴリズムから計算量を求める（条件分岐，繰り返し）
2. 既知のアルゴリズムを応用する問題
 - 文章で説明された状況に適切なアルゴリズムを選び，問題を解決する
 - アルゴリズムを組み合わせて問題を解決する練習
3. 未知のアルゴリズムの動作をプログラムから読み解く問題
 - 提示されたアルゴリズムを解読して，例示されたデータの処理を行う（条件分岐，繰り返し，関数）
 - アルゴリズムに基づくプログラムやITシステムの仕組みや使い方が，具体的な操作手順として書かれた難解な記述から理解する練習

アルゴリズムを使って問題を解決するため

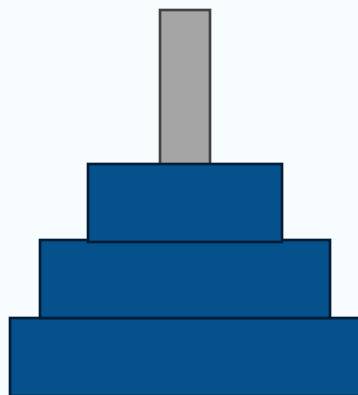
1. 既存アルゴリズムをそのまま使って問題を解決する
2. 頻出アルゴリズムを組み合わせて、状況に応じたアルゴリズムを作る
3. 革新的なアルゴリズムを開発する
 - これまで解決不能だった問題が解ける（高度な専門家の仕事）



アルゴリズムを使うとき同じ機能でも様々な方式から選ぶ必要がある

1. 既知のアルゴリズムを利用する問題
 - 授業で紹介したアルゴリズムを扱う練習
 - 例示されたデータに対する実際の動作を自分で再現する
 - アルゴリズムから計算量を求める（条件分岐，繰り返し）
2. 既知のアルゴリズムを応用する問題
 - 文章で説明された状況に適切なアルゴリズムを選び，問題を解決する
 - アルゴリズムを組み合わせて問題を解決する練習
3. 未知のアルゴリズムの動作をプログラムから読み解く問題
 - 提示されたアルゴリズムを解読して，例示されたデータの処理を行う（条件分岐，繰り返し，関数）
 - アルゴリズムに基づくプログラムやITシステムの仕組みや使い方が，具体的な操作手順として書かれた難解な記述から理解する練習

```
関数 ディスク移動（整数 ディスク数，文字 出発の棒，文字 目的の棒，文字 補助の棒）{  
    もし（ディスク数==1）ならば{  
        出発の棒 から 目的の棒 へ一番上のディスクを移動する  
    }  
    もし（ディスク数≠1）ならば{  
        ディスク移動（ディスク数-1， 出発の棒， 補助の棒， 目的の棒）  
        出発の棒から目的の棒へ一番上のディスクを移動する  
        ディスク移動（ディスク数-1， 補助の棒， 目的の棒， 出発の棒）  
    }  
}
```



A



B



C

```
関数 線形探索(配列 探索対象, 整数 探索値){  
    整数 長さ = 配列の要素数を取得する関数（探索対象）  
    整数 繰返し数 = 0  
    (長さ) 回だけ繰返す{  
        もし ( 配列[繰返し数] が 探したい値 ) と等しいとき{  
            整数 探索値の場所 == 繰返し数  
            プログラムを終了  
        }  
    }  
    繰返しが終了したら, 探したい値はないと判断する  
}
```

探索の開始位置を配列の先頭とする

探索の終了位置を配列の末尾とする

(開始位置 $\leq$ 終了位置) の間, 以下を繰り返す{

探索範囲の中央を計算する. ただし, 中央が少数なら切り捨てる.

中央の位置にある値と探している値を比べる

もし (中央の値 == 探している値) ならば, {

探している値の位置を中央の値として動作終了

}

もし (中央の値 < 探している値) ならば{

終了位置を中央の一つ後ろに変更する

}

もし (中央の値 > 探している値) ならば{

開始位置を中央の一つ後ろに変更する

}

}

繰り返しが終了したら, 探したい値は存在しないと判断する

今回はアルゴリズムのみをみて問題を解いてみてください

- ・ 次回（第10回）の授業で，スライドなどを用いた動きの解説を行います

「未知のアルゴリズムに対する対応」は演習での練習を基本とします

- 本授業は、授業時間内で対応可能な範囲を基本に設計しています
- アルゴリズムを読解できるようになるための時間としては十分ではないですが、
必要になった際の自力習得に必要な前提知識と練習方法を提示します
- エンジニア就職を考える人などは授業外でもアルゴリズムを読解・構築する練習を積むことを推奨します（授業を超えた発展的な質問も対応します）

項番	内容	対象	到達目標
DP1	知識	○	コンピュータシステムに関する基礎的な知識を体系的かつ総合的に身につけている
DP2	技能	◎	コンピュータシステムの分析と設計に関する手法を身につけている
DP3	思考・判断・表現力	○	コンピュータシステムについて、論理的に思考して解決策を探究し、専門的検知から論理的に表現することができる

大学の定める本講義のディプロマポリシー・到達目標（シラバスより）

新・明解Javaで学ぶアルゴリズムとデータ構造

- 著者：柴田 望洋
- SB Creative（2020） 2,500円 + 税
- <https://www.sbcr.jp/product/4815606008/>

世界標準MIT教科書 アルゴリズムイントロダクション 第4版総合版

- 著者：T. コルメン， C. ライザーソン， R. リベスト， C. シュタイン
- 翻訳：浅野 哲夫， 岩野 和生， 梅尾 博司， 小山 透， 山下 雅史，
和田 幸一
- 近代科学社（2024） 18,000円 + 税
- https://www.kindaikagaku.co.jp/book_list/detail/9784764906495/

V. 計算機のハードウェア構成を知る

10. 計算機の中の情報表現

11. 演算とハードウェア構成

- I. 計算機システムを学ぶ意味を見つける
 - 1. 計算機とプログラムによる自動化
 - 2. 計算機，インターネット，情報システム
- II. 計算機やデータ，AIにできることを知る
 - 3. 計算機の基本原理
 - 4. 計算機と統計
- III. 問題を「計算機の使える形」に整理する
 - 5. 情報システムのデザイン
- IV. 計算機の考え方を体感する
 - 6. プログラムの基本
 - 7. データ構造
 - 8. アルゴリズム
 - 9. 演習（データ構造とアルゴリズム）

- V. 計算機のハードウェア構成を知る
 - 10. 計算機の中の情報表現
 - 11. 演算とハードウェア構成
- VI. 計算機特有の数値表現に慣れる
 - 12. 演習（基数と論理演算）
- VII. 計算論的思考
 - 13. 問題解決のためのモデル：PERT図
 - 14. 問題解決のためのモデル：決定表
 - 15. まとめ + 演習（PERT図，決定表）

1. 日本政府や学科の教育方針を知り、現状や将来像に基づいて自分が計算機を学ぶ意味を見つける
2. 計算機やデータ・AIにできることを知り、目的達成に向けた人間と計算機、AIの効果的な分業を想像できるようになる
3. ITシステムの構造や仕組みを学び、問題を計算機が使える形に整理したり、エンジニアに伝えたりできるようになる
4. 計算機のハードウェア構成を知り、**目的達成に必要な装置の見積もりや購入**ができるようになる
5. 計算機に特有のデータ表現を学び、機器設定や開発でしばしば遭遇する数値・記号の意味を理解できるようになる
6. 計算機を使うための思考を計算機と無関係な場面で役立てる計算論的思考を体験する

2025年度 コンピュータシステム

第10回：計算機の中の情報表現

1. 計算機の中の情報表現（基数）

- 10進数, 2進数, 16進数

2. 基数表現の使用場面

- 情報の単位（2進数），カラーコード（16進数）

3. 第9回 演習の解説

- アルゴリズムの動作を理解するときのコツ
- 線形探索，二分探索，ハノイの塔の解説

1. 計算機の中の情報表現（基数）

- 10進数, 2進数, 16進数

2. 基数表現の使用場面

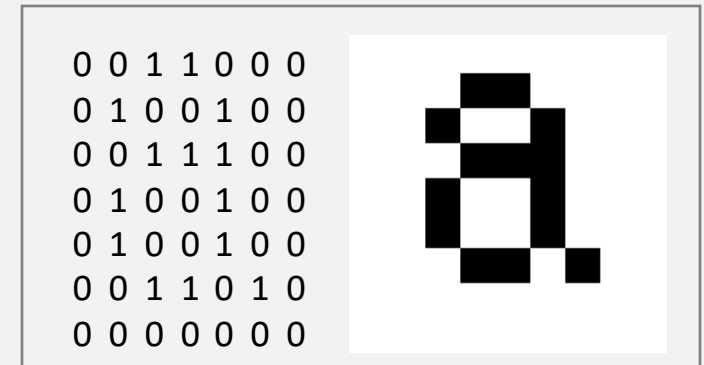
- 情報の単位（2進数），カラーコード（16進数）

3. 第9回 演習の解説

- アルゴリズムの動作を理解するときのコツ
- 線形探索，二分探索，ハノイの塔の解説

文字，画像，音声など様々な情報はすべて数値として表現される

文字	コード	文字	コード	文字	コード	文字	コード
a	97	b	98	c	99	d	100
e	101	f	102	g	103	h	104
i	105	j	106	k	107	l	108
m	109	n	110	o	111	p	112
q	113	r	114	s	115	t	116
u	117	v	118	w	119	x	120
y	121	z	122				

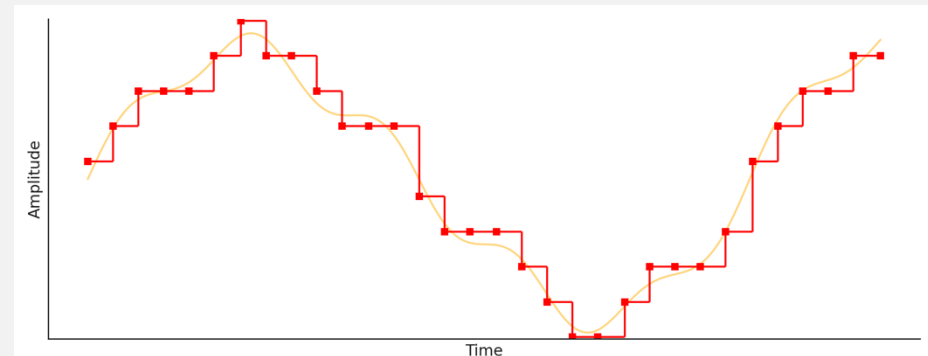


文字コードとフォント

000 030 060 090 120 150 180 210
030 060 090 120 150 180 210 240
060 090 120 150 180 210 240 255
090 120 150 180 210 240 255 225
120 150 180 210 240 255 225 195
150 180 210 240 255 225 195 165
180 210 240 255 225 195 165 135
210 240 255 225 195 165 135 105



画像データ



音声波形の数値化
(後で説明)

ハードウェアの中で物理的に表現される

計算機的设计によって色々な表現が可能

例：ENIACは10進数，アナログ時計は12進数と60進数の併用



そろばん：二・五進法

(+5で一段階進み，2段階進むと桁が上がる)



スイッチ：二進数

(0と1の二つの状態のみ表せる)

現在の電子計算機では、2進数のデジタル表現が標準的

- あらゆる情報（数値）はデジタル化されて離散的に近似される
- あらゆる情報（数値）は2進数で表現されて0と1のみで記述される



ハードウェアの中で物理的に表現される

(計算機的设计によって色々な表現が可能)

例：ENIACは10進数，アナログ時計は12進数と60進数の併用

現在の電子計算機では，2進数のデジタル表現が標準的

- あらゆる情報（数値）はデジタル化されて離散的に近似される
- あらゆる情報（数値）は2進数で表現されて0と1のみで表現される



デジタルとは何か？

2進数とは何か？

パソコンやスマホ，ゲーム機，電卓などは全て「デジタル」な計算機

デジタル：数字式

- digit + al：数字の+方式
- 物理量を**数字**で記述
 - 結び目の数，デジタル温度計，チャットなど

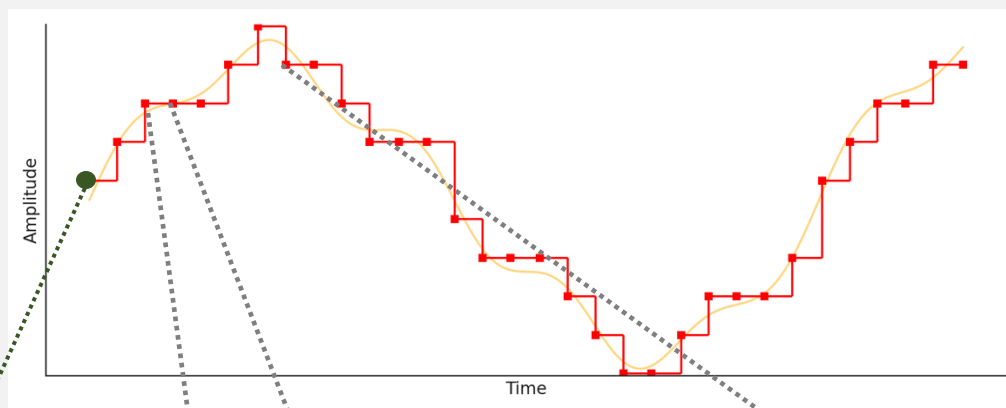
アナログ：相似式

- analog：語源はanalogy（類似・相似）
- 物理量を別の**物理量との対応**で表現
 - 紐の長さ，水銀温度計，糸電話など

デジタルは情報を完全に表現できないが，ノイズに強くて伝達しやすい

- 量を正しく表すには無限桁の数値が必要
- 数値は量よりも“伝達ミス”が起きにくい
 - 例1) 水銀温度計の結果を“正しく”伝えることは難しいが，数値を伝えるのは簡単
 - 例2) 紐の長さを正確に伝えることは難しいが，結び目の数を伝えるのは簡単

縦横に適当な間隔で区切り，棒状にすることで波形を数列に変換



Sample 1	Sample 2	Sample 3	Sample 4	Sample 5	Sample 6	Sample 7	Sample 8	Sample 9
0.143	0.429	0.714	0.714	0.714	1	1.286	1	1

ハードウェアの中で物理的に表現される

(計算機的设计によって色々な表現が可能)

例：ENIACは10進数，アナログ時計は12進数と60進数の併用

現在の電子計算機では，2進数のデジタル表現が標準的

- あらゆる情報（数値）はデジタル化されて離散的に近似される
- あらゆる情報（数値）は2進数で表現されて0と1のみで表現される



デジタル：情報を数値で表現すること

2進数とは何か？

通常の計算機は情報（=数値）を"0"と"1"で表現する

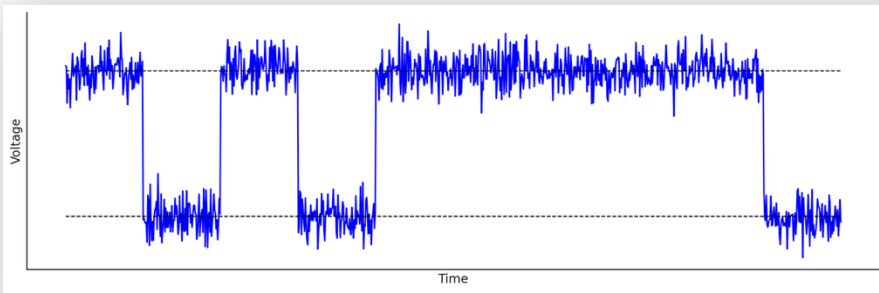
10進数：10種類の数字を用いて、10ごとに桁が上がる

2進数：2種類の数字を用いて、2ごとに桁が上がる

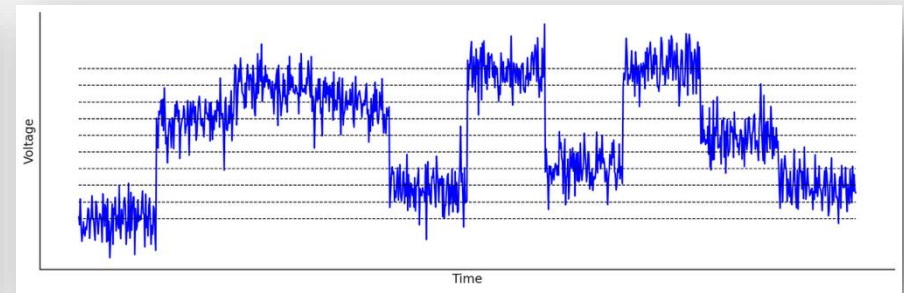
10進数	0	1	2	3	4	5	6	7	8	9	10	11
2進数	0	1	10	11	100	101	110	111	1000	1001	1010	1011

2進数は機械の仕組みとして表現しやすい

- スイッチ：オン(1), オフ(0)
- 電圧：高い(1), 低い(0)



電圧による数値表現のイメージ
(2進数)



電圧による数値表現のイメージ
(10進数)

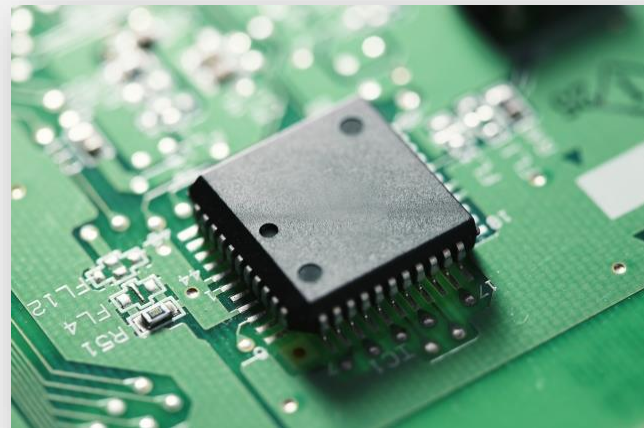
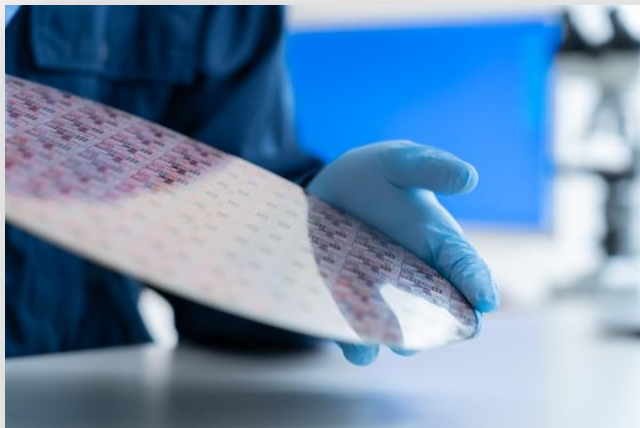
微細な電圧スイッチで0と1を表現

半導体：導体（電気を通す）と絶縁体（通さない）の中間的な物質

- 電気抵抗が導体と絶縁体の中間程度の物質
 - 例：シリコン，ゲルマニウム
- 不純物や外部刺激の有無によって，電気を通したり通さなかったりする
→ スイッチとして0/1を表現可能

ICチップ：大量のスイッチを並べた装置（集積回路，Integrated Circuit）

- 微細な半導体装置を大量に並べたデバイス
- 各半導体装置は電気信号の「遮断 or 通過」で「0 or 1」を表現



左：シリコン単結晶のインゴット

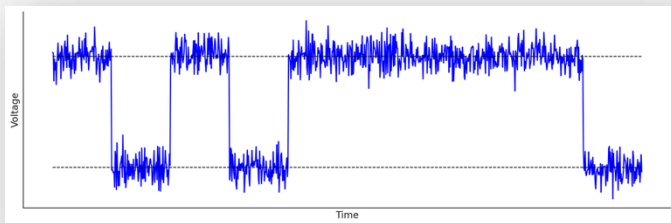
現在の電子計算機では、2進数のデジタル表現が標準的

- あらゆる情報（数値）はデジタル化されて離散的に近似される
- あらゆる情報（数値）は2進数で表現されて0と1のみで記述される

デジタル：情報を数値で表現する方法，保持・伝達しやすい

2進数：2で桁の上がる数値表現，機械で表現しやすい

10進数	0	1	2	3	4	5	6	7	8	9	10	11
2進数	0	1	10	11	100	101	110	111	1000	1001	1010	1011



文字の種類

10進数は10種類：{0,1,2,3,4,5,6,7,8,9}

2進数は2種類：{0,1}

桁上がり

10進数：桁が10に到達したら（対応文字がなくなるので）繰り上がる

...→8→9→10→11→12

2進数：桁が2に到達したら繰り上がる

...→0→1→10→11→100→...

数値の大きさ

10進数：末尾は1の数(10^0), その上は10の数(10^1), その上は100の数(10^2)

例) $3282_{(10)} = 3 \times 10^3 + 2 \times 10^2 + 8 \times 10 + 2 \times 1$

2進数：末尾は1の数(2^0), その上は2の数(2^1), その上は4の数(2^2)

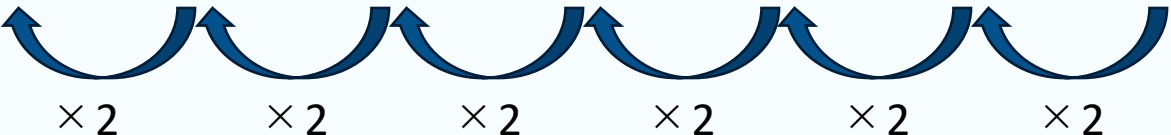
例) $1011_{(2)} = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2 + 1 \times 1$

※本講義では、2進数は添字に(2)をつけて、10進数は添字に(10)をつけて表記する

桁に応じた2の冪乗を掛け算する

$$\begin{aligned}\text{例) } 1001111_{(2)} &= 1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\ &= 64 + 0 + 0 + 8 + 4 + 2 + 1 \\ &= 79_{(10)}\end{aligned}$$

数値	1	0	0	1	1	1	1
桁	2^6	2^5	2^4	2^3	2^2	2^1	2^0
大きさ	64	32	16	8	4	2	1



10進数→2進数変換

ステップバイステップで学ぼう！

10進数を入力： 変換開始

12 (10進数) = 1100 (2進数)

1 1 0 0

変換手順

- 12 を2で割ります
 $12 \div 2 = 6$ 余り 0
- 商の6 を2で割ります
 $6 \div 2 = 3$ 余り 0
- 商の3 を2で割ります
 $3 \div 2 = 1$ 余り 1
- 商の1 を2で割ります
 $1 \div 2 = 0$ 余り 1
- 余りを下から上を読み上げると2進数になります
余り：1100 ← これが答え！

変換方法の説明

Made with Claude

Artifacts are user-generated and may contain unverified or potentially unsafe content.

Report Customize Artifact

<https://claude.ai/public/artifacts/0dfe2ebd-3e1a-45ab-95c1-c3c7174ea2ce>

アプリに不具合や間違いがあれば講師までご連絡ください

桁数の多い2進数を簡単な仕組みでコンパクトに表記する方法

2進数の4桁分を一つの記号で表現する

- $2^4=16$ なので、16種類の記号がある→16進数

10進数	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
2進数	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
16進数	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

2進数と16進数の変換：2進数を4桁に区切り、対応する記号に変える

- 例1) $10101100_{(2)} \rightarrow 1010 \mid 1100 \rightarrow A \mid E \rightarrow AE_{(16)}$
- 例2) $5F_{(16)} \rightarrow 5 \mid F \rightarrow 0101 \mid 1111 \rightarrow 101111_{(2)}$

2進数 ⇔ 16進数 変換

変換ツール

☒ 2進数 ☐ 16進数

例: 1010 または 10101110

1010 | 1110

↓

A | E

対応表

10進数	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
2進数	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
16進数	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

変換ルール

基本原理

- 2進数の4桁が16進数の1桁に対応します
- これは $2^4 = 16$ であるに基づいています
- 0000~1111の16通りが、0~Fの16文字に対応します

2進数から16進数への変換

Made with Claude © Artifacts are user-generated and may contain unverified or potentially unsafe content. Report Customize Artifact

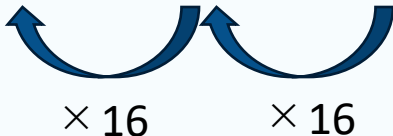
<https://claude.ai/public/artifacts/aba4536b-0ce9-4646-8732-47ea2cd625f9>

アプリに不具合や間違いがあれば講師までご連絡ください

桁に応じた16の冪乗を掛け算する

$$\begin{aligned}\text{例) } 1F3_{(16)} &= 1 \times 16^2 + F \times 16 + 3 \times 1 \\ &= 1 \times 256 + 15 \times 16 + 3 \times 1 \\ &= 499_{(10)}\end{aligned}$$

数値	1	1	1
桁	16^2	16^1	16^0
大きさ	256	16	1





<https://claude.ai/public/artifacts/af79b8ae-17da-4597-8c2e-477c036c482f>

アプリに不具合や間違いがあれば講師までご連絡ください

1. 計算機の中の情報表現（基数）

- 10進数, 2進数, 16進数

2. 基数表現の使用場面

- 情報の単位（2進数），カラーコード（16進数）

3. 第9回 演習の解説

- アルゴリズムの動作を理解するときのコツ
- 線形探索，二分探索，ハノイの塔の解説

値 (バイト数)	2進接頭辞	記号	SI風表記 (慣例)	SI記号 (慣例)
2^{10}	キビバイト	KiB	キロバイト	KB
2^{20}	メビバイト	MiB	メガバイト	MB
2^{30}	ギビバイト	GiB	ギガバイト	GB
2^{40}	テビバイト	TiB	テラバイト	TB
2^{50}	ペビバイト	PiB	ペタバイト	PB

※厳密には，SI記号で表記することは混同を招くため推奨されないが，慣例的には多く用いられている

CPU（A16チップ）

- CPUのクロック周波数（SI表記）
 - 高性能コア（2コア）：3.46GHz
 - 高効率コア（4コア）：2.02GHz
- （画像処理用のGPUとAI処理用のNeural Englinもあるがここでは省略）

メモリ（RAM）

- 8GB（2進接頭辞）

ストレージ

- 128GB（2進接頭辞）
- 256GB
- 512GB

項目	通常表記	2進乗乗表記	10進表記	Gの意味
CPU Pコア最大周波数	4.0 GHz	-	4,000,000,000 Hz	$G = 10^9$ （10進）
CPU Eコア最大周波数	2.2 GHz	-	2,200,000,000 Hz	$G = 10^9$ （10進）
メモリ (RAM)	8 GB	$2^{33} B$	8,589,934,592 B	$G = 2^{30}$ （2進）
ストレージ	128 GB	$2^{37} B$	137,438,953,472 B	$G = 2^{30}$ （2進）
ストレージ	256 GB	$2^{38} B$	274,877,906,944 B	$G = 2^{30}$ （2進）
ストレージ	512 GB	$2^{39} B$	549,755,813,888 B	$G = 2^{30}$ （2進）



<https://claude.ai/public/artifacts/41aaac6d-a888-40be-8020-3441e40fc942>

アプリに不具合や間違いがあれば講師までご連絡ください

1. 計算機の中の情報表現（基数）

- 10進数, 2進数, 16進数

2. 基数表現の使用場面

- 情報の単位（2進数），カラーコード（16進数）

3. 第9回 演習の解説

- アルゴリズムの動作を理解するときのコツ
- 線形探索，二分探索，ハノイの塔の解説

前回示したアルゴリズムは、比較的単純な動きをする

記述は複雑に見える

- 具体的な操作が細かく厳密に書かれているので全体像が見えにくい
→ 図やアニメーションにして全体像を概観すると手順を追やすい
- 変数や繰り返し、関数で記述が圧縮されているため手順を見失いやすい
→ 変数や繰り返し、関数を具体的な手順に展開すると分かりやすい

操作自体はシンプルなルールに基づく

- プログラムの分量は少ない（≡規則としては単純に書けている）
- 手を動かして確認すると直感的に理解しやすい
 - 例1) インターネットで図解や動画、動作を確認できるアプリを調べる
 - 例2) 紙に手順を1ステップごとの状態を絵に描いて確認する
 - 例3) 繰り返しや関数の部分を展開して、上から下への逐次的な手順として書き下す
（必要に応じて、変数に具体的な値を入れるとより分かりやすい）

```
関数 線形探索(配列 探索対象, 整数 探索値){  
    整数 長さ = 配列の要素数を取得する関数 (探索対象)  
    整数 繰返し数 = 0  
    (長さ) 回だけ繰返す{  
        もし ( 配列[繰返し数] が 探したい値 ) と等しいとき{  
            整数 探索値の場所 == 繰返し数  
            プログラムを終了  
        }  
    }  
    繰返しが終了したら, 探したい値はないと判断する  
}
```



<https://claude.ai/public/artifacts/d07b7208-44e6-4d47-8f54-079161fe30eb>

※PC・タブレット推奨，スマホはレイアウトが崩れる可能性が高いです

探索の開始位置を配列の先頭とする

探索の終了位置を配列の末尾とする

（開始位置 $\leq$ 終了位置）の間，以下を繰り返す{

探索範囲の中央を計算する。ただし，中央が少数なら切り捨てる。

中央の位置にある値と探している値を比べる

もし（中央の値 == 探している値）ならば，{

探している値の位置を中央の値として動作終了

}

もし（中央の値 < 探している値）ならば{

終了位置を中央の一つ後ろに変更する

}

もし（中央の値 > 探している値）ならば{

開始位置を中央の一つ後ろに変更する

}

}

繰り返しが終了したら，探したい値は存在しないと判断する

⚡ 二分探索アルゴリズム視覚化

探索する値:
7 ランダム選択

配列の値 (カンマ区切り):
3,1,7,9,2,5,10,15,8,12,4,6,11,14,13 配列更新 ランダム生成

開始位置 終了位置 中央位置 探索範囲 範囲外

[0] [1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14]

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

開始 終了

探索範囲: [0] ~ [14] | 中央位置: [7] | 中央の値: 8 | 探索値: 7

中央値 8 > 7: 左半分を探索

探索開始 ← 1つ戻す 1つ進める → リセット

現在の状態: ステップ 1: 中央値 8 が探索値より大きいため、左半分を探索します。

二分探索の疑似コード

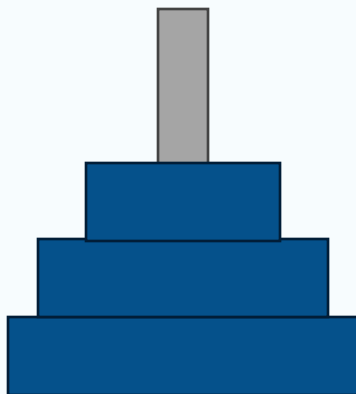
Made with Claude

Artifacts are user-generated and may contain unverified or potentially unsafe content. Report Customize Artifact

<https://claude.ai/public/artifacts/4ac68507-ef42-4c2e-9a05-9af4fa555d5e>

※PC・タブレット推奨，スマホはレイアウトが崩れる可能性が高いです

```
関数 ディスク移動（整数 ディスク数，文字 出発の棒，文字 目的の棒，文字 補助の棒）{  
    もし（ディスク数==1）ならば{  
        出発の棒 から 目的の棒 へ一番上のディスクを移動する  
    }  
    もし（ディスク数≠1）ならば{  
        ディスク移動（ディスク数-1， 出発の棒， 補助の棒， 目的の棒）  
        出発の棒から目的の棒へ一番上のディスクを移動する  
        ディスク移動（ディスク数-1， 補助の棒， 目的の棒， 出発の棒）  
    }  
}
```



A



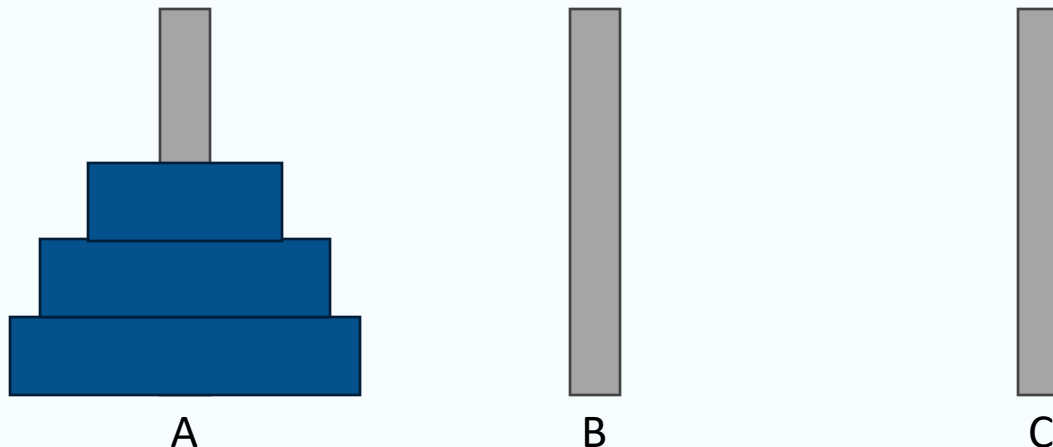
B



C

```
関数 ディスク移動（整数 ディスク数，文字 出発の棒，文字 目的の棒，文字 補助の棒）{  
    もし（ディスク数==1）ならば{  
        出発の棒 から 目的の棒 へ一番上のディスクを移動する  
    }  
    もし（ディスク数≠1）ならば{  
        ディスク移動（ディスク数-1， 出発の棒， 補助の棒， 目的の棒）  
        出発の棒から目的の棒へ一番上のディスクを移動する  
        ディスク移動（ディスク数-1， 補助の棒， 目的の棒， 出発の棒）  
    }  
}
```

関数の中で関数自身が呼び出されている（再帰）





<https://claude.ai/public/artifacts/8262c434-ed2f-4991-8bb8-e1cb54b8022f>

※PC・タブレット推奨, スマホはレイアウトが崩れる可能性が高いです
小ウィンドウの「最大ディスク移動」は「一番上のディスクを移動」の誤植です

2025年度 コンピュータシステム 第11回：演算とハードウェア構成

1. コンピュータのハードウェア構成

- CPU, メモリ, ストレージ

2. 計算機を行う演算

- 二進数に対する操作（論理演算）
- 四則演算の実装

3. 身近に遭遇するかもしれない2進数に由来する現象

- アプリケーションの操作
- パソコンの設定項目
- 計算誤差

コンピュータ（計算機）

Compute + er : 計算する人・もの

計算

情報を決められた方法で処理すること(演算), その結果を求めること(算出)

例1) 四則演算

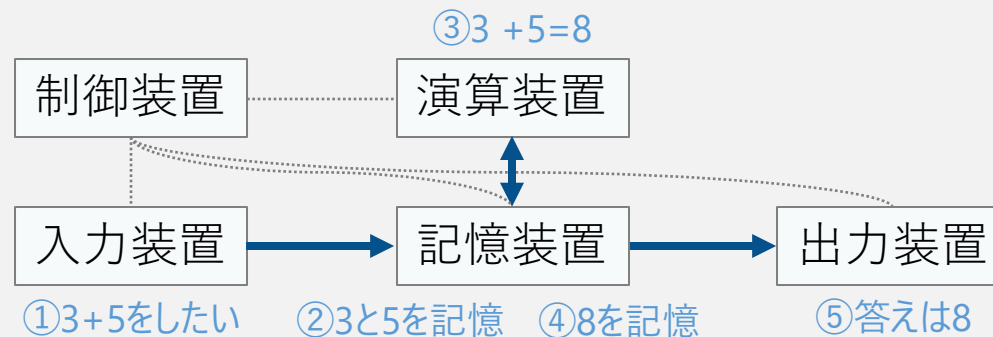
- $3+5=8$, $10-3=7$, $4\times 8=32$, $7\div 2=3.5$ (3あまり1)

例2) 数学の計算問題

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi} \quad x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x - a)^n$$

自動で計算をする装置は次の仕組みで実現可能

- 入力装置：数値や計算方法を入力する装置
- 記憶装置：演算過程の情報を一時的に保持
- 演算装置：足し算や掛け算などを行う回路
- 出力装置：計算結果を出力する装置
- 制御装置：計算を行うために装置同士を連携



自動で計算をする装置は図に示すようなハードウェア構成で実現できる

まず、計算をするは入力装置が必要。例えば、キーボードのようなもの

そして、結果を出力する装置が必要。例えば、ディスプレイのようなもの

当然、数値同士の足し算や掛け算など、演算を行う装置が必要。

また、筆算をするときに書いておかないと分からなくなるように、計算過程の数値を覚えておくためのメモのような記憶装置が必要

最後に、これらがバラバラにあっても自動で計算ができることにはならないので、これら4つの装置を連携させて動作させるための制御装置が必要

これら5つの装置があれば、「計算」を自動実行する機械を作ることができる

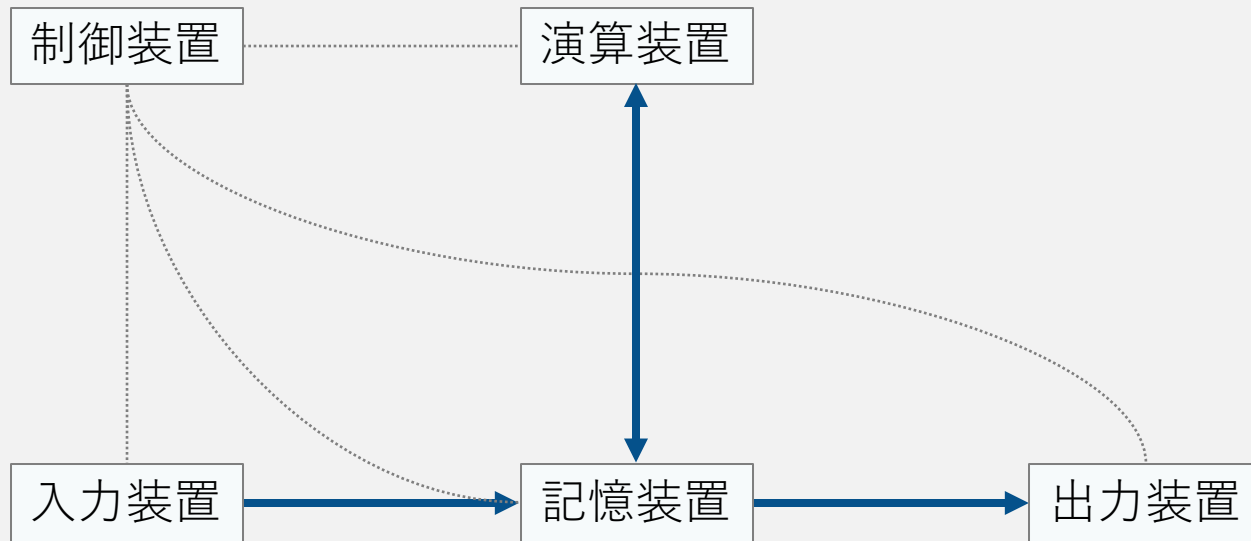
例えば、 $3+5$ という演算を考える

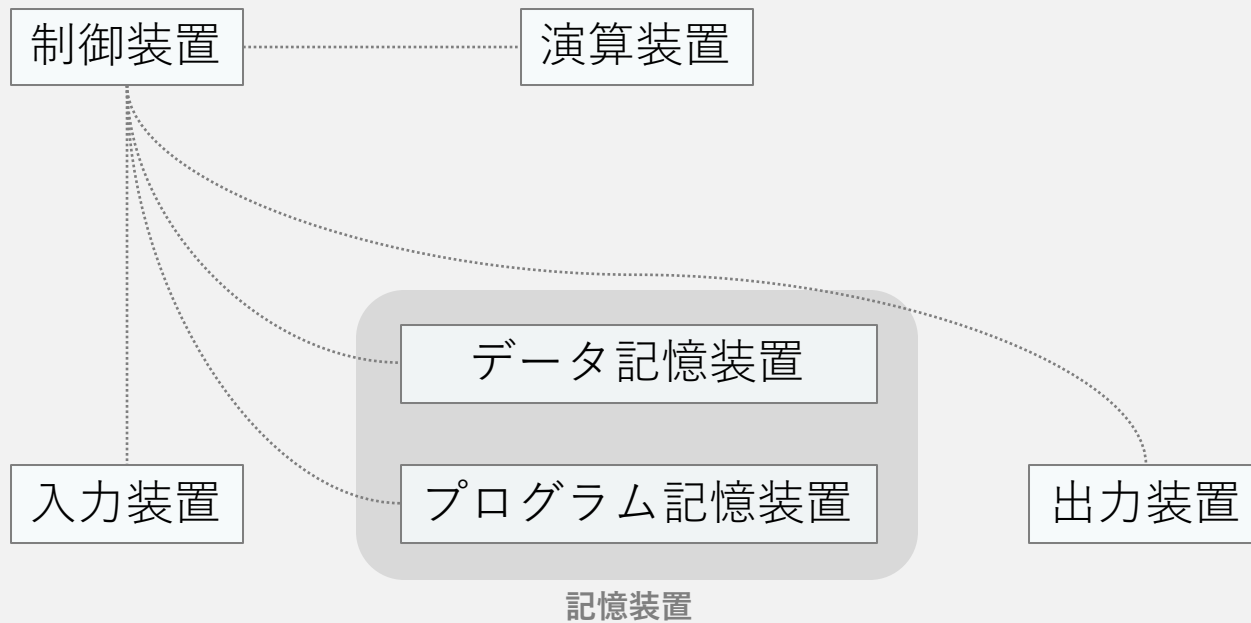
1. 入力装置で数値と行いたい演算を入力する
2. 記憶装置に入力情報が記憶される
3. 記憶した情報に基づいて、演算装置で足し算されて、その結果が導出される
4. 導出された結果は記憶装置に記憶される
5. 最後に、その結果がディスプレイから出力される

制御装置はこれら一連の動作を自動で取り持ってくれる

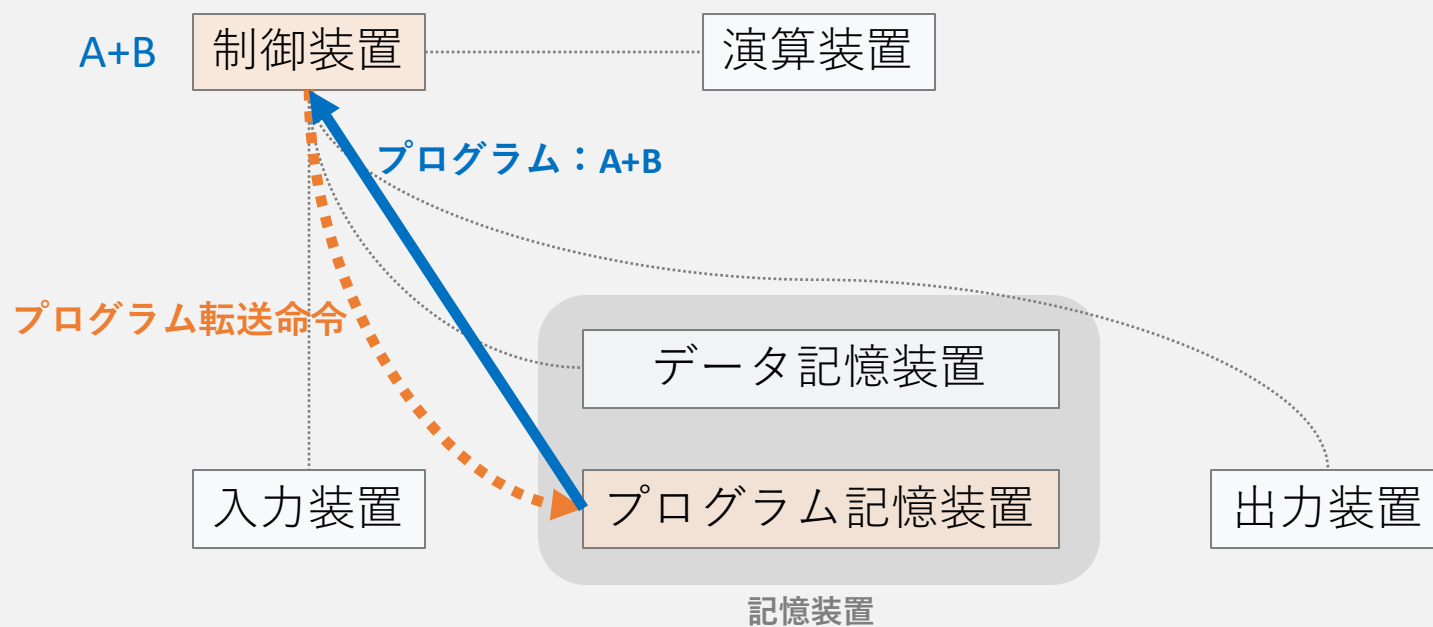
このように、5つの装置があれば計算は人間が介在することなく自動で実行される。

これまでをまとめると、計算機とはこのようなもので、「自動で計算する機械」である。そして、これがあれば色々な仕事ができる

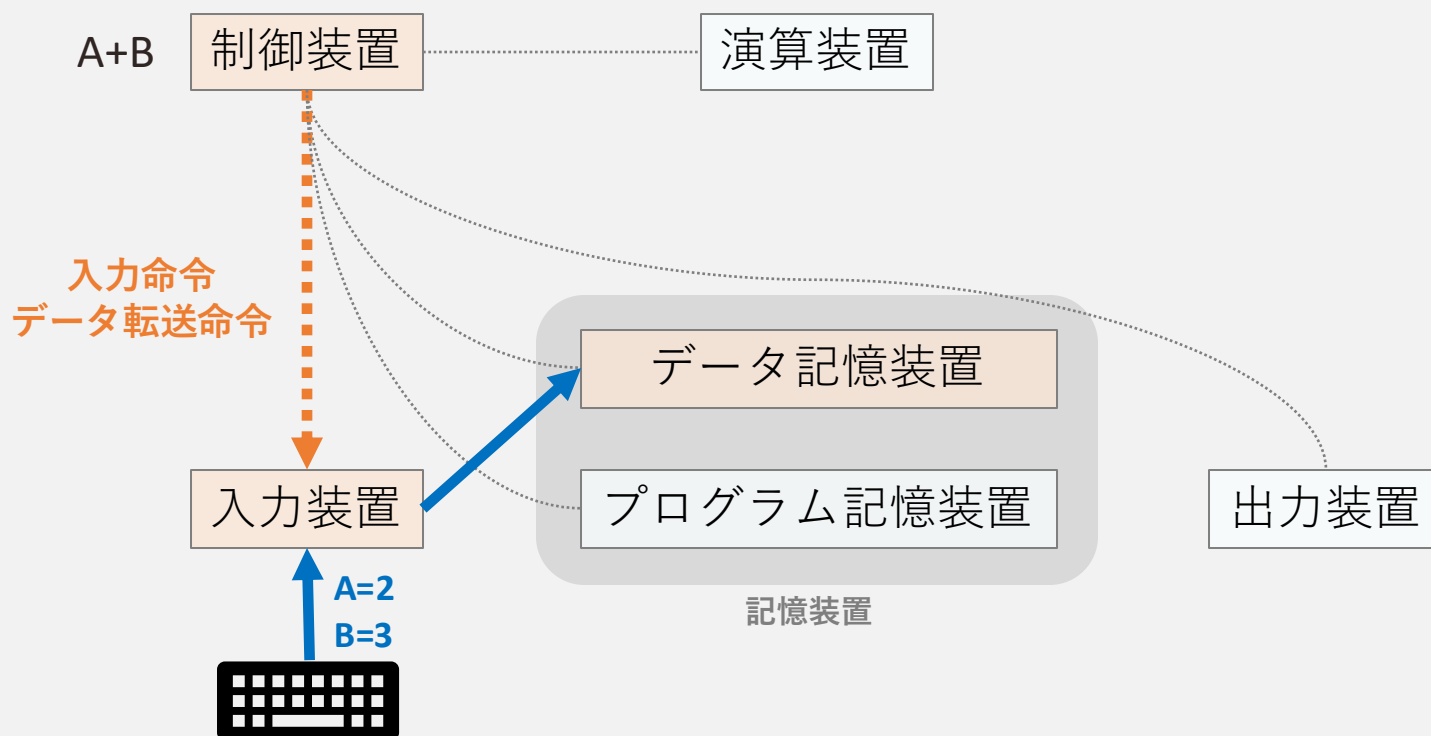




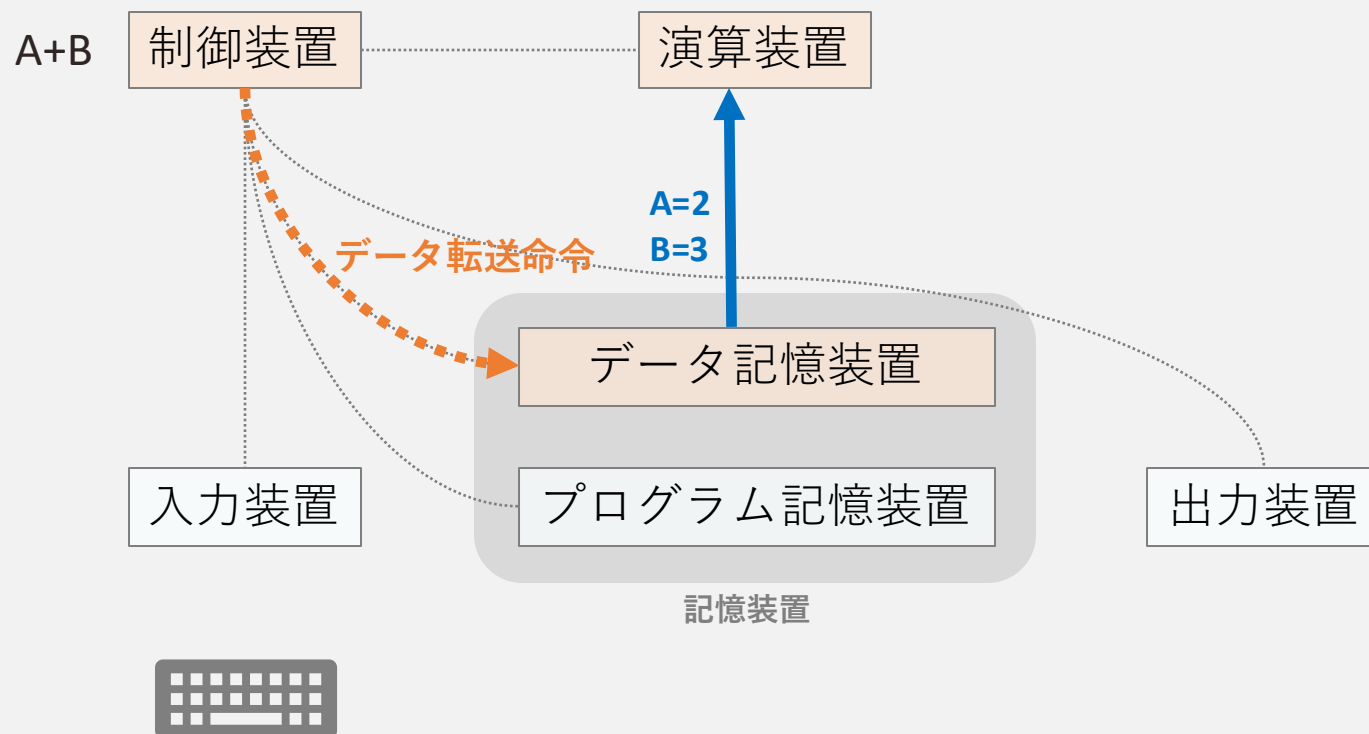
- ①事前準備：プログラム転送命令
- ②入力受付：入力命令，データ転送命令
- ③演算実行：演算命令，データ転送命令
- ④結果出力：データ転送命令，出力命令



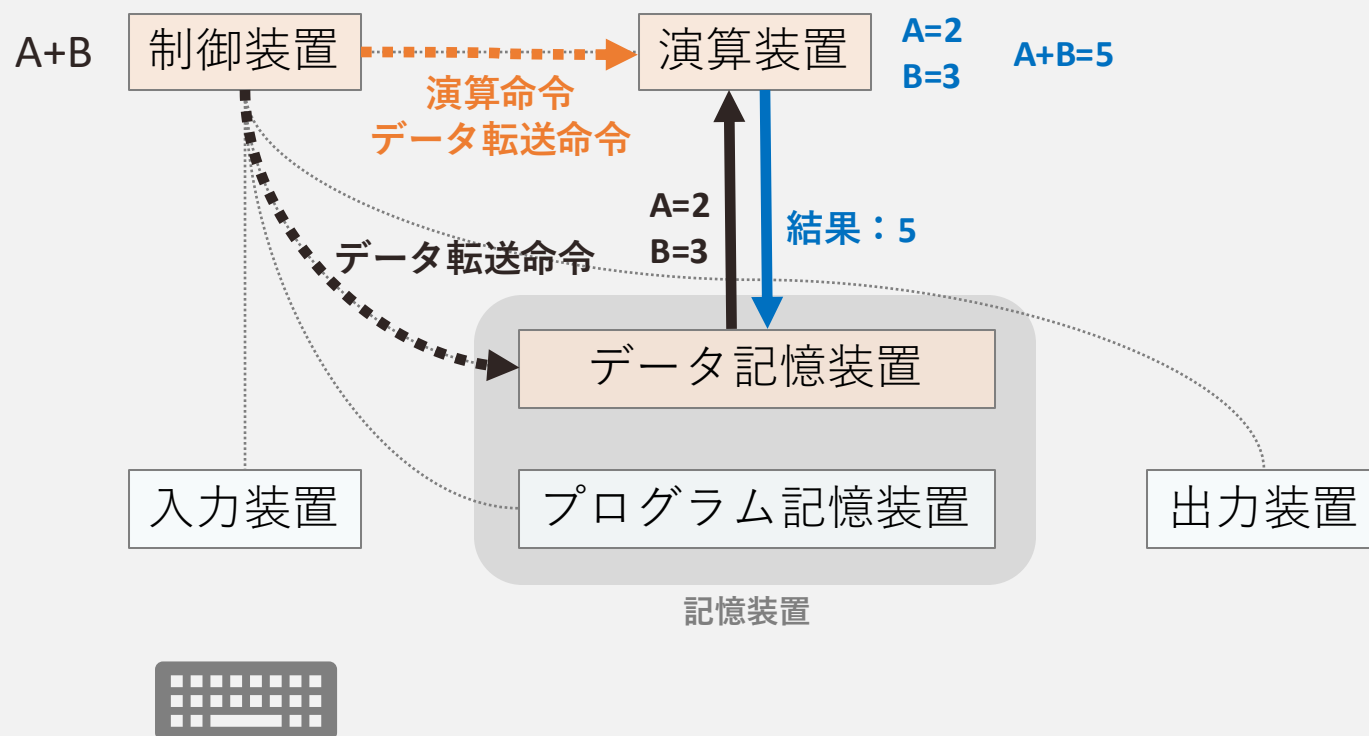
- ①事前準備：プログラム転送命令
- ②入力受付：入力命令，データ転送命令
- ③演算実行：演算命令，データ転送命令
- ④結果出力：データ転送命令，出力命令



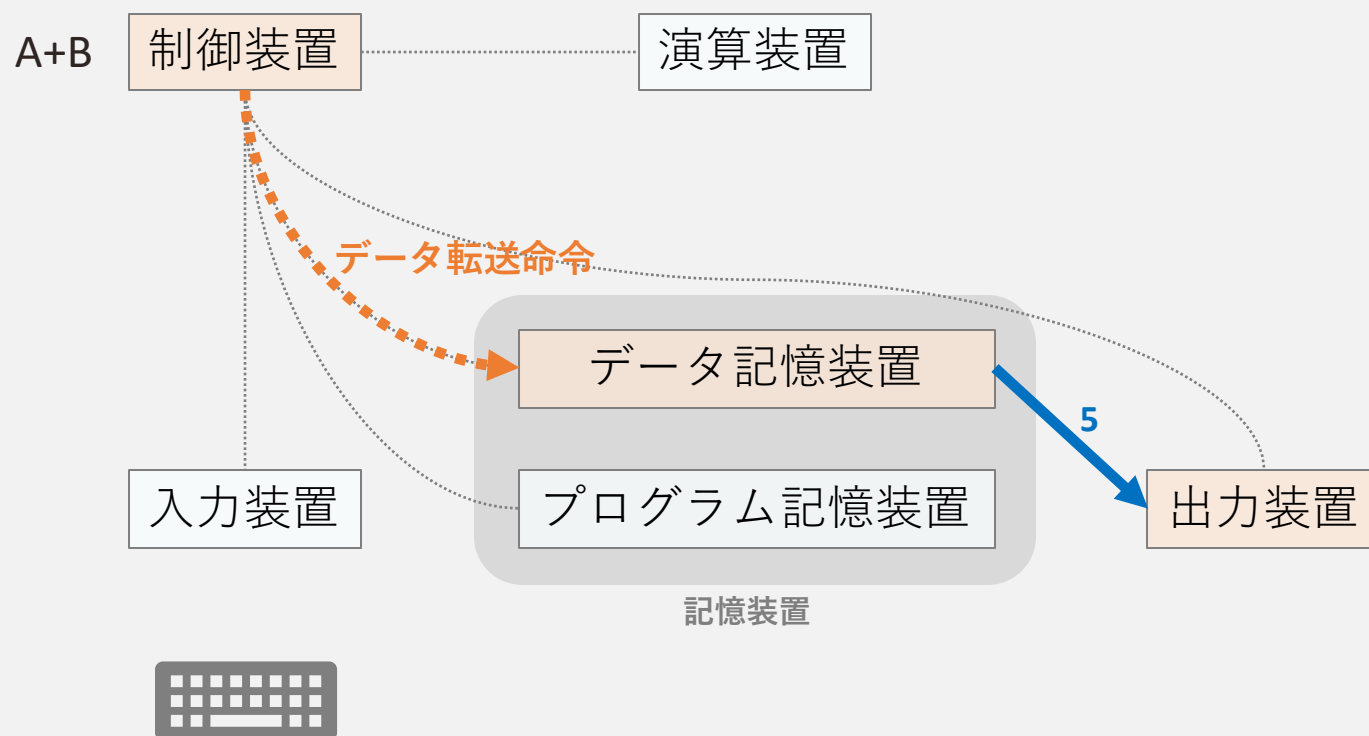
- ①事前準備：プログラム転送命令
- ②入力受付：入力命令，データ転送命令
- ③演算実行：演算命令，データ転送命令
- ④結果出力：データ転送命令，出力命令



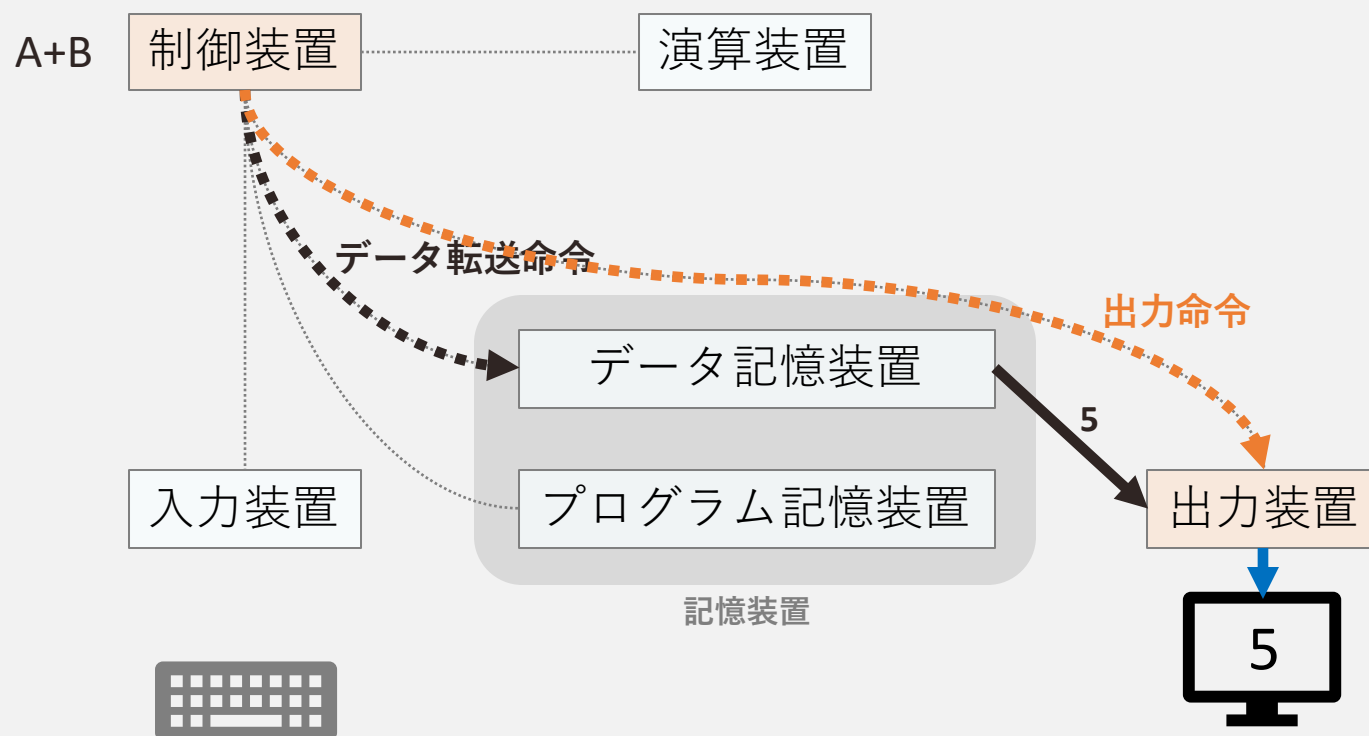
- ①事前準備：プログラム転送命令
- ②入力受付：入力命令，データ転送命令
- ③演算実行：演算命令，データ転送命令
- ④結果出力：データ転送命令，出力命令



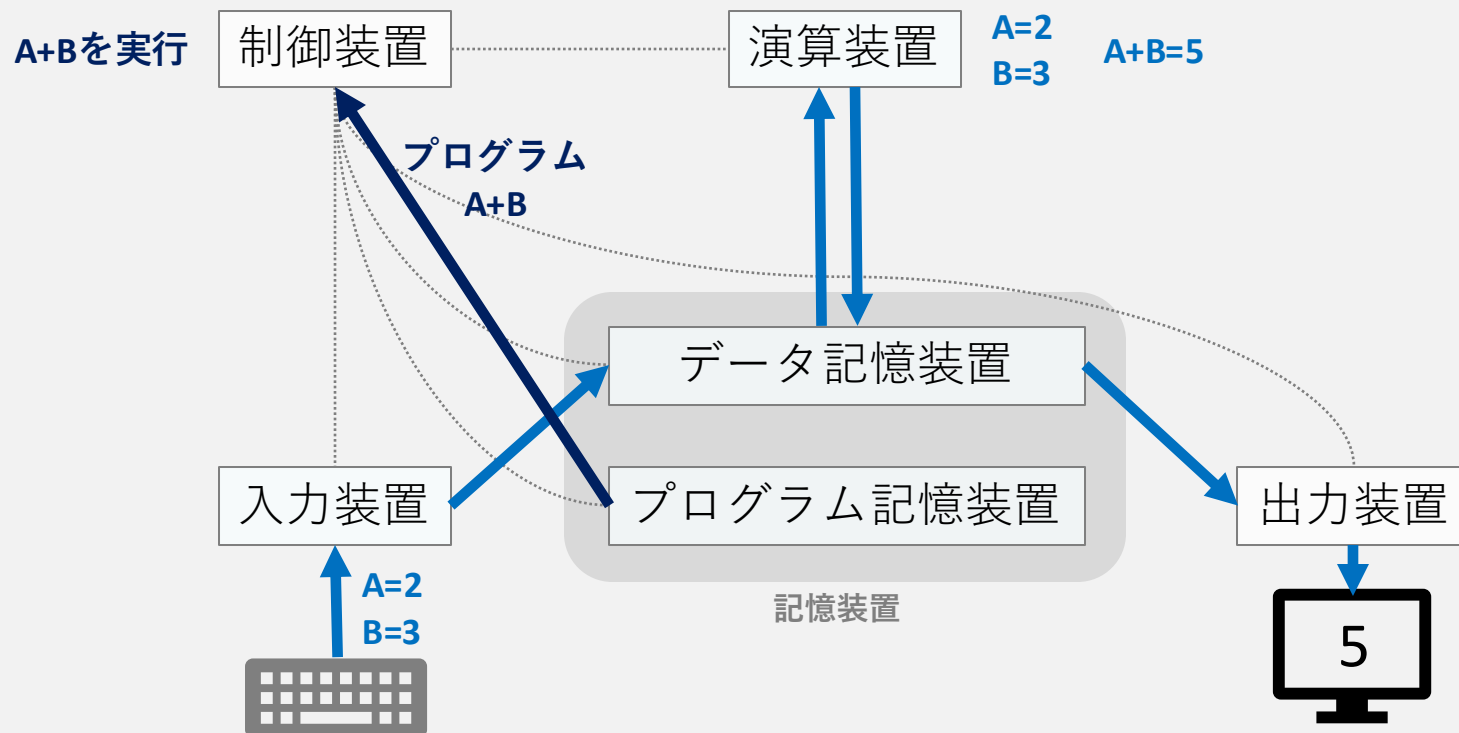
- ①事前準備：プログラム転送命令
- ②入力受付：入力命令，データ転送命令
- ③演算実行：演算命令，データ転送命令
- ④結果出力：データ転送命令，出力命令



- ①事前準備：プログラム転送命令
- ②入力受付：入力命令，データ転送命令
- ③演算実行：演算命令，データ転送命令
- ④結果出力：データ転送命令，出力命令

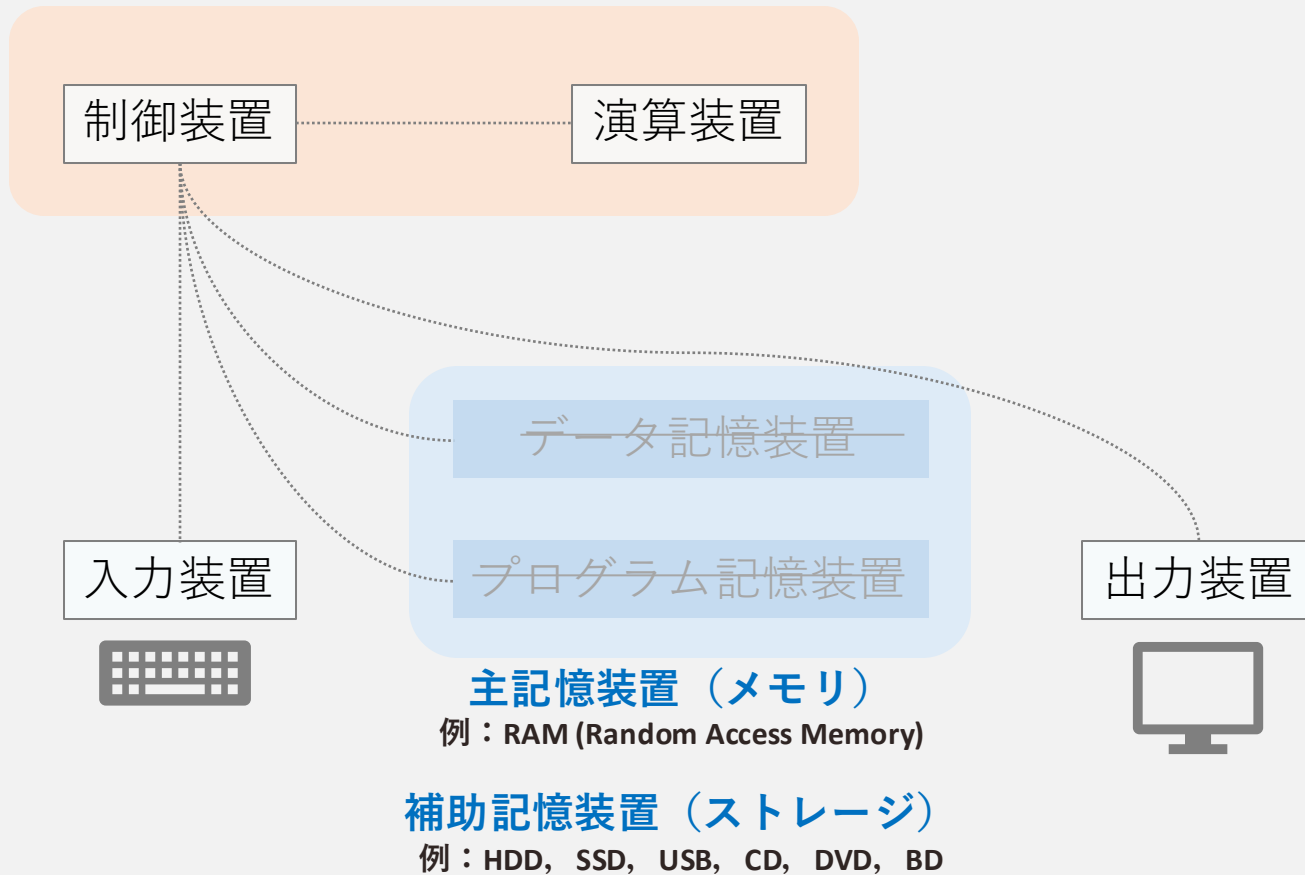


- ①事前準備：プログラム転送命令
- ②入力受付：入力命令，データ転送命令
- ③演算実行：演算命令，データ転送命令
- ④結果出力：データ転送命令，出力命令



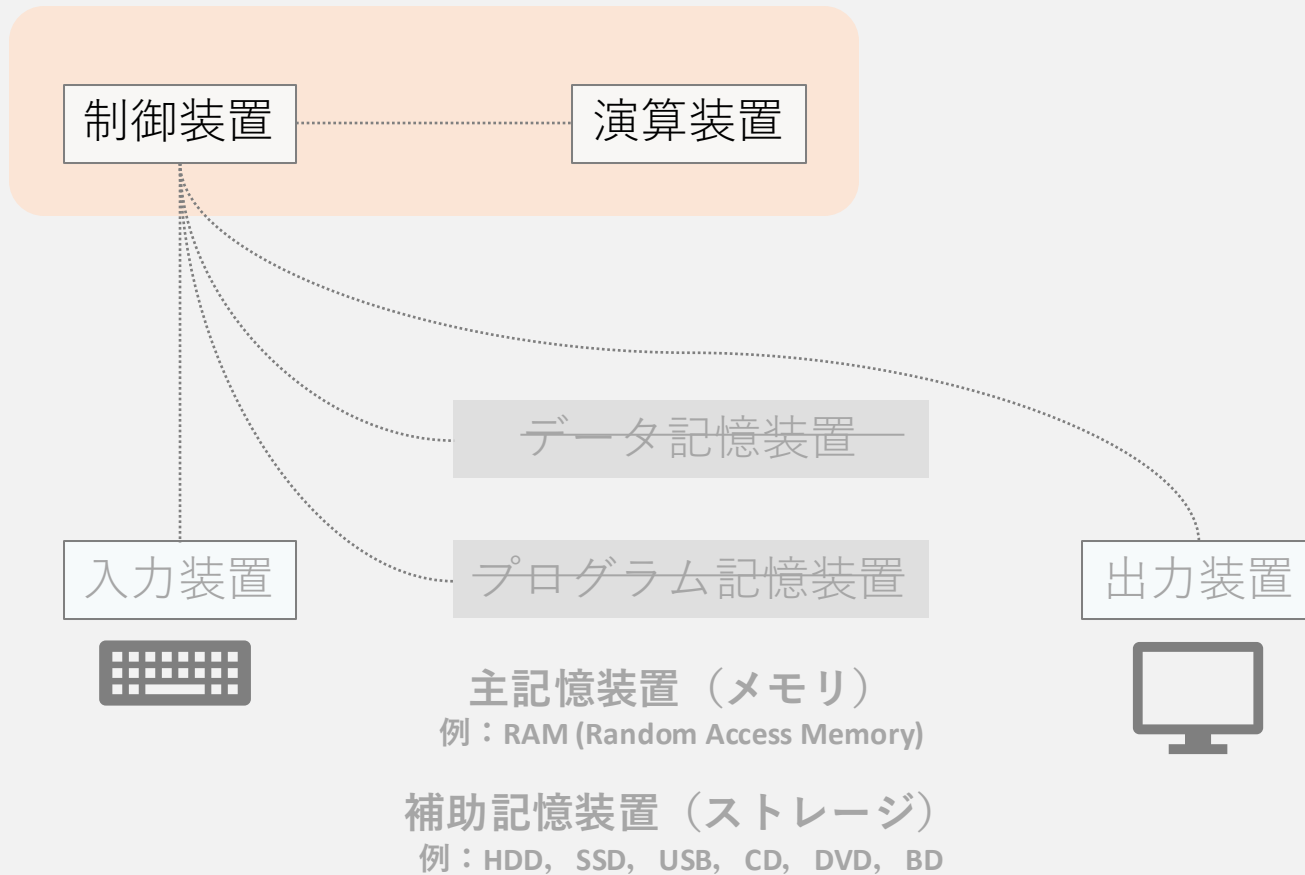
中央処理装置（CPU, Central Processing Unit）

例：Intel Core i5, Apple M1, Ryzen5 2500U, Snapdragon S2



中央処理装置（CPU, Central Processing Unit）

例：Intel Core i5, Apple M1, Ryzen5 2500U, Snapdragon S2



演算や計算機全体の制御を行う，中心的な役割を担う装置

クロック数/クロック周波数

- ・ 一秒間に行える計算（処理できる命令）の数
- ・ 2025年現在では1GHz（ $=10^9\text{Hz}$ → 10億回/秒）以上のものが多い

コア数

- ・ 一度に並列して動作する演算装置の数
- ・ 多様なコアを搭載して使い分ける装置も多い
 - ・ 消費電力の少ないコア（待機中など），高速なコア（ゲーム時など）

スレッド数

- ・ 一度に並列して処理できる計算の数
- ・ 各コアに余力がある場合は，コア数を超えて並列動作できることがある

画像処理装置（GPU, Graphic Processing Unit）

画像描画のように複雑ではないが、並列的で高速さが求められる処理を効率的に実行するための装置

- CPU：桁の多い数の演算を、次々と高速に処理できる
 - GPU：桁の少ない数の演算を、同時並行で処理できる
- 比喩）100マス計算は、大人10人より、小学生100人の方が速い

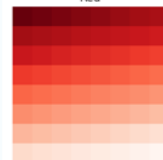
000 030 060 090 120 150 180 210
030 060 090 120 150 180 210 240
060 090 120 150 180 210 240 255
090 120 150 180 210 240 255 225
120 150 180 210 240 255 225 195
150 180 210 240 255 225 195 165
180 210 240 255 225 195 165 135
210 240 255 225 195 165 135 105



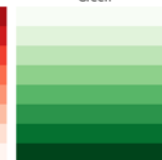
255 250 245 240 235 230 225 220 000 000 000 000 000 000 000 755 192 150 130 130 150 192 255
225 220 215 210 205 200 195 190 036 036 036 036 036 036 036 192 130 088 067 067 088 130 192
195 190 185 180 175 170 165 160 072 072 072 072 072 072 072 150 088 046 026 026 046 088 150
165 160 155 150 145 140 135 130 109 109 109 109 109 109 109 130 067 026 005 005 026 067 130
135 130 125 120 115 110 105 100 145 145 145 145 145 145 145 130 067 026 005 005 026 067 130
105 100 095 090 085 080 075 070 182 182 182 182 182 182 182 150 088 046 026 026 046 088 150
075 070 065 060 055 050 045 040 218 218 218 218 218 218 218 192 130 088 067 067 088 130 192
045 040 035 030 025 020 015 010 255 255 255 255 255 255 255 755 192 150 130 130 150 192 255



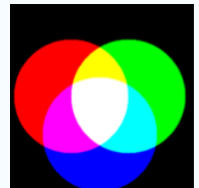
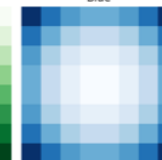
Red



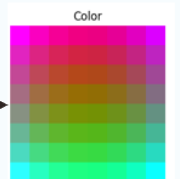
Green



Blue



光の三原色



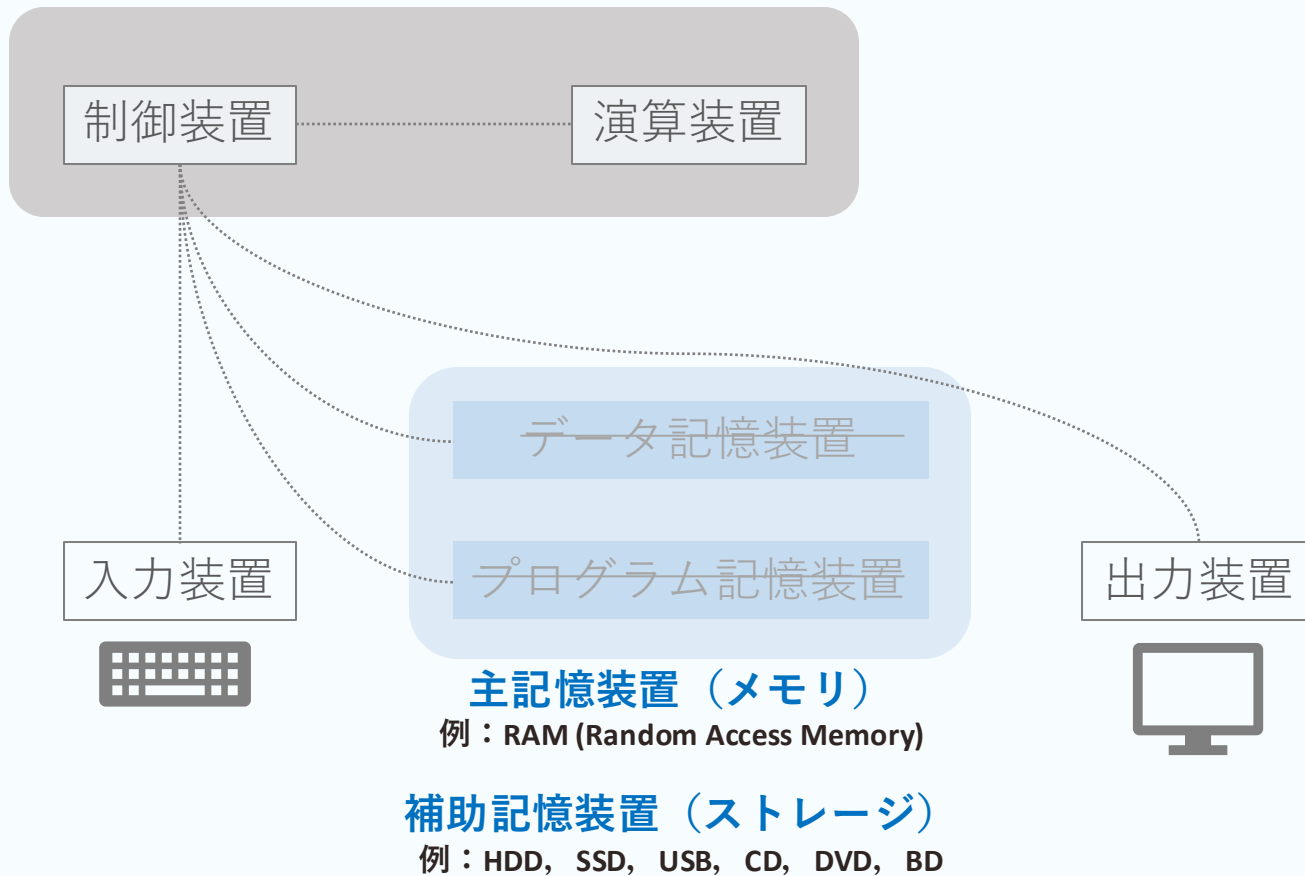
AI専用チップ

画像処理や音声処理，言語処理など，身近なアプリケーションとなりつつある人工知能（AI）演算を効率的に実行するための装置

例）AppleのNeural Engine，IntelやAMDのNPU，GoogleのTPUなど膨大な並列処理が基本となるため，GPUで代用されることもある

中央処理装置（CPU, Central Processing Unit）

例：Intel Core i5, Apple M1, Ryzen5 2500U, Snapdragon S2



CPUがデータを一時的に保存するための記憶装置

代表例）Random Access Memory（RAM）

- 一般に，後述するストレージより容量が小さいが，高速に読み書きできる

使用例）iPhone 16のメモリ（RAM）容量は8GB

処理装置の種類に関する特別なメモリ

キャッシュメモリ

- CPUに内蔵される小さくて高速なメモリ
- この容量に収まる計算なら，CPUはメインメモリを使うよりも高速に動作する

GPUメモリ（VRAM，Video RAM）

- GPUが計算のために使うメモリ
- この容量の範囲内であれば，GPUは高速に並列計算できる

ユニファイドメモリ

- 直訳：統合されたメモリ
- GPUもCPUも使うことのできるメモリ

計算機がデータを長期的に保存するための部品

- 具体例）SSD，HDD，USBメモリ，CD，DVD，クラウドストレージ
- 使用例）iPhone 16のストレージは128GB，256GB，512GBのいずれか

値（バイト数）	2進接頭辞	記号	SI風表記（慣例）	SI記号（慣例）
2^{10}	キビバイト	KiB	キロバイト	KB
2^{20}	メビバイト	MiB	メガバイト	MB
2^{30}	ギビバイト	GiB	ギガバイト	GB
2^{40}	テビバイト	TiB	テラバイト	TB
2^{50}	ペビバイト	PiB	ペタバイト	PB

※厳密には，SI記号で表記することは10進法との混同を招くため推奨されないが，慣例的には多く用いられる

1. コンピュータのハードウェア構成

- CPU, メモリ, ストレージ

2. 計算機を行う演算

- 二進数に対する操作（論理演算）
- 四則演算の実装

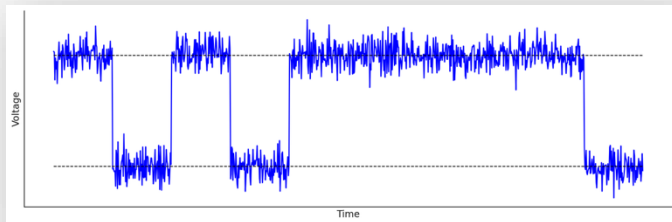
3. 身近に遭遇するかもしれない2進数に由来する現象

- アプリケーションの操作
- パソコンの設定項目
- 計算誤差

現在の電子計算機では、2進数のデジタル（数字式）表現が標準

- あらゆる情報はデジタル化されて有限桁の数字に近似される
- 数字化された情報は2進数で表現されて0と1のみで記述される

10進数	0	1	2	3	4	5	6	7	8	9	10	11
2進数	0	1	10	11	100	101	110	111	1000	1001	1010	1011



デジタル（数字式）：情報を保持・伝達しやすい
2進数（0と1のみ）：機械で表現しやすい

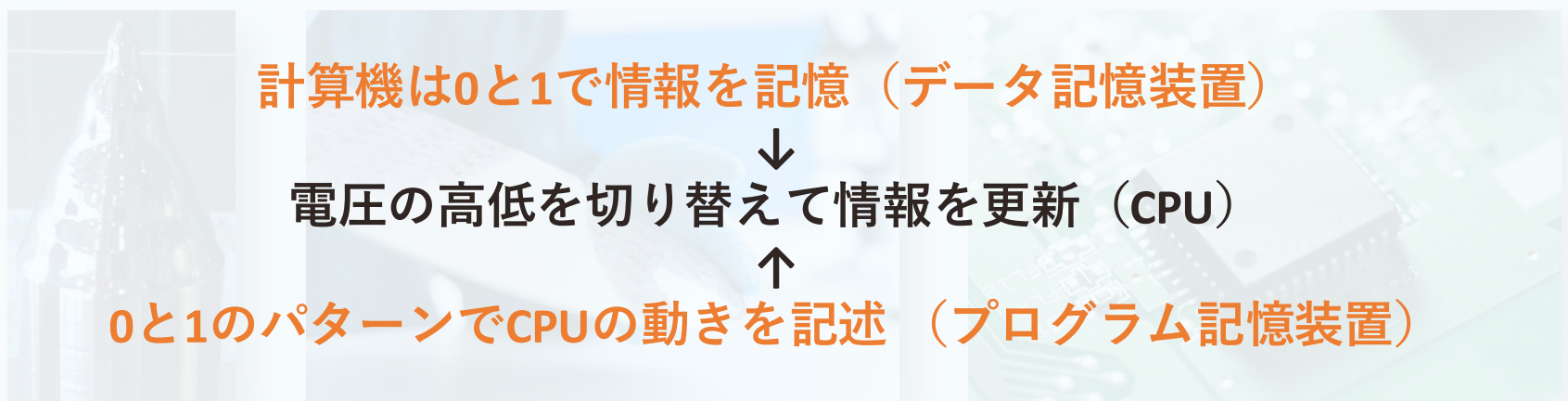
微細な電圧スイッチで0と1を表現

半導体：導体（電気を通す）と絶縁体（通さない）の中間的な物質

- ・ 電気抵抗が導体と絶縁体の中間程度の物質
 - ・ 例：シリコン，ゲルマニウム
- ・ 不純物や外部刺激の有無によって，電気を通したり通さなかったりする
→ スイッチとして0/1を表現可能

ICチップ：大量のスイッチを並べた装置（集積回路，Integrated Circuit）

- ・ 微細な半導体装置を大量に並べたデバイス
- ・ 各半導体装置は電気信号の「遮断 or 通過」で「0 or 1」を表現



計算機は0と1で情報を記憶（データ記憶装置）

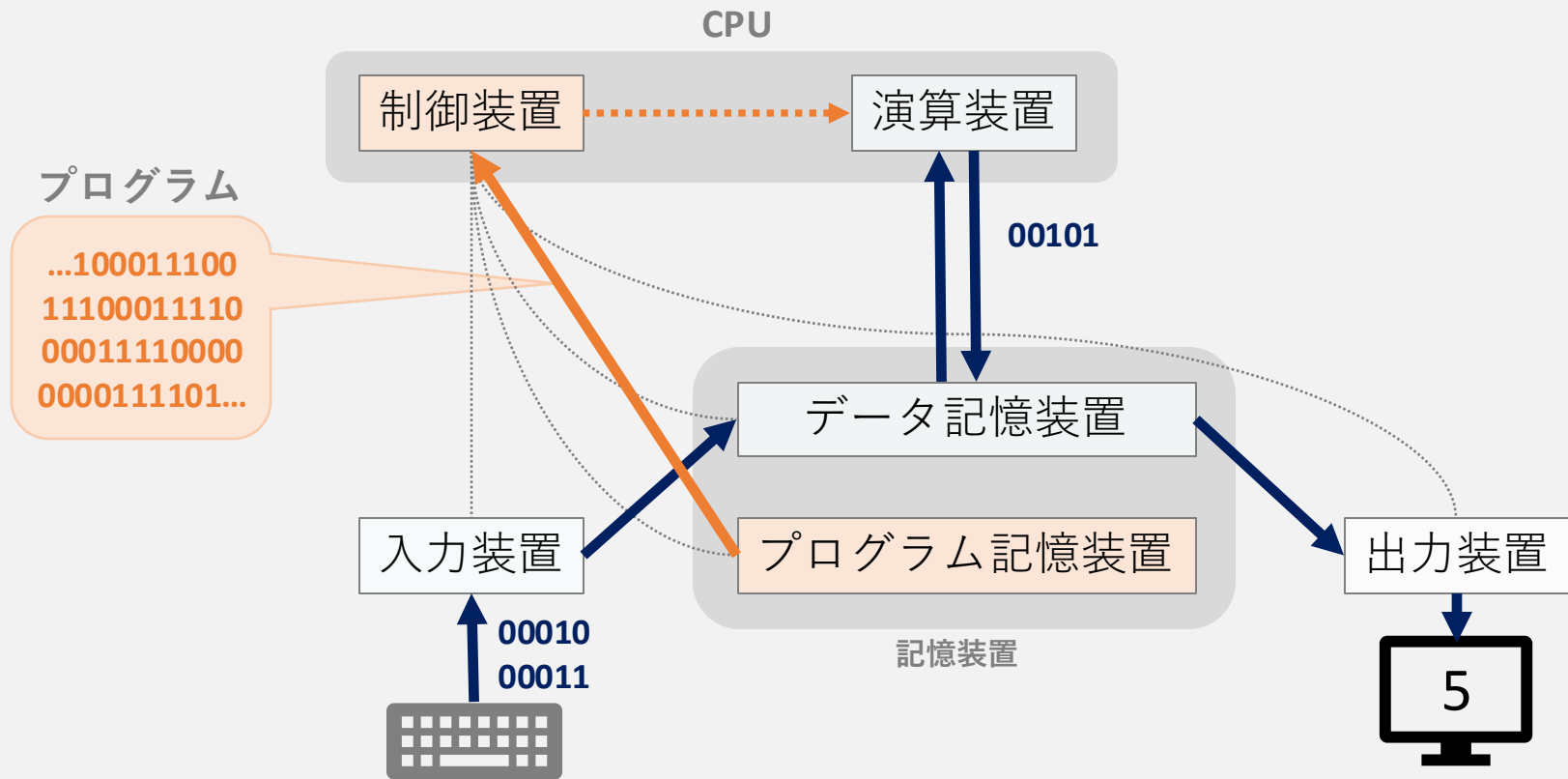
↓
電圧の高低を切り替えて情報を更新（CPU）

↑
0と1のパターンでCPUの動きを記述（プログラム記憶装置）

左：シリコン単結晶のインゴット

CC表示-継承 3.0, <https://commons.wikimedia.org/w/index.php?curid=314282>

0と1のパターンでCPUの動きを制御



0と1からなる数（真偽値）に対して定義される直感的な操作

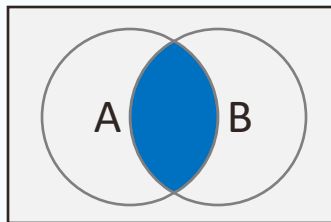
- 論理演算（集合演算）
- ビット反転・シフト演算

四則演算

- 加算
- 減算（ビット反転と加算）
- 乗算（シフト演算と加算）
- 除算（シフト演算と減算）

AND（論理積，かつ）

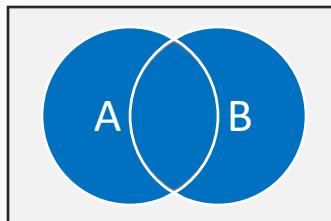
どちらも真ならば真（どちらかが偽ならば偽）



A	B	A and B
0	0	0
0	1	0
1	0	0
1	1	1

OR（論理和，または）

どちらかが真であれば真（どちらも偽ならば偽）

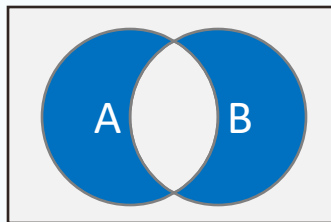


A	B	A or B
0	0	0
0	1	1
1	0	1
1	1	1

XOR（排他的論理和）

2つの真偽が異なれば真（どちらも真 or どちらも偽ならば偽）

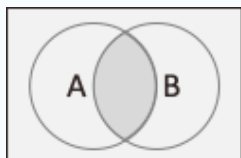
例: 2つスイッチのある部屋



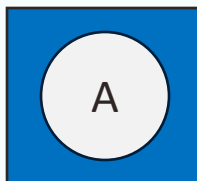
A	B	A xor B
0	0	0
0	1	1
1	0	1
1	1	0

NOT（否定）

真は偽に，偽は真に反転させる演算



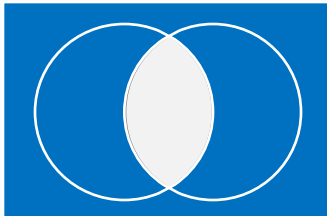
青色：真
灰色：偽



A	Not A
0	1
1	0

NAND（否定論理積）

AND演算の否定→NOT（AND（A, B））

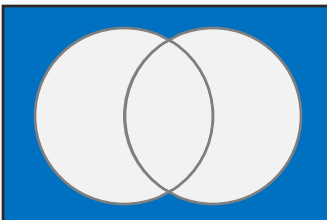


A	B	A and B	A nand B
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

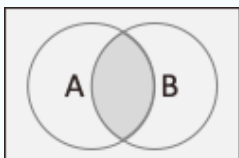
※NAND演算を組み合わせると、あらゆる論理式を実装できる
回路製造コストも低いため、計算機の基本部品として広く利用されている

NOR（否定論理和）

OR演算の否定→NOT（OR（A, B））



A	B	A or B	A nor B
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0



青色：真
灰色：偽

ビット反転：0と1を逆にする

例) 0001→1110, 0101→1010

シフト演算

左シフト：ビットを左にずらして，上位を押し出し，下位に0を詰める

- 00101100→01011000

右シフト：ビットを右にずらして，下位を押し出し，上位に0を詰める

- 10101101→01010110

循環シフト：ビットをずらして，押し出されたものを詰める

- 循環左シフトの例) 10101100→01011001
- 循環右シフトの例) 10101101→11010110

0と1からなる数（真偽値）に対して定義される直感的な操作

- 論理演算（集合演算）
- ビット反転・シフト演算

四則演算

- 加算
- 減算（ビット反転と加算）
- 乗算（シフト演算と加算）
- 除算（シフト演算と減算）

加算

- 10進数と同様に筆算で計算できる
- Webアプリでの解説
 - <https://claude.ai/public/artifacts/1dabe594-0783-405e-a1a4-0979ca0bd6fe>



2024年度資料より許可を得て引用

加算

- 10進数と同様に筆算で計算できる
- Webアプリでの解説
 - <https://claude.ai/public/artifacts/1dabe594-0783-405e-a1a4-0979ca0bd6fe>

減算

- 10進数と同様に筆算で計算できる
- 「補数」という特殊な数値表現を用いれば加算回路を流用して実装できる



1の補数： $a + a_1$ で全ての桁が1となる a_1 は、 a に対する「1の補数」
ビット反転で得られる

- 例) $101 \rightarrow 010$, 検算： $101_{(2)} + 010_{(2)} = 111_{(2)}$

2の補数： $a + a_2 = 2^n$ となる a_2 は、 a に対する「2の補数」
ビット反転と加算 (+1) で得られる

- 例) $101 \rightarrow 010 \rightarrow 011$, 検算： $101_{(2)} + 011_{(2)} = 1000_{(2)} = 2^4_{(10)}$

反対の操作で元に戻せる (-1してビット反転)

- 例) $011 \rightarrow 010 \rightarrow 101$

符号付き2の補数表現

最も大きい桁を正負の符号と解釈する方式

例) 1111 1111

- 符号付き2の補数：符号が1で、2の補数の「111 1111」
 - 2の補数の111 1111 $\rightarrow$ 111 1110 $\rightarrow$ 000 0001 $_{(2)} \rightarrow 1_{(10)}$
 - 答えは-1
- 通常の2進数と解釈する場合： $2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = 255_{(10)}$

加算回路を用いたA-Bの計算手順

1. 「Bの2の補数」を求める
2. $A + \text{「Bの2の補数」}$ を計算する
3. 計算結果の符号を確認する
4. もし**符号が正**ならば、そのまま**2進数**と解釈する
5. もし**符号が負**ならば、**符号付き2の補数**と解釈する

2の補数二進数減算計算機

入力設定

二進数A:
1010

二進数B:
0011

ビット幅:
8ビット

計算式: 1010 - 0011

手順1: 入力値の8ビット表現

A:	0	0	0	0	1	0	1	0
B:	0	0	0	0	0	0	1	1

手順2: Bの1の補数を求める (各ビットを反転)

Made with Claude
Artifacts are user-generated and may contain unverified or potentially unsafe content.

Customize Artifact

加算

- 10進数と同様に筆算で計算できる
- <https://claude.ai/public/artifacts/1dabe594-0783-405e-a1a4-0979ca0bd6fe>

減算

- 「2の補数」と呼ばれる表記を使えば加算の仕組みで計算できる
- <https://claude.ai/public/artifacts/2ec434b1-5e8b-4464-84b0-bbc81dfba806>

乗算

- 10進数と同様に筆算で計算できる

剰算

- 10進数と同様に筆算で計算できる





0と1の列のみで，符号や小数点のある数を表すための工夫

「**符号部**」 「**指数部**」 × 「**仮数部**」

- 符号部：0を正，1を負とする
- 指数部：一番小さな指数が0となるように適当な数値を加えて調整したもの
- 仮数部：最上位の桁は常に1となるので，1を省略して次の2番目の桁から書く

例) 16ビット浮動小数点数

- 符号部 1bit, 指数部 5bit (+15), 仮数部 10bitとする

例) 10進数の2.5を16bit浮動小数点数にする

- $2.5 = 2 + 0.5 = 2^1 \times 1 + 2^0 \times 0 + 2^{-1} \times 1 = 10.1_{(2)}$
- $10.1 \rightarrow + 2^1 \times 1.01 \rightarrow$ 符号+, 指数部 $1+15 = 16$, 仮数部 01
→ 0 10000 0100000000

1. コンピュータのハードウェア構成
 - CPU, メモリ, ストレージ
2. 計算機を行う演算
 - 二進数に対する操作（論理演算）
 - 四則演算の実装
3. 身近に遭遇するかもしれない2進数に由来する現象
 - アプリケーションの操作
 - パソコンの設定項目
 - 計算誤差

例) Notionのフィルター

Filter and sort are available. Changes affect only your page.

フィルターと並べ替えが利用できます。変更は他の閲覧者に反映されません。

year type on site all Authorship type-review year (lead author only) 新規

このページは tomiokario.notion.site で公開されています サイトを表示 サイト設定

リセット このフィルターを保存

↓ Date 3件のルール + フィルター

条件 Review が Reviewed と一致

2025年3月3日 AND Date 開始日 が 今日 より前

AND 条件 Authorship が Lead author を含む

+ フィルタールールを追加

+ フィルタールールを追加

フィルターを削除

Date	type	Review
2025年3月3日 2025年3月4日	Research paper (international conference) : 国際会議	Reviewed
2024年7月10日 → 2024年7月12日	Research paper (international conference) : 国際会議	Reviewed
2024年3月1日 → 2024年3月2日	Research paper (international conference) : 国際会議	Reviewed

カウント 8

AND検索（積集合）

例）北九州市，会食，海鮮全てのワードを含む記事を探す

北九州市 AND 会食 AND ホテル

OR検索（和集合）

例）北九州市，会食に加えて，ホテルか料亭のどちらかを含む記事を探す

北九州市 AND 会食 AND (ホテル OR 料亭)

除外検索（差集合）

特定の語句を除外するときは「-（半角マイナス）」をつける

例）飛行機のトランプを探すにあたり，大統領の記事は除外したい

飛行機 トランプ -大統領

参考：演算子を使用して検索を絞り込む - Android - Cloud Search ヘルプ

<https://support.google.com/cloudsearch/answer/6172299?hl=ja&co=GENIE.Platform%3DAndroid#zippy=%2C%E8%A8%98%E5%8F%B7%E3%81%A8%E6%A8%99%E6%BA%96%E7%9A%84%E3%81%AA%E6%BC%94%E7%AE%97%E5%AD%90>

有線接続のパソコンなど※でネットワーク接続時に現れる設定項目

例) 192.168.1.168/24 (IPアドレス), 255.255.255.0 (サブネットマスク)

- 255.255.255.0 → 11111111.11111111.11111111.00000000 (2進数)
- 上から24桁がLANに共通の住所, 下8桁が端末固有の住所を表す
- 192.168.1.168と255.255.255.0の論理積を取れば, 192.168.1.0となる
- サブネットマスクを反転させて論理積を取れば, 0.0.0.168となる

The image shows a network configuration window for a device connected to a network named 'KITASPOT'. The 'TCP/IP' section is selected and highlighted in blue. Below it, other options like DNS, WINS, 802.1X, Proxies, and Hardware are listed. The 'Configure IPv4' section is set to 'Manually' and displays the following values: IP address: 0.0.0.0, Subnet mask: 255.255.252.0, and Router: 172.17.15.254. The 'Configure IPv6' section is set to 'Automatically' and displays the Router: Router.

Section	Setting	Value
Configure IPv4 (Manually)	IP address	0.0.0.0
	Subnet mask	255.255.252.0
	Router	172.17.15.254
Configure IPv6 (Automatically)	Router	Router

※厳密にはDHCPサーバのないネットワーク接続。

通常, WiFi通信設備には自動的にIPアドレスを割り振るDHCPサーバが用意されている

数を10進数のまま取り扱うための表現

10進数では有限桁の値でも、浮動小数点では無限桁になることがある

金融など、10進数の値を厳密に処理しなければならない場面で問題になる

例) $0.1_{(10)} = 0.00011001100110011001100..._{(2)}$ と無限桁になる → $0.1 \times 10 \neq 1$

比喩) 10進数でも1/3は0.33333...と無限桁になる

パック10進数

- 10進数の各桁をそれぞれ2進数で表現する方法
 - 1桁に4bit (0~10が収まる最小の桁数) を用いる
- 例) 185 → 1 8 5 → 0001 1000 0101
- 金融や会計関連のシステムなどで用いられる

ゾーン10進数

- 直訳: 区域10進数
 - 区切りを表す4bitの情報を、桁と桁の間に挟む方法
- 例1) 185 → 0011 0001 0011 1000 1100 0101 (1100は正を表す符号)
- 例2) -185 → 0011 0001 0011 1000 1101 0101 (1101は負を表す符号)
- 記録効率は悪いが8bit (=1Byte) になり扱いやすい (文字との整合など)

VI. 計算機特有の数値表現に慣れる

12. 演習（基数と論理演算）

- I. 計算機システムを学ぶ意味を見つける
 - 1. 計算機とプログラムによる自動化
 - 2. 計算機，インターネット，情報システム
- II. 計算機やデータ，AIにできることを知る
 - 3. 計算機の基本原理
 - 4. 計算機と統計
- III. 問題を「計算機の使える形」に整理する
 - 5. 情報システムのデザイン
- IV. 計算機の考え方を体感する
 - 6. プログラムの基本
 - 7. データ構造
 - 8. アルゴリズム
 - 9. 演習（データ構造とアルゴリズム）

- V. 計算機のハードウェア構成を知る
 - 10. 計算機の中の情報表現
 - 11. 演算とハードウェア構成

- VI. 計算機特有の数値表現に慣れる
 - 12. 演習（基数と論理演算）

- VII. 計算論的思考
 - 13. 問題解決のためのモデル：PERT図
 - 14. 問題解決のためのモデル：決定表
 - 15. まとめ + 演習（PERT図，決定表）

1. 日本政府や学科の教育方針を知り、現状や将来像に基づいて自分が計算機を学ぶ意味を見つける
2. 計算機やデータ・AIにできることを知り、目的達成に向けた人間と計算機、AIの効果的な分業を想像できるようになる
3. ITシステムの構造や仕組みを学び、問題を計算機が使える形に整理したり、エンジニアに伝えたりできるようになる
4. 計算機のハードウェア構成を知り、目的達成に必要な装置の見積もりや購入ができるようになる
5. 計算機に特有のデータ表現を学び、
機器設定や開発でしばしば遭遇する数値・記号の意味を理解できるようになる
6. 計算機を使うための思考を計算機と無関係な場面で役立てる計算論的思考を体験する

2025年度 コンピュータシステム 第12回：演習（基数と論理演算）

1. カラーコード
2. 真理値表
3. IPアドレスとサブネットマスク

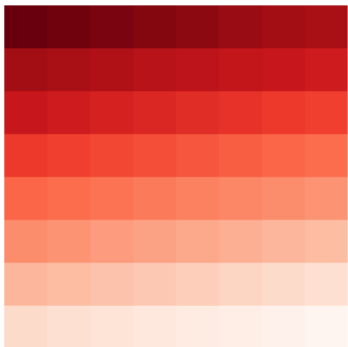
一つの画素に赤緑青の3種類の明るさ情報を持たせる

- 赤青緑を混ぜ合わせると、その割合により様々な色ができる

255 250 245 240 235 230 225 220
225 220 215 210 205 200 195 190
195 190 185 180 175 170 165 160
165 160 155 150 145 140 135 130
135 130 125 120 115 110 105 100
105 100 095 090 085 080 075 070
075 070 065 060 055 050 045 040
045 040 035 030 025 020 015 010



Red



000 000 000 000 000 000 000 000
036 036 036 036 036 036 036 036
072 072 072 072 072 072 072 072
109 109 109 109 109 109 109 109
145 145 145 145 145 145 145 145
182 182 182 182 182 182 182 182
218 218 218 218 218 218 218 218
255 255 255 255 255 255 255 255



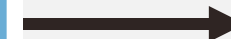
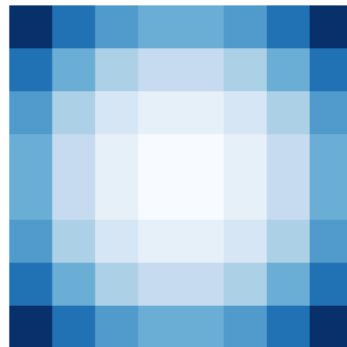
Green



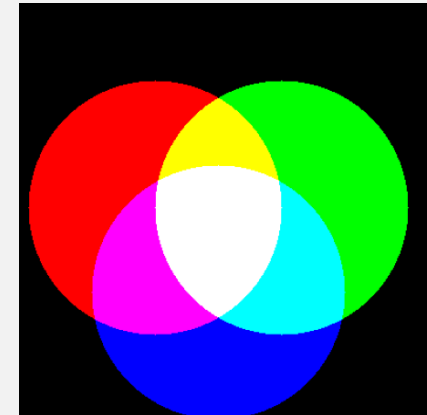
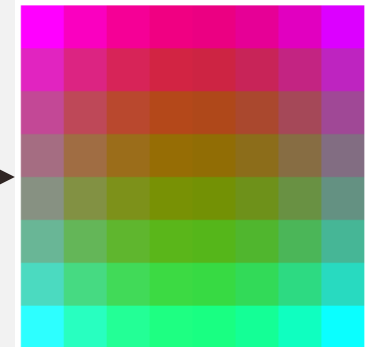
255 192 150 130 130 150 192 255
192 130 088 067 067 088 130 192
150 088 046 026 026 046 088 150
130 067 026 005 005 026 067 130
130 067 026 005 005 026 067 130
150 088 046 026 026 046 088 150
192 130 088 067 067 088 130 192
255 192 150 130 130 150 192 255



Blue



Color



光の三原色

括弧の中にRed, Green, Blueの強さを順番に書く表現方法

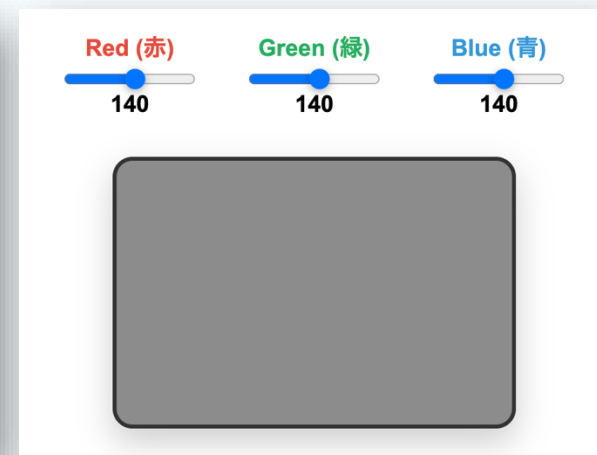
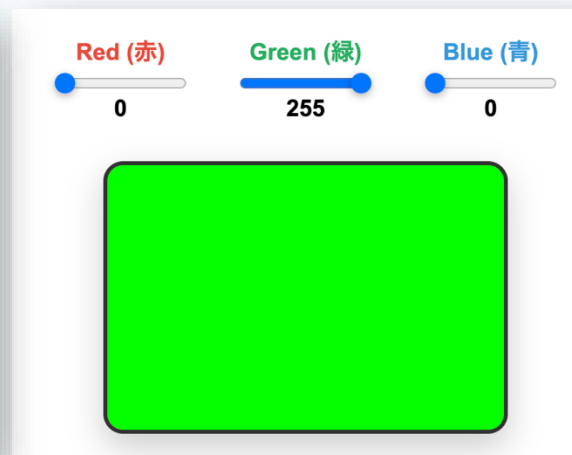
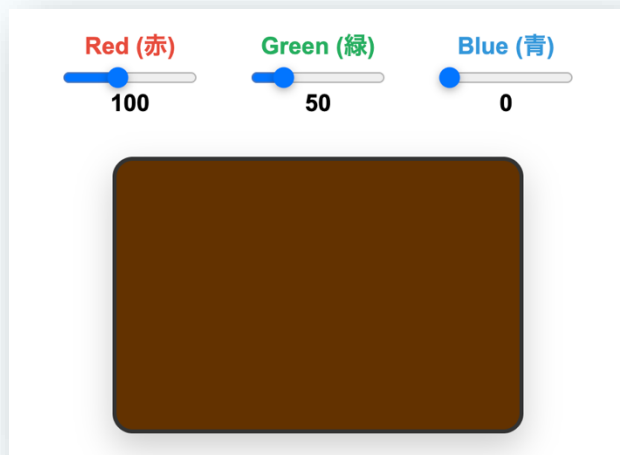
3つの数字を並べて3色の混合比率（各色の強さ）を表現

例1) (100, 50, 0) → Rが100, Gが50, Bが0の割合で混ざったもの

例2) (255, 0, 0) は赤, (0, 255, 0) は緑, (0, 0, 255) は青

8bitの2進数表現：最小値は $0_{(10)} = 0_{(2)}$, 最大値は $255_{(10)} = 1111111_{(2)}$

例3) (0,0,0) は黒色, (255, 255, 255)は白, 全て同じ値なら灰色



16進数により色を表現する方法

- HEX : hexadecimal (16進数) の略
- 先頭に「#」をつけた6桁の16進数で色を表現
 - 6桁の16進数は、24bit (8bit×3色) の2進数に対応
 - 例) #1AFF31 → ($1A_{(16)}$, $FF_{(16)}$, $31_{(16)}$) → (26, 255, 49)



10進数表記 (RGB)



`rgb(26, 255, 49)`

16進数表記 (HEX)



`#1AFF31`



<https://claude.ai/public/artifacts/41aaac6d-a888-40be-8020-3441e40fc942>

アプリに不具合や間違いがあれば講師までご連絡ください

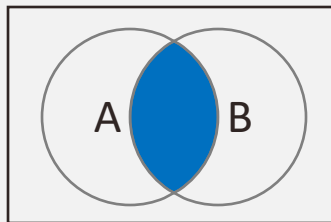
1. カラーコード

2. 真理値表

3. IPアドレスとサブネットマスク

AND（論理積，かつ）

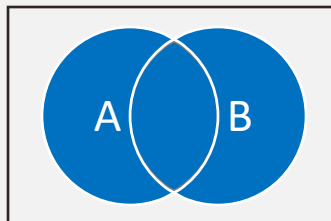
どちらも真ならば真（どちらかが偽ならば偽）



A	B	A and B
0	0	0
0	1	0
1	0	0
1	1	1

OR（論理和，または）

どちらかが真であれば真（どちらも偽ならば偽）

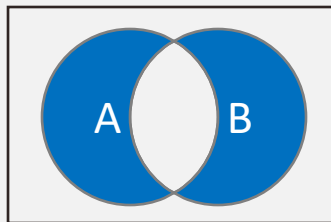


A	B	A or B
0	0	0
0	1	1
1	0	1
1	1	1

XOR（排他的論理和）

2つの真偽が異なれば真（どちらも真 or どちらも偽ならば偽）

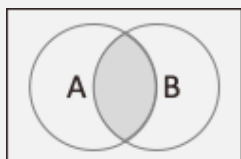
例: 2つスイッチのある部屋



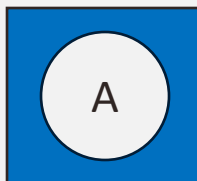
A	B	A xor B
0	0	0
0	1	1
1	0	1
1	1	0

NOT（否定）

真は偽に，偽は真に反転させる演算



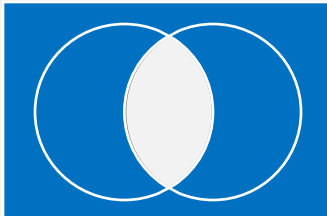
青色：真
灰色：偽



A	Not A
0	1
1	0

NAND（否定論理積）

AND演算の否定→NOT（AND（A, B））

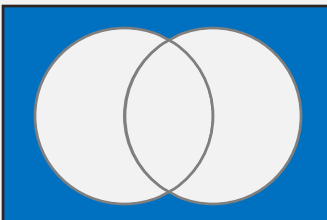


A	B	A and B	A nand B
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

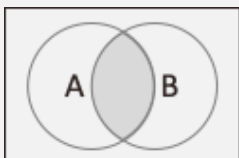
※NAND演算を組み合わせると、あらゆる論理式を実装できる
回路製造コストも低いため、計算機の基本部品として広く利用されている

NOR（否定論理和）

OR演算の否定→NOT（OR（A, B））



A	B	A or B	A nor B
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0



青色：真
灰色：偽

論理関数の全入出力パターンをまとめた表

A	B	A and B
0	0	0
0	1	0
1	0	0
1	1	1

入力

出力

A	B	A or B
0	0	0
0	1	1
1	0	1
1	1	1

入力

出力

A	B	A xor B
0	0	0
0	1	1
1	0	1
1	1	0

入力

出力

A	B	not (A and B)
0	0	1
0	1	0
1	0	0
1	1	0

入力

出力

A	B	not (A or B)
0	0	1
0	1	0
1	0	0
1	1	0

入力

出力

詳細演算子:

- ↑ (NAND): $A \uparrow B$
- ↓ (NOR): $A \downarrow B$
- ⇔ (同値): $A \leftrightarrow B$
- (含意): $A \rightarrow B$

優先順位:

- 括弧 ()
- NOT (¬)
- AND (∧), NAND (↑)
- OR (∨), NOR (↓), XOR (⊕)
- 含意 (→), 同値 (⇔)

使用例:

- $A \wedge B$ (AとB)
- $\neg A \vee B$ (AでないまたはB)
- $(A \vee B) \wedge \neg C$ (AまたはBかつCでない)
- $A \rightarrow B$ (AならばB)
- $A \leftrightarrow B$ (AとBが同値)

真理値表

A	B	C	$A \wedge B \vee C$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Made with Claude

© Artifacts are user-generated and may contain unverified or potentially unsafe content. [Report](#) [Customize Artifact](#)

<https://claude.ai/public/artifacts/47f22590-2bad-4803-888b-620148d6292e>

アプリに不具合や間違いがあれば講師までご連絡ください

AND検索（積集合）

例）北九州市，会食，海鮮全てのワードを含む記事を探す

北九州市 AND 会食 AND ホテル

OR検索（和集合）

例）北九州市，会食に加えて，ホテルか料亭のどちらかを含む記事を探す

北九州市 AND 会食 AND (ホテル OR 料亭)

除外検索（差集合）

特定の語句を除外するときは「-（半角マイナス）」をつける

例）飛行機のトランプを探すにあたり，大統領の記事は除外したい

飛行機 トランプ -大統領

参考：演算子を使用して検索を絞り込む - Android - Cloud Search ヘルプ

<https://support.google.com/cloudsearch/answer/6172299?hl=ja&co=GENIE.Platform%3DAndroid#zippy=%2C%E8%A8%98%E5%8F%B7%E3%81%A8%E6%A8%99%E6%BA%96%E7%9A%84%E3%81%AA%E6%BC%94%E7%AE%97%E5%AD%90>

1. カラーコード
2. 真理値表
3. IPアドレスとサブネットマスク

有線接続のパソコンなど※でネットワーク接続時に現れる設定項目

例) 192.168.1.168/24 (IPアドレス), 255.255.255.0 (サブネットマスク)

- 255.255.255.0 → 11111111.11111111.11111111.00000000 (2進数)
- 上から24桁がLANに共通の住所, 下8桁が端末固有の住所を表す
- 192.168.1.168と255.255.255.0の論理積を取れば, 192.168.1.0となる
- サブネットマスクを反転させて論理積を取れば, 0.0.0.168となる

KITASPOT
Connected

TCP/IP ⚠

DNS

WINS

802.1X

Proxies

Hardware

Configure IPv4 Manually ▾

IP address 0.0.0.0

Subnet mask 255.255.252.0

Router 172.17.15.254

Configure IPv6 Automatically ▾

Router Router

※厳密にはDHCPサーバのないネットワーク接続。

通常, WiFi通信設備には自動的にIPアドレスを割り振るDHCPサーバが用意されている

計算機がネットワークに接続するときに使用する識別番号

ネットワーク部：計算機の所属するネットワークを示す部分

ホスト部：ネットワークにおけるその計算機を示す部分

サブネットマスク：ネットワーク部とホスト部を区別するための番号

例1) 255.255.252.0 → 11111111.11111111.1111100.00000000

例2) 255.255.255.0 → 11111111.11111111.11111111.00000000

CIDR（サイダー）

- Classless Inter-Domain Routing：クラス依存のないドメイン間の経路制御
C：「.」の区切りにとらわれず、好きな位置でネットワーク部とホスト部を区別
IDR：ネットワークの間で通信するための経路を決定
- IPアドレスとサブネットマスクを同時に表記する記法
例1) 192.168.10.100/24：24はネットワークアドレス部の桁数（255.255.255.0）
例2) 192.168.10.100/22：22はネットワークアドレス部の桁数（255.255.252.0）

※本授業では広く普及しているIPv4のみ説明する。他にIPv6と呼ばれる方式も存在する

IPアドレス・サブネット計算機

IPアドレス

172 . 17 . 15 . 251

サブネットマスク

255 . 255 . 252 . 0

計算結果

ネットワーク部
172.17.12.0

ホスト部
0.0.3.251

CIDR表記
172.17.15.251/22

計算過程

Made with Claude © 1
Artifacts are user-generated and may contain unverified or potentially unsafe content.

Report Customize Artifact

<https://claude.ai/public/artifacts/8466c5b1-61df-4e6e-9f5c-7dac5a16fa5f>

アプリに不具合や間違いがあれば講師までご連絡ください

※本講義では、マスクされた部分を0で埋めた結果がネットワーク部やホスト部として表示する

2025年度 コンピュータシステム 第12回：演習 基数と論理演算の解説

2025/7/4(金)

担当講師 富岡莉生

問題1：加算回路での演算

問題2：カラーコード

問題3：論理演算（集合演算）

問題4：サブネットマスク

加算

- 10進数と同様に筆算で計算できる
- Webアプリでの解説
 - <https://claude.ai/public/artifacts/1dabe594-0783-405e-a1a4-0979ca0bd6fe>

減算

- 10進数と同様に筆算で計算できる
- 「補数」という特殊な数値表現を用いれば加算回路を流用して実装できる

10進数：13－6＝7

2進数：1101－110＝111

Borrow (借り)	1	1	0	1
	—	1	1	0
		1	1	1

1の補数： $a + a_1$ で全ての桁が1となる a_1 は、 a に対する「1の補数」
ビット反転で得られる

- 例) $101 \rightarrow 010$, 検算： $101_{(2)} + 010_{(2)} = 111_{(2)}$

2の補数： $a + a_2 = 2^n$ となる a_2 は、 a に対する「2の補数」
ビット反転と加算 (+1) で得られる

- 例) $101 \rightarrow 010 \rightarrow 011$, 検算： $101_{(2)} + 011_{(2)} = 1000_{(2)} = 2^4_{(10)}$

反対の操作で元に戻せる (-1してビット反転)

- 例) $011 \rightarrow 010 \rightarrow 101$

符号付き2の補数表現

最も大きい桁を正負の符号と解釈する方式

例) 1111 1111

- 符号付き2の補数：符号が1で、2の補数の「111 1111」
 - 2の補数の111 1111 $\rightarrow$ 111 1110 $\rightarrow$ 000 0001 $_{(2)} \rightarrow 1_{(10)}$
 - 答えは-1
- 通常の2進数と解釈する場合： $2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = 255_{(10)}$

加算回路を用いたA-Bの計算手順

1. 「Bの2の補数」を求める
2. $A + \text{「Bの2の補数」}$ を計算する
3. 計算結果の符号を確認する
4. もし**符号が正**ならば、そのまま**2進数**と解釈する
5. もし**符号が負**ならば、**符号付き2の補数**と解釈する

2の補数二進数減算計算機

入力設定

二進数A:
1010

二進数B:
0011

ビット幅:
8ビット

計算式: 1010 - 0011

手順1: 入力値の8ビット表現

A:	0	0	0	0	1	0	1	0
B:	0	0	0	0	0	1	1	1

手順2: Bの1の補数を求める (各ビットを反転)

Made with Claude
Artifacts are user-generated and may contain unverified or potentially unsafe content.

Customize Artifact

<https://claude.ai/public/artifacts/c92f5941-714b-426a-9510-d6148391f101>

問題1：加算回路での演算

問題2：カラーコード

問題3：論理演算（集合演算）

問題4：サブネットマスク

括弧の中にRed, Green, Blueの強さを順番に書く表現方法

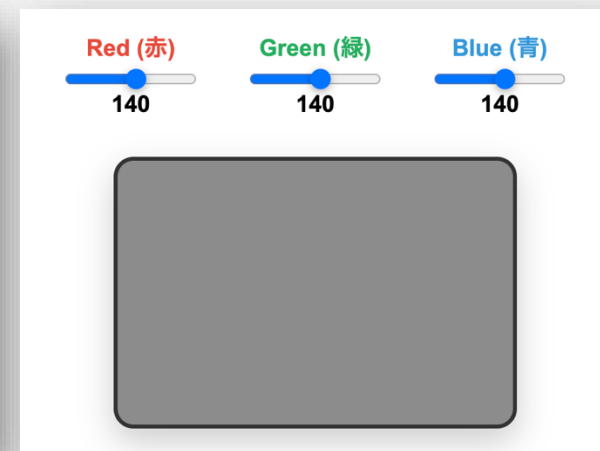
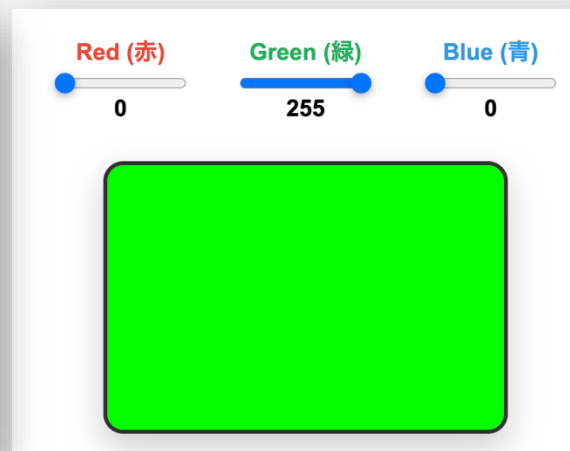
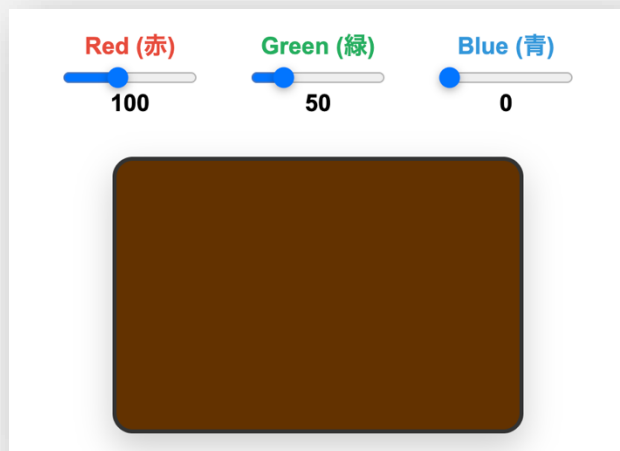
3つの数字を並べて**3色の混合比率**（各色の強さ）を表現

例1) (100, 50, 0) → Rが100, Gが50, Bが0の割合で混ざったもの

例2) (255, 0, 0) は赤, (0, 255, 0) は緑, (0, 0, 255) は青

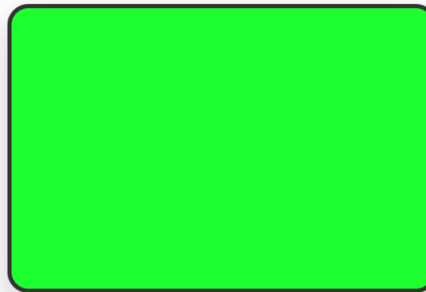
8bitの2進数表現：最小値は $0_{(10)} = 0_{(2)}$, 最大値は $255_{(10)} = 1111111_{(2)}$

例3) (0,0,0) は黒色, (255, 255, 255)は白, 全て同じ値なら灰色



16進数により色を表現する方法

- HEX : hexadecimal (16進数) の略
- 先頭に「#」をつけた6桁の16進数で色を表現
 - 6桁の16進数は、24bit (8bit×3色) の2進数に対応
 - 例) #1AFF31 → ($1A_{(16)}$, $FF_{(16)}$, $31_{(16)}$) → (26, 255, 49)



10進数表記 (RGB)



`rgb(26, 255, 49)`

16進数表記 (HEX)



`#1AFF31`

- **RGB→HEX**

- 10進数→16進数の変換
- 3つの16進数を結合

- **HEX→RGB**

- 2桁ごとに16進数を分離
- それぞれを16進数→10進数に変換

桁に応じた16の冪乗を掛け算する

$$\begin{aligned}\text{例) } 1F3_{(16)} &= 1 \times 16^2 + F \times 16 + 3 \times 1 \\ &= 1 \times 256 + 15 \times 16 + 3 \times 1 \\ &= 499_{(10)}\end{aligned}$$

数値	1	1	1
桁	16^2	16^1	16^0
大きさ	256	16	1



10進数→16進数変換
ステップバイステップで学ぼう！

10進数を入力: 変換開始

16進数対応表
10進数の余りを16進数に変換する際に使用します

0 → 0	1 → 1	2 → 2	3 → 3
4 → 4	5 → 5	6 → 6	7 → 7
8 → 8	9 → 9	10 → A	11 → B
12 → C	13 → D	14 → E	15 → F

3294 (10進数) = CDE (16進数)

C D E

変換手順

- 3294 を16で割ります
 $3294 \div 16 = 205$ 余り 14
余り 14 は16進数では E
- 商の 205 を16で割ります
 $205 \div 16 = 12$ 余り 13
余り 13 は16進数では D
- 商の 12 を16で割ります
 $12 \div 16 = 0$ 余り 12
余り 12 は16進数では C
- 余りを下から上に読み上げると16進数になります
16進数: CDE ... これが答え！

Made with Claude

©Artifacts are user-generated and may contain unverified or potentially unsafe content. [Report](#) [Customize Artifact](#)



問題1：加算回路での演算

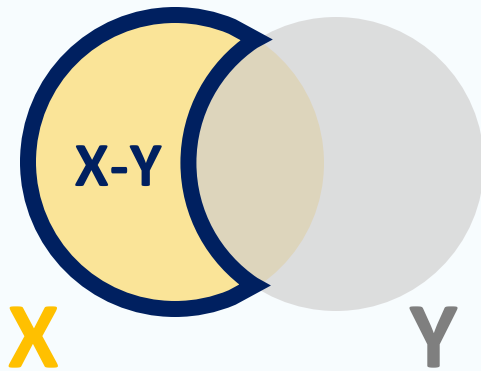
問題2：カラーコード

問題3：論理演算（集合演算）

問題4：サブネットマスク

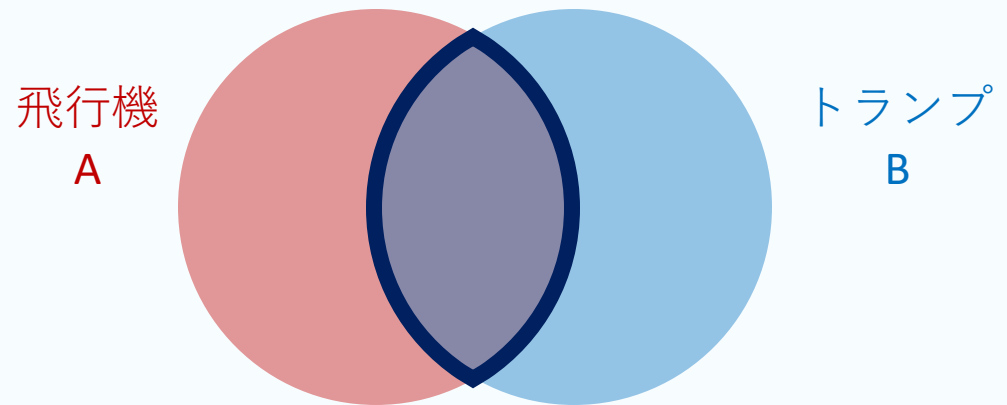
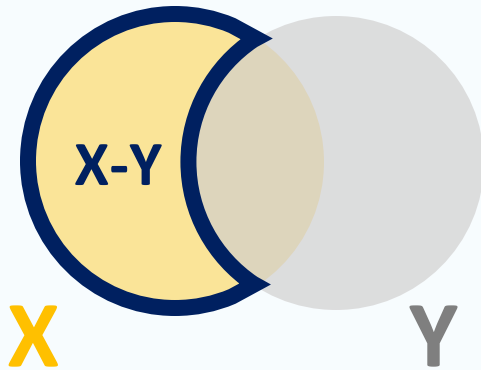
差集合： $X-Y$

- 集合 X から、集合 Y を引いた集合のこと
- 例) 数学履修者 (X) であり、英語履修者(Y)でない生徒の名簿を作る
→ “ X ” かつ “not Y ”



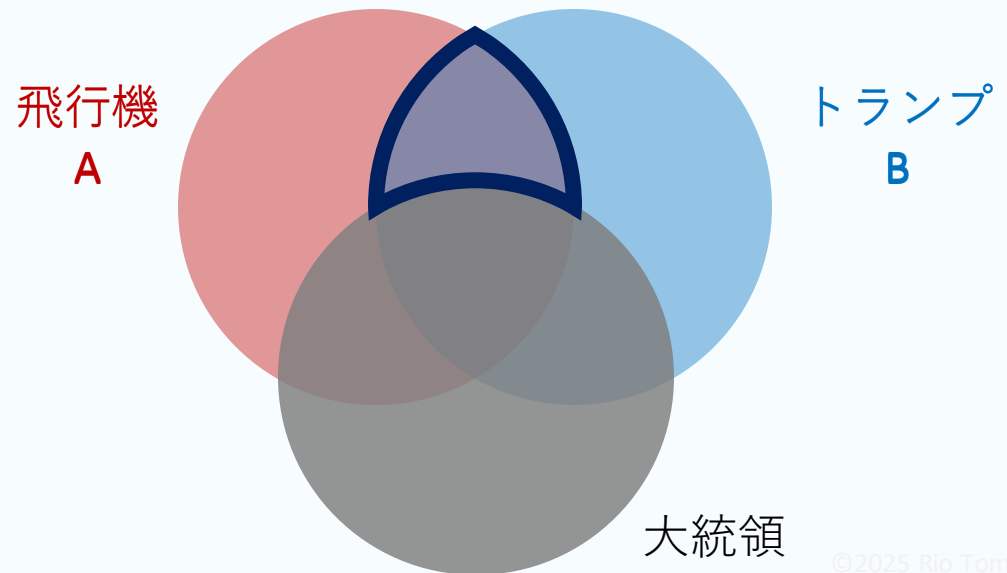
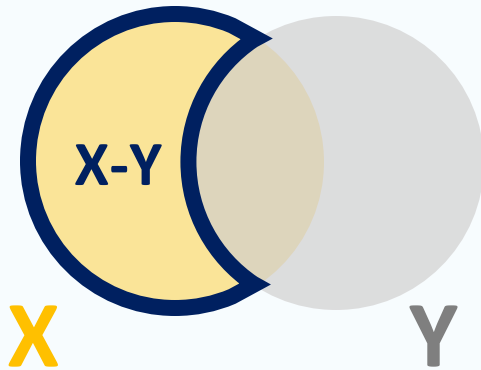
差集合： $X-Y$

- 集合 X から、集合 Y を引いた集合のこと
- 例) 数学履修者 (X) であり、英語履修者(Y)でない生徒の名簿を作る
→ “ X ” かつ “not Y ”



差集合： $X-Y$

- 集合 X から、集合 Y を引いた集合のこと
- 例) 数学履修者 (X) であり、英語履修者(Y)でない生徒の名簿を作る
→ “ X ” かつ “not Y ”



問題1：加算回路での演算

問題2：カラーコード

問題3：論理演算（集合演算）

問題4：サブネットマスク

IPアドレス・サブネット計算機

IPアドレス

172 . 17 . 15 . 251

サブネットマスク

255 . 255 . 252 . 0

計算結果

ネットワーク部
172.17.12.0

ホスト部
0.0.3.251

CIDR表記
172.17.15.251/22

計算過程

Made with Claude © 1
Artifacts are user-generated and may contain unverified or potentially unsafe content.

Report Customize Artifact

<https://claude.ai/public/artifacts/8466c5b1-61df-4e6e-9f5c-7dac5a16fa5f>

アプリに不具合や間違いがあれば講師までご連絡ください

※本講義では、マスクされた部分を0で埋めた結果がネットワーク部やホスト部として表示する

IPアドレス : 172.17.14.173

↓ 2進数

IPアドレス : 10101100.00010001.00001110.10101101

サブネットマスク : 255.255.252.0

↓ 2進数

サブネットマスク : 11111111.11111111.11111100.00000000

ネットワーク部 (IP AND サブネットマスク):

10101100.00010001.00001110.10101101 : IPアドレス (2進数)

11111111.11111111.11111100.00000000 : サブネットマスク (2進数)



IPアドレス AND サブネットマスク = ネットワーク部 (2進数)

10101100.00010001.00001100.00000000



ネットワーク部 (10進数) : ?????

ホスト部(IP AND (NOT サブネットマスク)):

10101100.00010001.00001110.10101101 : IPアドレス (2進数)

00000000.00000000.00000011.11111111 : NOT サブネットマスク



IPアドレス AND (NOT サブネットマスク) = ホスト部

00000000.00000000.00000010.10101101



ホスト部 (10進数) : ???

VII. 計算論的思考

- 13. 問題解決のためのモデル：PERT図
- 14. 問題解決のためのモデル：決定表
- 15. 演習

コンピュータシステム

情報社会において**情報技術（IT）**を活用できるようになるために
そのキーテクノロジーである**コンピュータの仕組み**を学ぶ

授業目的

1. コンピュータの仕組みを理解する
2. コンピュータの引き起こす現象をその仕組みから正しく把握できるようになる
3. ITシステムを仕事や生活で活用できるようになる

授業内容

- コンピュータ，データ・AIにできること
- ITシステムの依頼から，デザイン，実装されるまでの流れ
- コンピュータのハードウェア構成（CPU，メモリ，ストレージなど）
- コンピュータが様々な情報処理を行える仕組み（アルゴリズム，基数など）
- コンピュータの外でも役立つコンピュータを使うための思考（計算論的思考）

I. 計算機システムを学ぶ意味を見つける

1. 情報社会
2. 高校「情報I」の復習

II. 計算機やデータ，AIにできることを知る

3. 計算機の基本原理
4. 計算機と統計

III. 問題を「計算機の使える形」に整理する

5. 情報システムのデザイン

IV. 計算機の考え方を体感する

6. プログラムの基本
7. データ構造
8. アルゴリズム
9. 演習（データ構造とアルゴリズム）

VI. 計算機のハードウェア構成を知る

10. 計算機の中の情報表現
11. 計算機が行う演算

VII. 計算機特有の数値表現に慣れる

12. 演習（基数と論理演算）

VIII. 計算論的思考

13. 問題解決のためのモデル：PERT図
14. 問題解決のためのモデル：決定表
15. まとめ + 演習（PERT図，決定表）

書籍：『計算論的思考ってなに？』

- 近代科学社（2022）， 中島 秀之， 2200円+税

PDF：Computational Thinking - 計算論的思考

- Jeannette M. Wing（Microsoft Research and Carnegie Mellon University）
- 翻訳：中島秀之（公立はこだて未来大学）
- 情報処理 Vol.56 No.6 June 2015
- <https://www.cs.cmu.edu/afs/cs/usr/wing/www/ct-japanese.pdf> 2025/07/10アクセス

書籍：『演習形式で学ぶオペレーションズ・リサーチ』

- 共立出版（2008）， 宮地 功， 2,300円+税

書籍：『初等 オペレーションズ・リサーチ』

- コロナ社（2008）， 小田中 敏男， 正道寺 勉， 3,300円+税

2025年度 コンピュータシステム

第13回：問題解決のためのモデル（PERT図）

1. 計算論的思考

- 計算機科学の考え方を，日常の問題一般に活用する思考方法
- 世界の事象を計算機の比喻で捉えて，問題解決に計算理論を応用する

2. オペレーションズ・リサーチ

- オペレーションの科学的な研究

3. 矢線図（アローダイアグラム）

- スケジューリングを評価するための数学モデル

計算機科学の考え方を，日常の問題一般に活用する思考方法

計算機科学 (Computer Science)

- 計算機による情報処理（計算）の理論と応用を研究する学問
- 問題の難しさや最善の解決の道筋を正確に捉えるための理論や概念を保有

あらゆる人が「計算機科学者のように考える方法」

- 科学分野のみならずあらゆる問題の解決に使える手法
 - プログラミングに限らず，生活や仕事の場面におけるあらゆる思考が対象
- 計算機科学の概念を，複雑な問題を体系的に解決する思考に活用
 - 計算の概念は複雑な問題を体系的に分析して解決策を見つけるために有用

計算論的思考とは、計算機科学者のように考えること

× コンピュータに詳しくなること

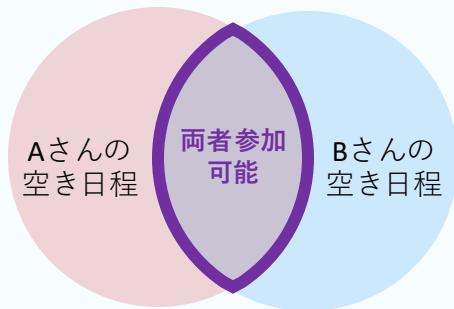
- ・ ハードウェア、ソフトウェア、システムを理解することが計算論的思考ではない

× プログラミング的思考を身につけること

- ・ 「コンピュータのように論理的に考える」ことが計算論的思考ではない

○ 計算機科学者のように考えること

- ・ 現実世界の事物や出来事について、その本来の性質や関係性を損なわずに**計算機の仕組みや動作に喩えて考える**ことができる
- ・ 問題の事象を計算機の比喻で捉えられれば、計算機科学の考え方を活用して**その難しさを測ったり、确实・簡単な解決手順を導いたり**できる



日程調整は集合演算

鍵A	鍵B	ステップ数
開	開	2 (確認2回 + 回転0回, 最良計算時間)
開	閉	3 (確認2回 + 回転1回)
閉	開	3 (確認2回 + 回転1回)
閉	閉	4 (確認2回 + 回転2回, 最悪計算時間)

2つの鍵のついたドアはand演算
(最悪計算時間は4ステップ)

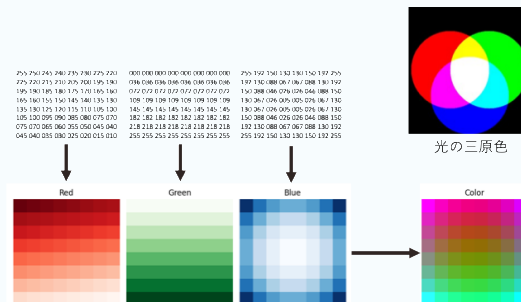
商品ID (product_id)	商品名 (product_name)	単価 (price)
A01	ノートパソコン	120000
A02	マウス	2000
A03	キーボード	5000

通販サイトはデータベース

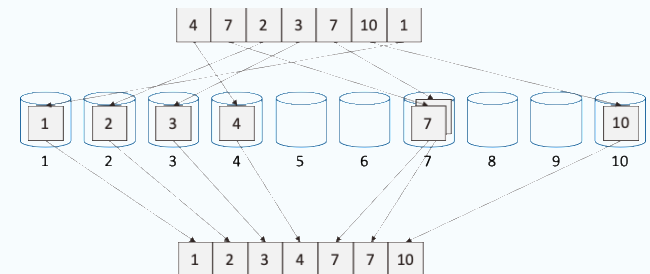


スイッチは2進数

部屋に2つある照明スイッチはXOR



画像は配列



分別してから順序を揃える操作はバケットソート

CC0: hashcc/railmaps:
Railway maps of
world's famous cities.
<https://github.com/hashcc/railmaps>



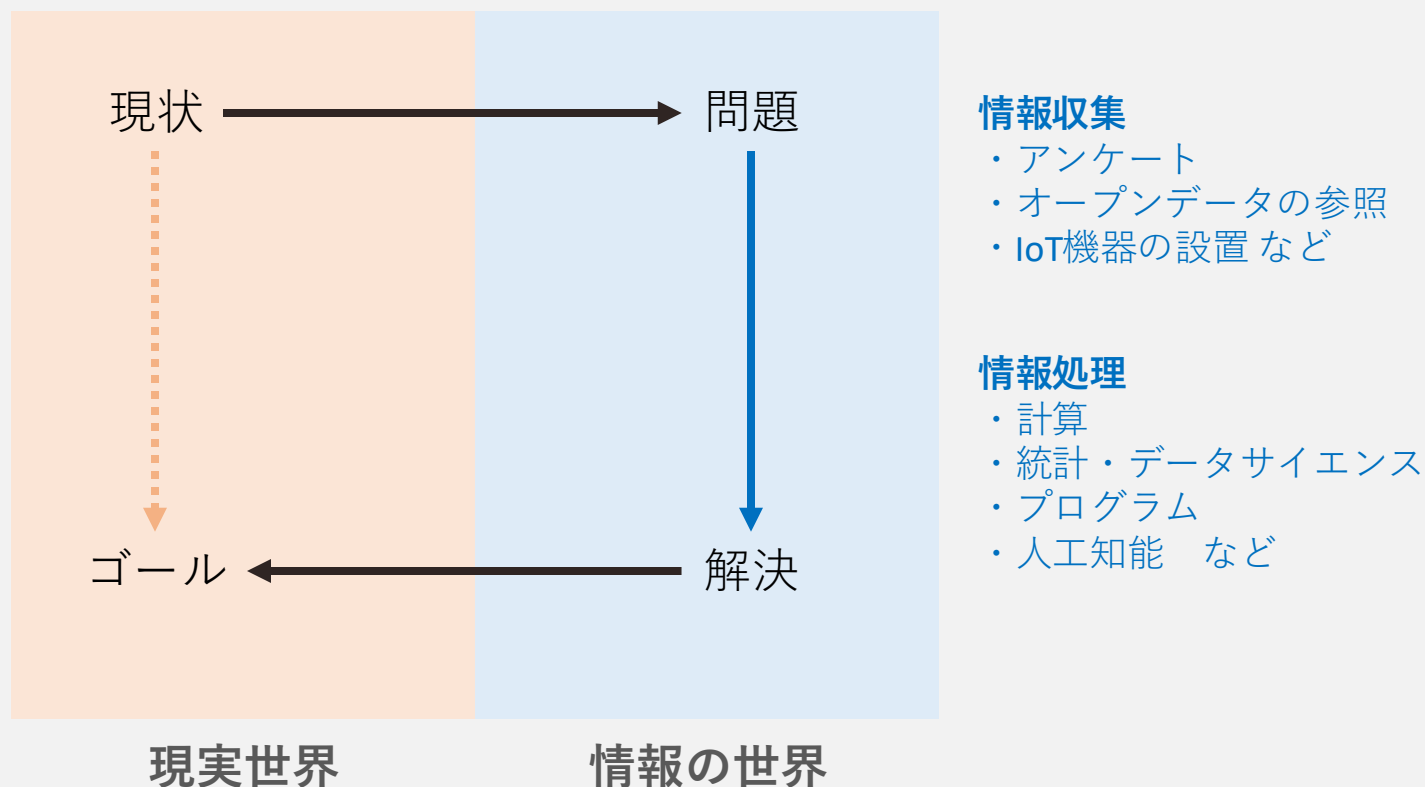
路線図・地図はグラフ



本棚の端から探すのやり方は線形探索

情報世界を介して現実世界の問題を解決する

1. 現実世界の問題を**情報世界の問題**に変換
2. 情報処理の**知識や道具を活用**して情報世界で問題を解決
3. 情報世界で解決した結果を**現実世界の結果**に変換



1. 計算論的思考

- 計算機科学の考え方を，日常の問題一般に活用する思考方法
- 世界の事象を計算機の比喻で捉えて，問題解決に計算理論を応用する

2. オペレーションズ・リサーチ

- オペレーションの科学的な研究

3. 矢線図（アローダイアグラム，PERT図）

- スケジューリングを評価するための数学モデル

オペレーションの科学的な研究

オペレーション（operation, 作戦）

設定目標に向けて、規則に従ってなされる行動のパターン

例1) 軍隊における戦略的な意思決定, 長期的な組織計画, 攻撃作戦

例2) 企業経営の意思決定, 資金配分, 生産管理, マーケティング

オペレーションの科学的な研究

観察や実験, 理論的分析により, ①色々なオペレーションの行動を理解し,
②変化後の結果を予測し, ③より良く管理して成果の改善を図る取り組み

例) 専門家を集めて, 実践的な効果を上げるための潜水艦攻撃作戦を立案

典型的には, 数学モデルや数学的方法論が用いられる

例1) 倉庫から店舗までの運送網をグラフにして, 輸送費が最安の経路を計算

例2) 複数種の仕事を複数台の機械で捌くとき, ガントチャートで最速の分業を計算

仕事の日程の改善を計る問題

スケジューリング（日程管理）の問題に関するORの一分野

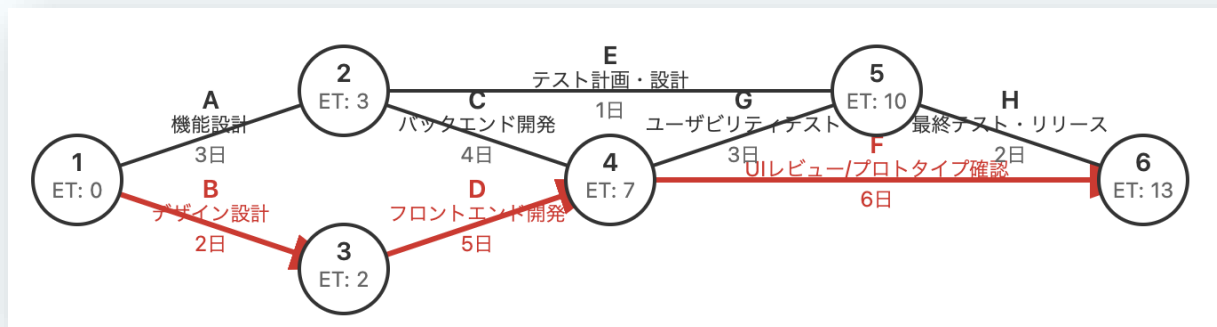
PERT（Program Evaluation and Review Technique）

日本語訳）計画評価と見直しの技法

- プロジェクト型の仕事に対するスケジューリングの有効な方法

アプローチ：工程間の依存関係を可視化して，工程管理に役立てる

- 仕事の順序関係を矢印で示したグラフ（アロダイアグラム）で表現する
- 仕事全体の日程においてボトルネックとなる重要経路（クリティカルパス）を見つけ出す
- ボトルネックを踏まえて工程の改善をはかる



※日程管理だけでなく，費用を含めた作業計画を指してPERTということもある

1. 計算論的思考

- 計算機科学の考え方を，日常の問題一般に活用する思考方法
- 世界の事象を計算機の比喻で捉えて，問題解決に計算理論を応用する

2. オペレーションズ・リサーチ

- オペレーションの科学的な研究

3. 矢線図（アローダイアグラム）

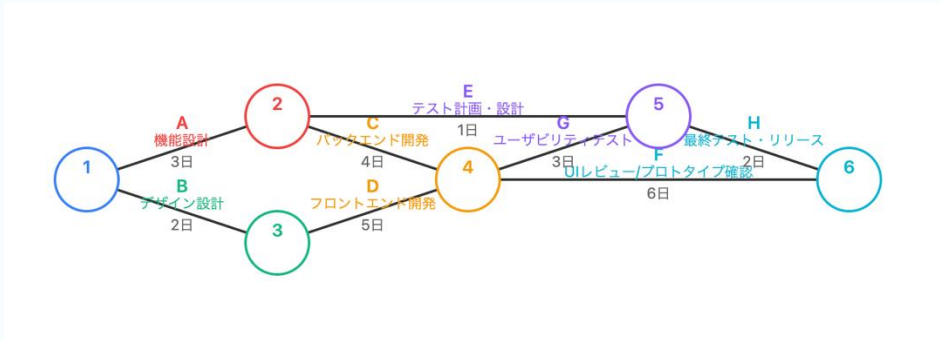
- スケジューリングを評価するための数学モデル

依存関係のある作業日程を矢印付きのグラフに表現する

- 1. 各作業は結節点をつなぐ矢印
作業A, B, C, ..., Hは, グラフの辺で表現（作業にかかる日数を表示）
- 2. 直接前後の関係のある作業は直列に接続
Aの後に始められるCとEは, Aの終点である結節点2を始点とする矢印で表現
- 3. 前後関係のない作業は並列に配置
AとB, EとFなどは互いに依存しないので直列配置されない

記号：具体的な作業内容 (エッジ, 辺)	作業(i,j) 結合点iからjを結ぶ辺	前提作業	所要日数D _{ij}
A 機能設計	(1,2)	-	3
B デザイン設計	(1,3)	-	2
C バックエンド開発	(2,4)	A	4
D フロントエンド開発	(3,4)	B	5
E テスト計画・設計	(2,5)	A	1
F UIレビュー/プロトタイプ確認	(4,6)	C, D	6
G ユーザビリティテスト	(4,5)	C, D	3
H 最終テスト・リリース	(5,6)	E, G	2

作業表



アローダイアグラム※

※辺が矢印として描かれていないが, 左側が始点, 右側が終点となっている

開始から各結節点（node）に到達するまでにかかる時間

1. 開始地点のETを0に設定
 - ET_1 は0
2. 各結合点のETを計算
 - 計算手順
 - 終点のET = 始点のET + 作業日数
 - ただし、複数の経路がある場合は、最も大きい（遅い）値をETとする
 - 意味：全作業を終えて結合点への到達が完了し、次に進めるようになる時刻

結合点 (node)

- 結合点は番号 i で表記する

作業 (辺, 矢印)

- 2つの結合点 i と j を用いて, 作業 (i, j) のように表記する
- 作業の所要日数は, $D_{i,j}$ と表記する

最早結合点時刻 (ET : earliest node time)

- ET_i と表記する
- ET_i は, 最初の結合点1と結合点 i を結ぶ順方向の合計の最大値である
- ET_i の実際の意味は, 結合点 i に達しうる最も早い時刻である

計算方法

- $ET_1 = 0$
- $ET_{k+1} = \max_k(ET_k) \quad k = 1, 2, \dots, n - 1$

書籍：『計算論的思考ってなに？』

- 近代科学社（2022）， 中島 秀之， 2200円+税

PDF：Computational Thinking - 計算論的思考

- Jeannette M. Wing（Microsoft Research and Carnegie Mellon University）
- 翻訳：中島秀之（公立はこだて未来大学）
- 情報処理 Vol.56 No.6 June 2015
- <https://www.cs.cmu.edu/afs/cs/usr/wing/www/ct-japanese.pdf> 2025/07/10アクセス

書籍：『演習形式で学ぶオペレーションズ・リサーチ』

- 共立出版（2008）， 宮地 功， 2,300円+税

書籍：『初等 オペレーションズ・リサーチ』

- コロナ社（2008）， 小田中 敏男， 正道寺 勉， 3,300円+税

2025年度 コンピュータシステム

第14回：問題解決のためのモデル（決定表）

書籍：『演習形式で学ぶオペレーションズ・リサーチ』

- ・ 共立出版（2008）， 宮地 功， 2,300円+税

書籍：『初等 オペレーションズ・リサーチ』

- ・ コロナ社（2008）， 小田中 敏男， 正道寺 勉， 3,300円+税

書籍：『PERT・CPM 改定（ORライブラリー 11）』

- ・ 日科技連出版社（1965）， 関根 智明

1. PERT

- 日程管理計画問題
- PERTとアローダイアグラム
- 最早結合点時刻（ET）と最遅結合点時刻（LT）

2. 作業の日程計画

- 結合点の時刻に基づく作業日程の計算
- 余裕とクリティカルパス

3. 決定表

- 条件と結果の対応関係を一覧できる表

仕事の日程の改善を計る問題

スケジューリング（日程管理）の問題に関するORの一分野

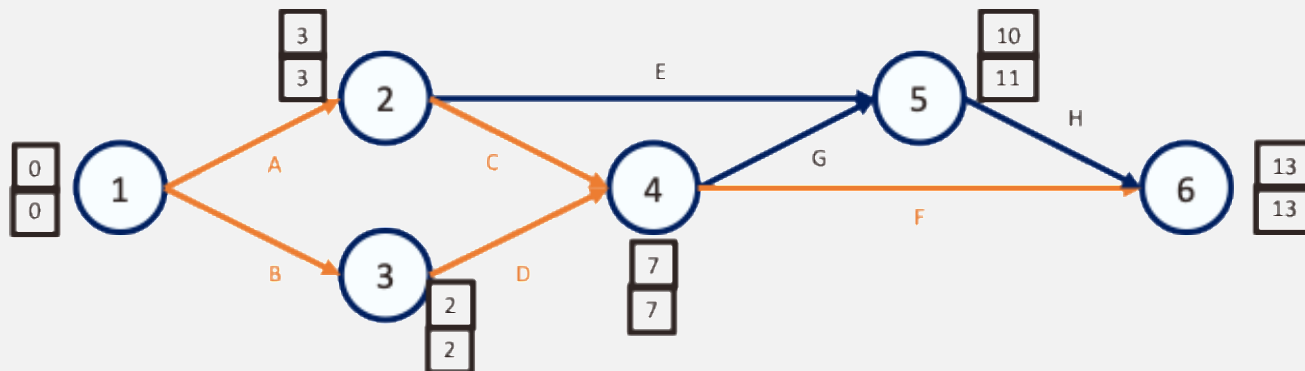
PERT（Program Evaluation and Review Technique）

日本語訳）計画評価と見直しの技法

- プロジェクト型の仕事に対するスケジューリングの有効な方法

アプローチ：**工程間の依存関係を可視化**して、工程管理に役立てる

- 仕事の順序関係を矢印で示したグラフ（**アロダイアグラム**）で表現する
- 仕事全体の日程においてボトルネックとなる重要経路（**クリティカルパス**）を見つけ出す
- ボトルネックを踏まえて工程の改善をはかる



※日程管理だけでなく、費用を含めた作業計画を指してPERTということもある

依存関係のある作業日程を矢印付きのグラフに表現する

1. 各作業は結節点をつなぐ矢印

作業A, B, C, ..., Hは, グラフの辺で表現（作業にかかる日数を表示）

2. 直接前後の関係のある作業は直列に接続

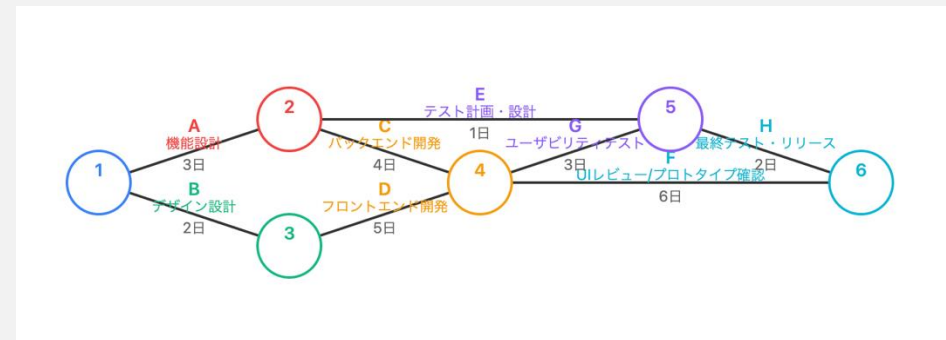
Aの後に始められるCとEは, Aの終点である結節点2を始点とする矢印で表現

3. 前後関係のない作業は並列に配置

AとB, EとFなどは互いに依存しないので直列配置されない

記号：具体的な作業内容 (エッジ, 辺)	作業(i,j) 結合点iからjを結ぶ辺	前提作業	所要日数D _{ij}
A 機能設計	(1,2)	-	3
B デザイン設計	(1,3)	-	2
C バックエンド開発	(2,4)	A	4
D フロントエンド開発	(3,4)	B	5
E テスト計画・設計	(2,5)	A	1
F UIレビュー/プロトタイプ確認	(4,6)	C, D	6
G ユーザビリティテスト	(4,5)	C, D	3
H 最終テスト・リリース	(5,6)	E, G	2

作業表



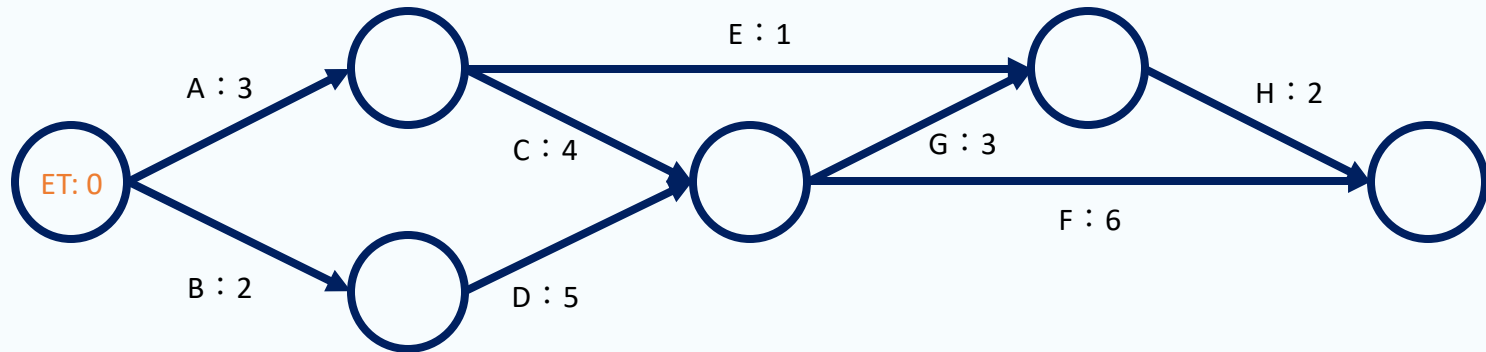
アロダイアグラム※

※辺が矢印として描かれていないが, 左側が始点, 右側が終点となっている

開始から各結節点（node）に到達するまでにかかる時間
＝結合点を最も早く出発して、次の作業を開始できる時間

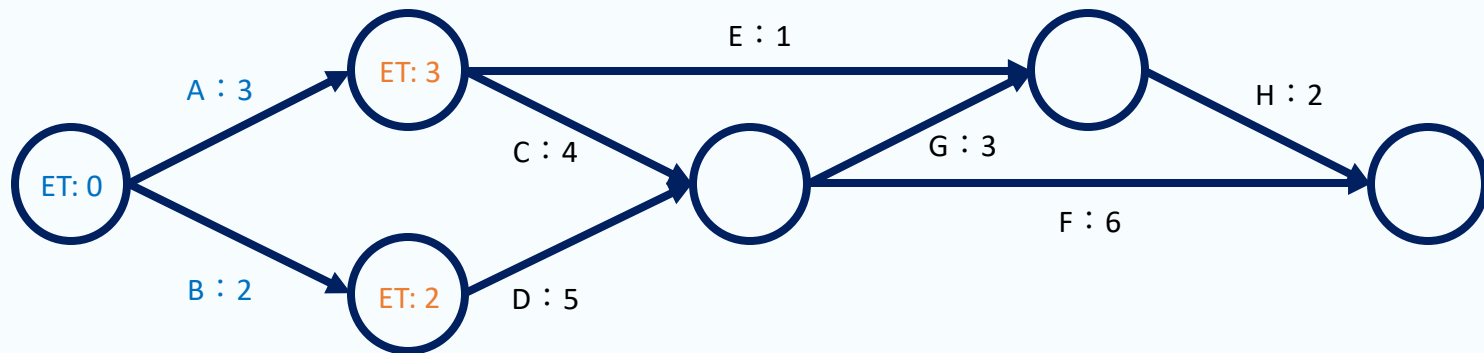
1. 開始地点のETを0に設定

- ET_1 は0



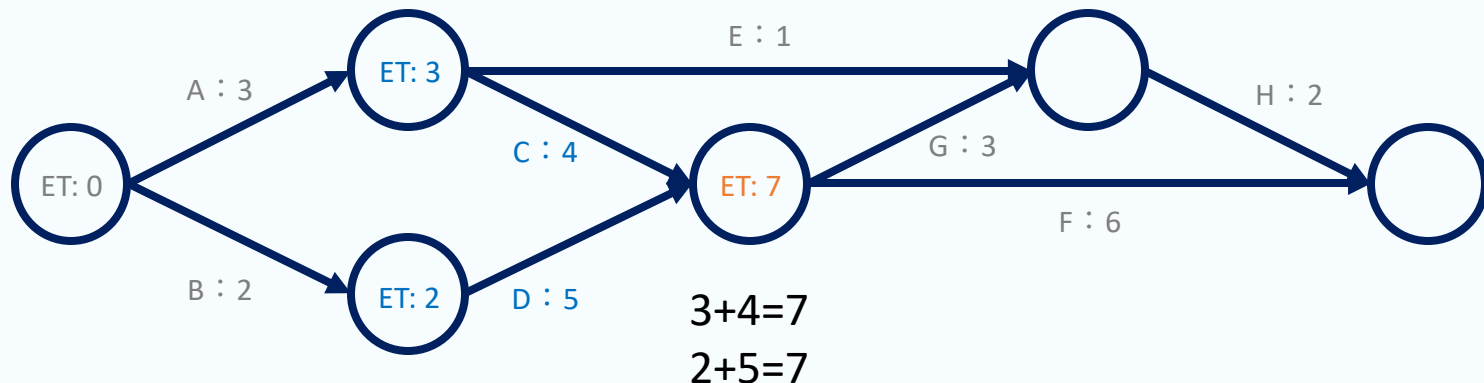
開始から各結節点（node）に到達するまでにかかる時間
＝結合点を最も早く出発して、次の作業を開始できる時間

1. 開始地点のETを0に設定
 - ET_1 は0
2. 各結合点のETを計算
 - 終点のET = 始点のET + 作業日数



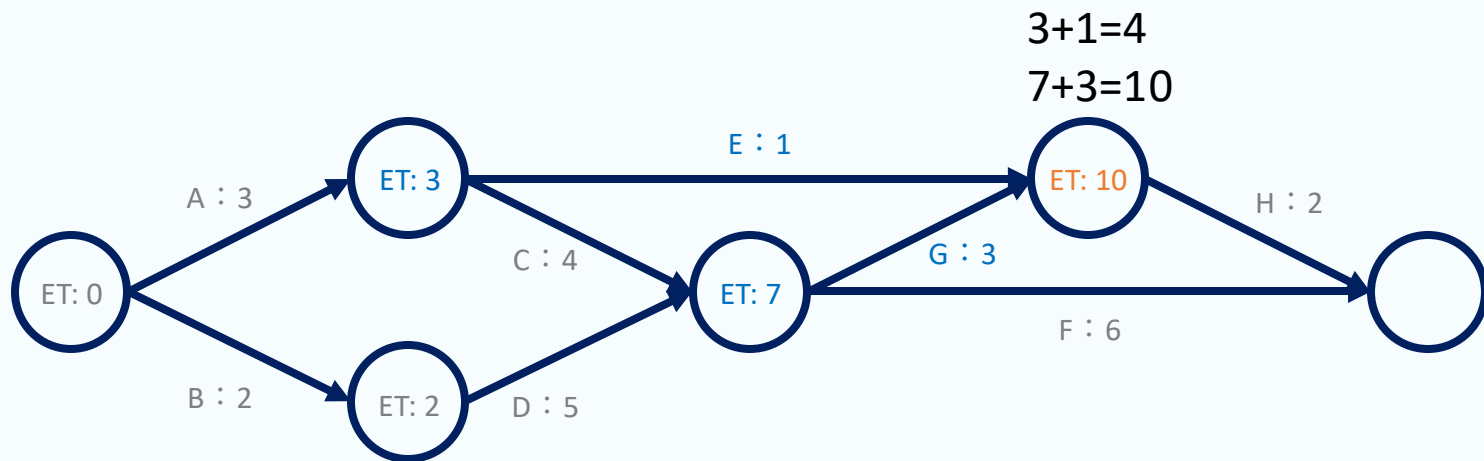
開始から各結節点（node）に到達するまでにかかる時間
＝結合点を最も早く出発して、次の作業を開始できる時間

1. 開始地点のETを0に設定
 - ET_1 は0
2. 各結合点のETを計算
 - 終点のET = 始点のET + 作業日数
 - 複数の経路がある場合は、最も大きい（遅い）値をETとする
(遅れた方の作業が終わるまで次の作業が始められないため)



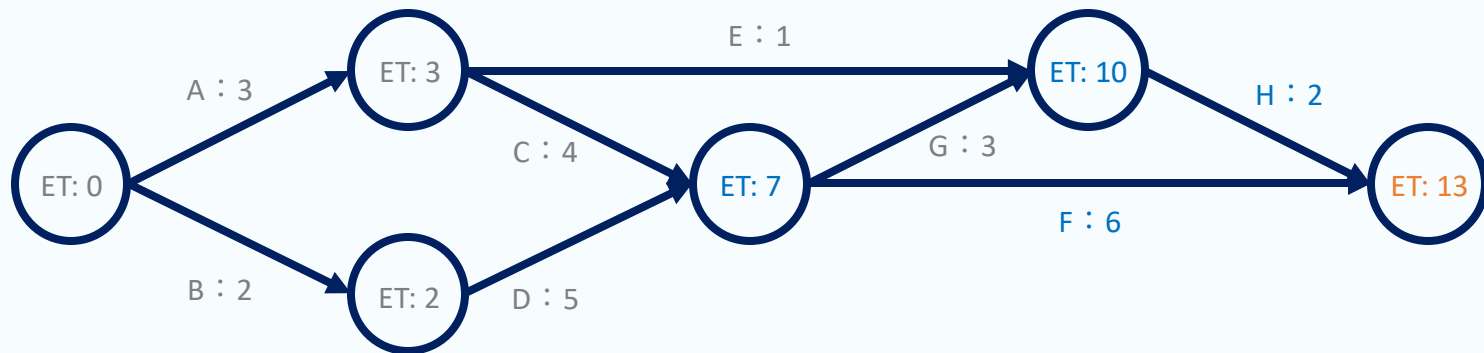
開始から各結節点（node）に到達するまでにかかる時間
＝結合点を最も早く出発して、次の作業を開始できる時間

1. 開始地点のETを0に設定
 - ET_1 は0
2. 各結合点のETを計算
 - 終点のET = 始点のET + 作業日数
 - 複数の経路がある場合は、最も大きい（遅い）値をETとする
(遅れた方の作業が終わるまで次の作業が始められないため)



開始から各結節点（node）に到達するまでにかかる時間
＝結合点を最も早く出発して、次の作業を開始できる時間

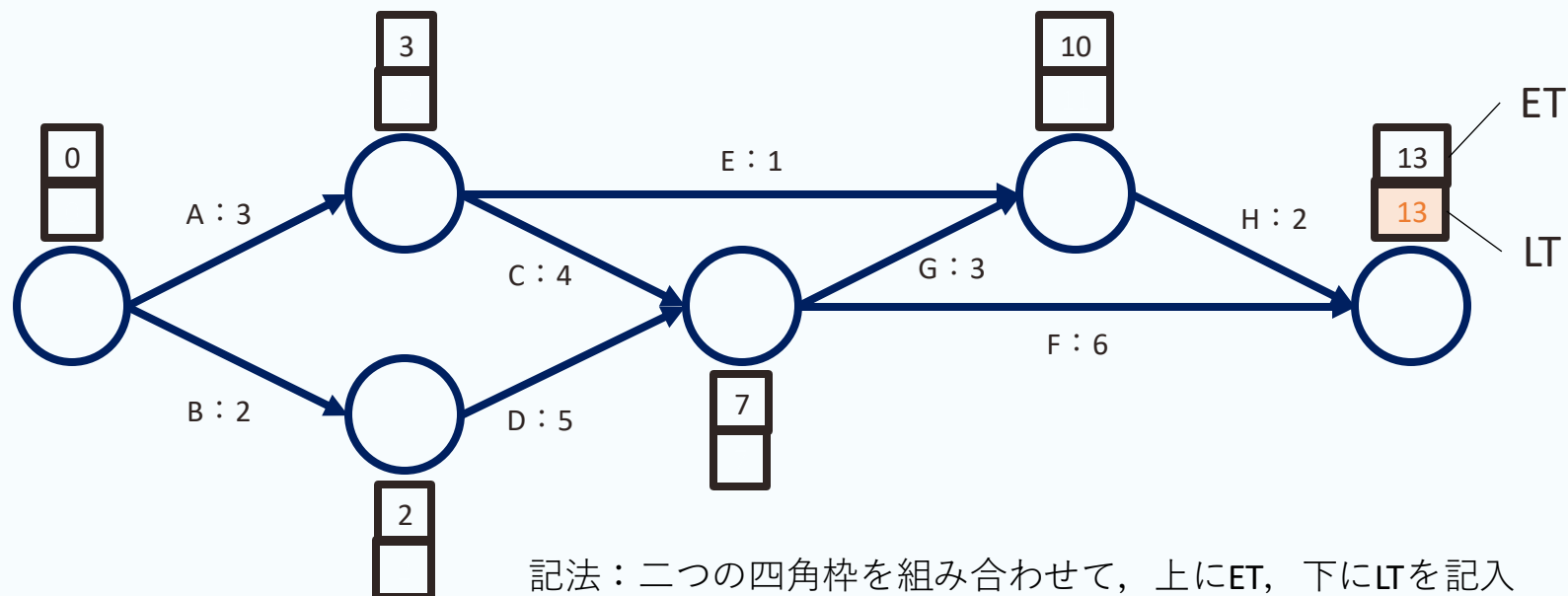
1. 開始地点のETを0に設定
 - ET_1 は0
2. 各結合点のETを計算
 - 終点のET = 始点のET + 作業日数
 - 複数の経路がある場合は、最も大きい（遅い）値をETとする（遅れた方の作業が終わるまで次の作業が始められないため）



結合点を最も遅く出発して、全体に遅延を出さない限界の時刻

=各作業の終点の最早結合点時刻から作業時間を引いた値

1. 終了地点のLTをETと同じ値に設定



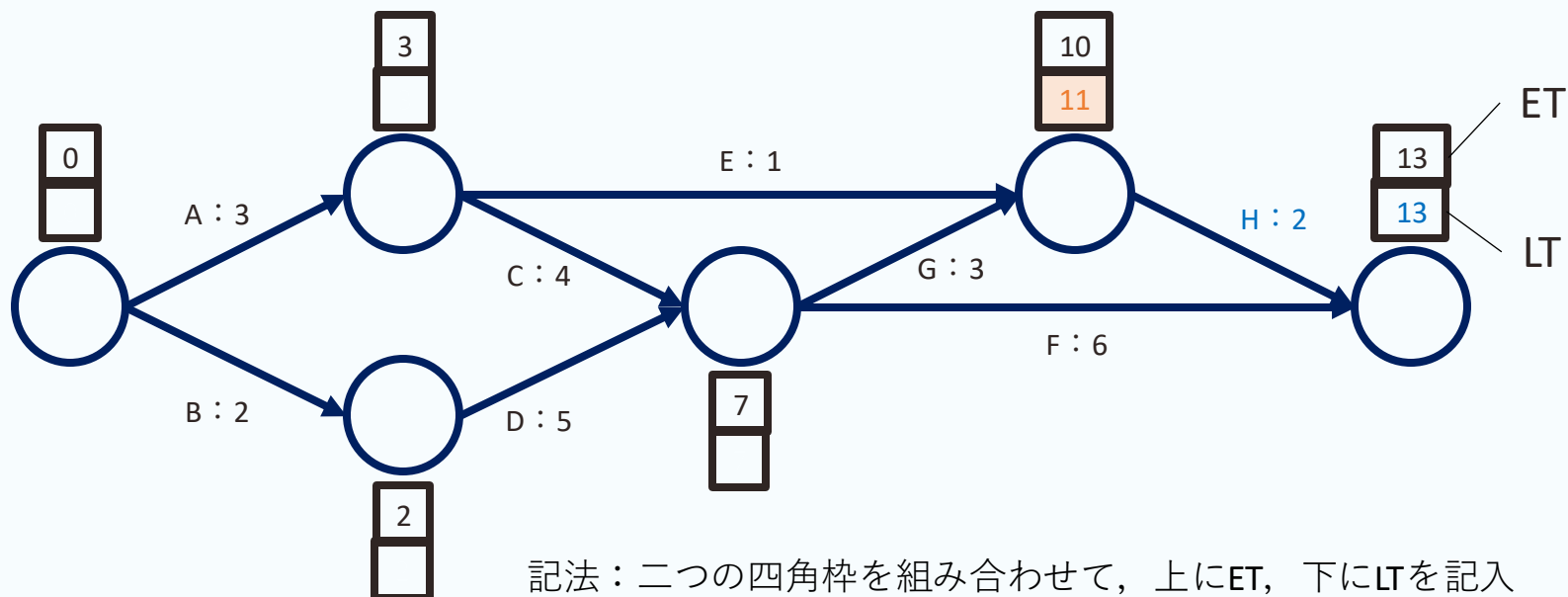
結合点を最も遅く出発して、全体に遅延を出さない限界の時刻

=各作業の終点の最早結合点時刻から作業時間を引いた値

1. 終了地点のLTをETと同じ値に設定

2. 各結合点のLTを計算

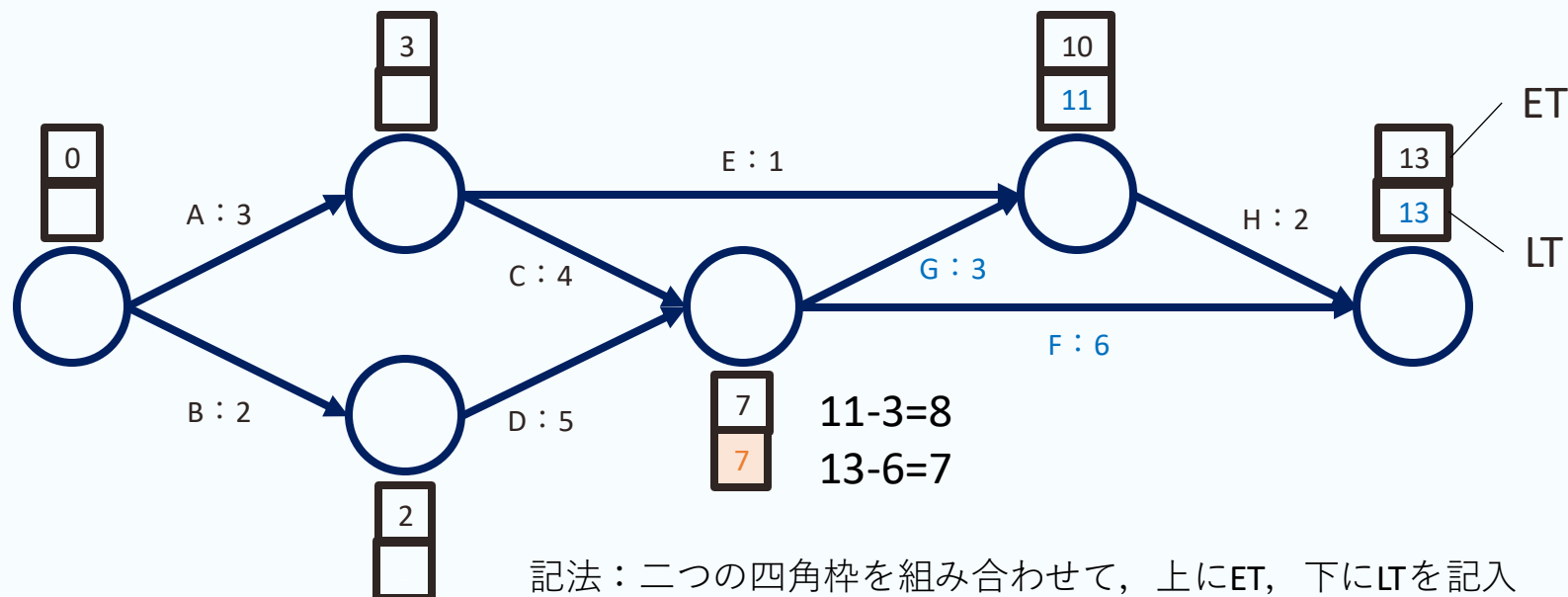
・ 始点のLT = 終点のLT - 作業日数



結合点を最も遅く出発して，全体に遅延を出さない限界の時刻

=各作業の終点の最早結合点時刻から作業時間を引いた値

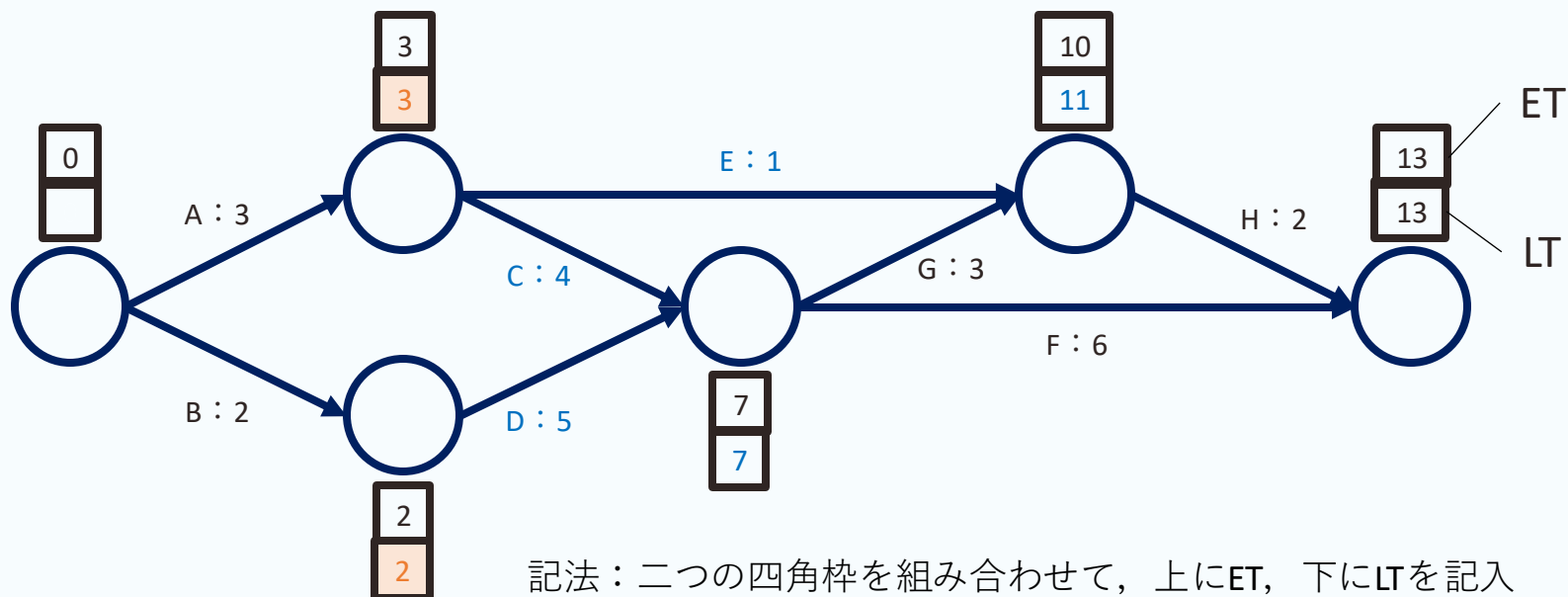
1. 終了地点のLTをETと同じ値に設定
2. 各結合点のLTを計算
 - 始点のLT = 終点のLT - 作業日数
 - 複数の経路がある場合は，最小の値をLTとする
（早く出発しないと時間がかかる方の作業に遅延が生じるため）



結合点を最も遅く出発して、全体に遅延を出さない限界の時刻

=各作業の終点の最早結合点時刻から作業時間を引いた値

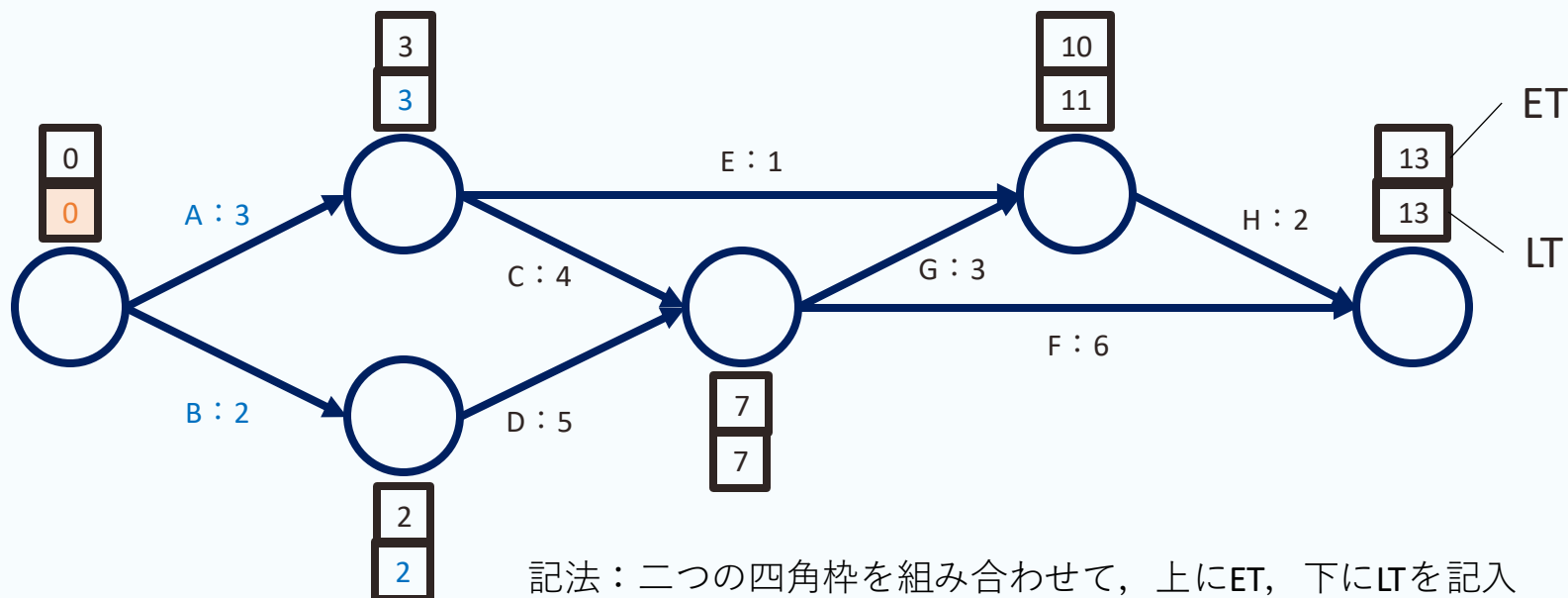
1. 終了地点のLTをETと同じ値に設定
2. 各結合点のLTを計算
 - 始点のLT = 終点のLT - 作業日数
 - 複数の経路がある場合は、最小の値をLTとする
(早く出発しないと時間がかかる方の作業に遅延が生じるため)



結合点を最も遅く出発して、全体に遅延を出さない限界の時刻

=各作業の終点の最早結合点時刻から作業時間を引いた値

1. 終了地点のLTをETと同じ値に設定
2. 各結合点のLTを計算
 - 始点のLT = 終点のLT - 作業日数
 - 複数の経路がある場合は、最小の値をLTとする
(早く出発しないと時間がかかる方の作業に遅延が生じるため)



1. PERT

- 日程管理計画問題
- PERTとアローダイアグラム
- 最早結合点時刻（ET）と最遅結合点時刻（LT）

2. 作業の日程計画

- 結合点の時刻に基づく作業日程の計算
- 余裕とクリティカルパス

3. 決定表

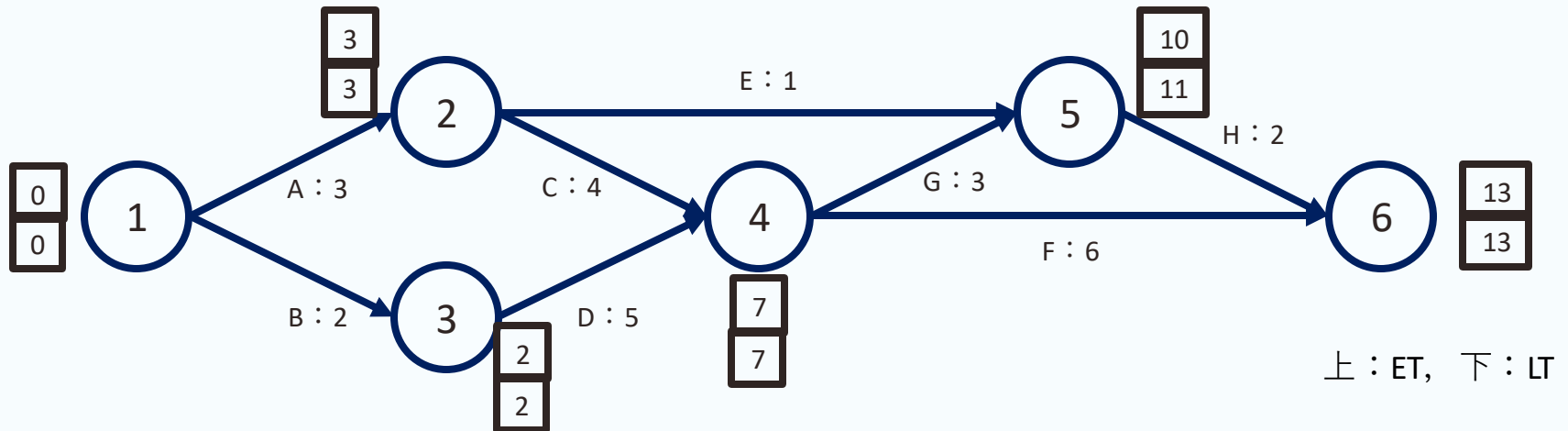
- 条件と結果の対応関係を一覧できる表

結合点の日程を用いた作業日程の計算

結合点よりも作業についての日程の方が扱いやすい

結合点の日程：作業終了と同時に次作業を開始するという意味における日程

→作業の日程：作業の開始時刻，終了時刻，遅延が生じた際の余裕



PERT（スケジューリング手法）に定められた4種類の時刻

最早開始時刻（ES）：その作業を最も早く始められる時刻
始点の最早結合点時刻（ET）

最早終了時刻（EF）：その作業を最も早く終了できる時刻
始点のET+その作業にかかる時間

最遅開始時刻（LS）：その作業を遅くとも始めなければならない時刻
終点のLT-その作業にかかる時間

最遅終了時刻（LF）：その作業を終えなければならないギリギリの時刻
終点の最遅結合点時刻（LT）

全余裕時間（TF：Total Float）

- 余裕時間の最大限界
- プロジェクト全体に遅れを生じさせないために、作業に許される最大の遅延

自由余裕時間（FF：Free Float）

- 他の作業に影響を与えない余裕時間
- 他の作業のTFを削減しないような範囲で、作業に許される最大の遅延

余裕時間の最大限界

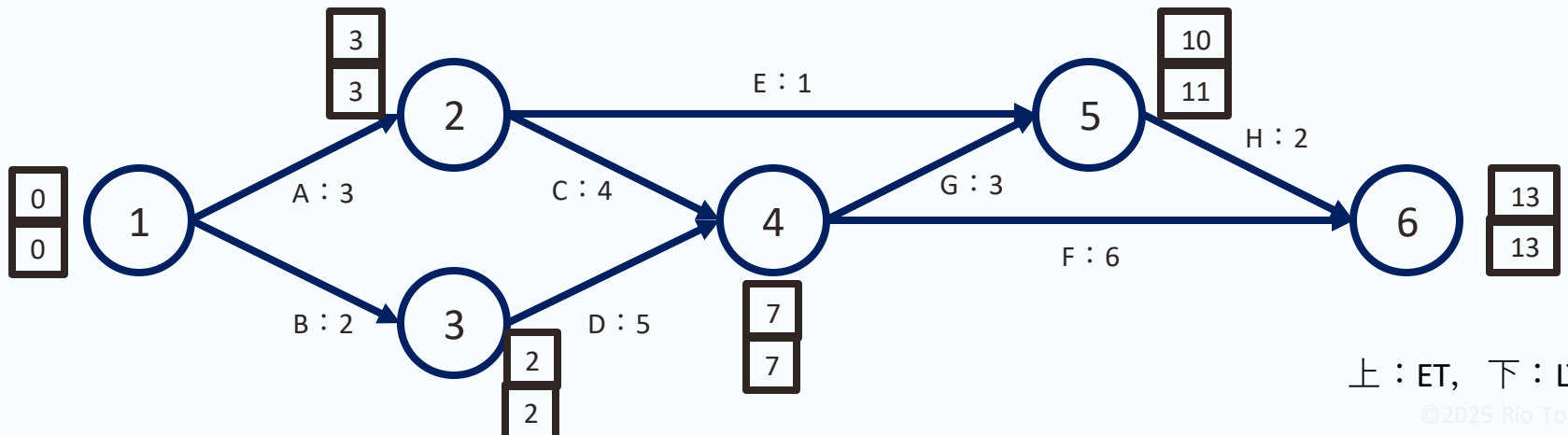
=プロジェクト全体の遅延を起こさないために、ある作業に許される遅れの最大値

作業記号	作業(i,j)	所要日数 $D_{i,j}$	TF
A	(1,2)	3	$3 - (0 + 3) = 0$
B	(1,3)	2	$2 - (0 + 2) = 0$
C	(2,4)	4	$7 - (3 + 4) = 0$
D	(3,4)	5	$7 - (2 + 5) = 0$
E	(2,5)	1	$11 - (3 + 1) = 7$
F	(4,6)	6	$13 - (7 + 6) = 0$
G	(4,5)	3	$11 - (7 + 3) = 1$
H	(5,6)	2	$13 - (10 + 2) = 1$

$$TF_{i,j} = LT_j - (ET_i + D_{i,j})$$

この作業の余裕（許容される最大の遅延） =
終点の最遅結合点時刻 - (始点最早結合点時刻 + 予定作業時間)

例： $TF_{5,6} = LT_6 - (ET_5 + D_{5,6})$



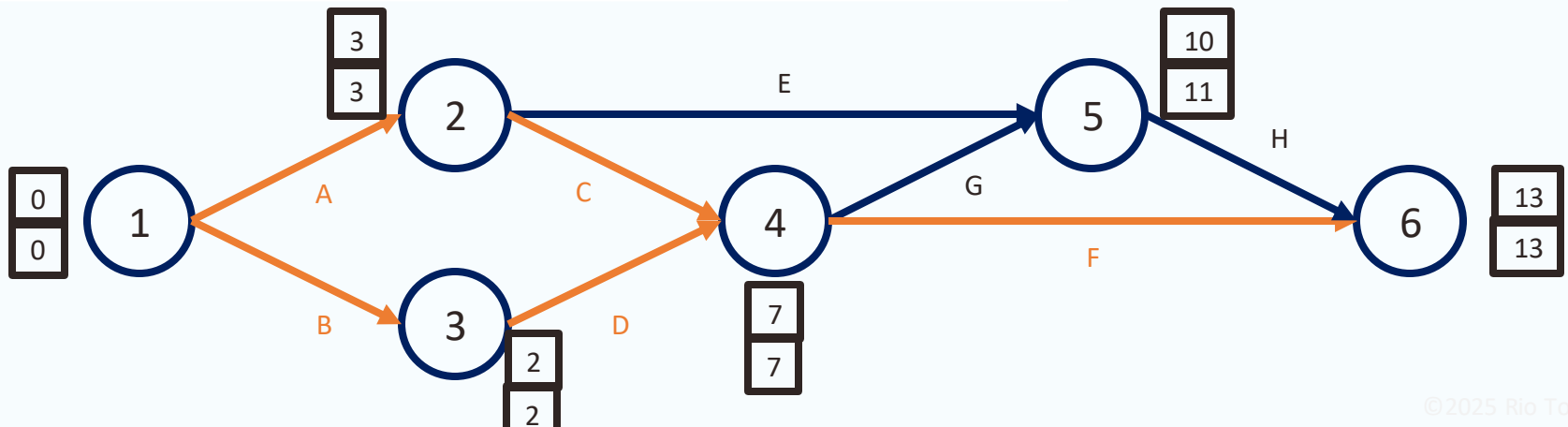
上 : ET, 下 : LT

プロジェクト全体の完了までに最も時間がかかる作業経路

=全余裕時間がゼロ (TF=0) の作業順からなる経路

作業記号	作業(i,j)	所要日数 $D_{i,j}$	TF	CP
A	(1,2)	3	0	*
B	(1,3)	2	0	*
C	(2,4)	4	0	*
D	(3,4)	5	0	*
E	(2,5)	1	7	
F	(4,6)	6	0	*
G	(4,5)	3	1	
H	(5,6)	2	1	

CPに該当する作業は*を記入



余裕時間の最大限界

=プロジェクト全体の遅延を起こさないために、ある作業に許される遅れの最大値

作業記号	作業(i,j)	所要日数 $D_{i,j}$	TF
A	(1,2)	3	0
B	(1,3)	2	0
C	(2,4)	4	0
D	(3,4)	5	0
E	(2,5)	1	7
F	(4,6)	6	0
G	(4,5)	3	1
H	(5,6)	2	1

$$TF_{i,j} = LF_{i,j} - EF_{i,j}$$

この作業の余裕（許容される最大の遅延） = 最遅終了時刻 - 最早開始時刻

他の作業に影響を与えない余裕時間

=他の作業のTFを削減しないような範囲で，作業に許される最大の遅延

- TFの範囲内の遅延は，計画全体の遅延はもたらさない一方で，後続作業の開始時刻を遅らせて，余裕を減らすことがある
- 他の作業の余裕を減らすことのない遅延の範囲はFFで確認できる

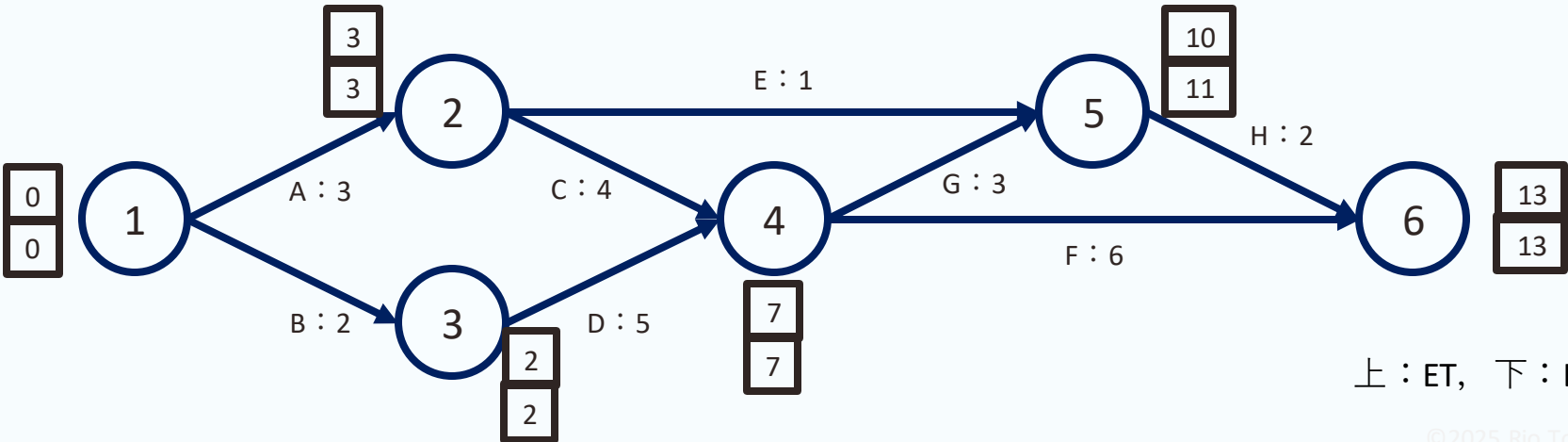
作業記号	作業(i,j)	所要日数 $D_{i,j}$	TF	FF
A	(1,2)	3	0	
B	(1,3)	2	0	
C	(2,4)	4	0	
D	(3,4)	5	0	
E	(2,5)	1	7	
F	(4,6)	6	0	
G	(4,5)	3	1	
H	(5,6)	2	1	

$$FF_{i,j} = ES_{j,k} - EF_{i,j}$$

※FFの計算は演習課題に含まれます

この作業の余裕（他に影響しない遅延）= 次の作業の最早作業時刻 - この作業の最遅終了時刻

記号	(i,j)	$D_{i,j}$	ES	EF	LS	LF	TF	FF	CP
A	(1,2)	3					0		
B	(1,3)	2					0		
C	(2,4)	4					0		
D	(3,4)	5					0		
E	(2,5)	1					7		
F	(4,6)	6					0		
G	(4,5)	3					1		
H	(5,6)	2					1		



1. PERT

- 日程管理計画問題
- PERTとアローダイアグラム
- 最早結合点時刻（ET）と最遅結合点時刻（LT）

2. 作業の日程計画

- 結合点の時刻に基づく作業日程の計算
- 余裕とクリティカルパス

3. 決定表

- 条件と結果の対応関係を一覧できる表

条件と結果の対応関係を一覧できる表

条件1	○	○	×	×
条件2	○	×	○	×
結果1	○			
結果2		○		○
結果3			○	

- 条件1 and 条件2 → 結果1
- 条件1 and not (条件2) → 結果2
- not(条件1) and 条件2 → 結果3
- not(条件1) and not(条件2) → 結果2

条件と結果の対応関係を一覧できる表

条件1	○	○	×	×
条件2	○	×	○	×
結果1	○			
結果2		○		○
結果3			○	

条件1 and 条件2 → 結果1
条件1 and not (条件2) → 結果2
not(条件1) and 条件2 → 結果3
not(条件1) and not(条件2) → 結果2

マナーモード	○	○	○	○	×	×	×	×
重要な連絡先	○	○	×	×	○	○	×	×
就寝時間	○	×	○	×	○	×	○	×
通知音						○		○
バイブレーション	○	○			○			
画面表示のみ				○			○	
通知無し			○					

各部分の名称

条件記述部	条件1	○	○	×	×	条件部
	条件2	○	×	○	×	
動作記述部	結果1	○				動作部
	結果2		○		○	
	結果3			○		
記述部		指定部				

表現方法

- 国家規格（JIS X 0125:1986）として定められた厳密な記法が存在するが、
本授業では簡単のため概念的な説明のみに留める
- 同一の内容をプログラミング言語（**条件分岐**）やフローチャート（**ツリー**）で表現することもできる

「もし（条件）なら {処理} 」という形で書く

- 条件が真のとき、処理内容が実行される
- 条件が偽のとき、処理内容は実行されない

数値 月 = 5

文字列 天気 = “晴れ”

文字列 危険 = “なし”

もし（月==12 or 月==1 or 月==2）なら {
文字列 季節 = “冬”

}

もし（月==3 or 月==4 or 月==5）なら {
文字列 季節 = “春”

}

もし（月==6 or 月==7 or 月==8）なら {
文字列 季節 = “夏”

}

もし（月==9 or 月==10 or 月==11）なら {
文字列 季節 = “秋”

}

もし（季節==冬 and 天気== 晴れ）なら {
危険 = “火事”

}

例1) 季節判定プログラム

数値 価格 = 100

文字列 商品カテゴリ = 食品

もし（商品カテゴリ == “食品”）なら {
数値 税率 = 0.08

}

もし（商品カテゴリ == “その他”）なら {
数値 税率 = 0.10

}

数値 消費税 = 価格 × 税率

数値 合計金額 = 価格 + 消費税

例2) 消費税計算プログラム

階層関係を枝分かれによって表現した構造

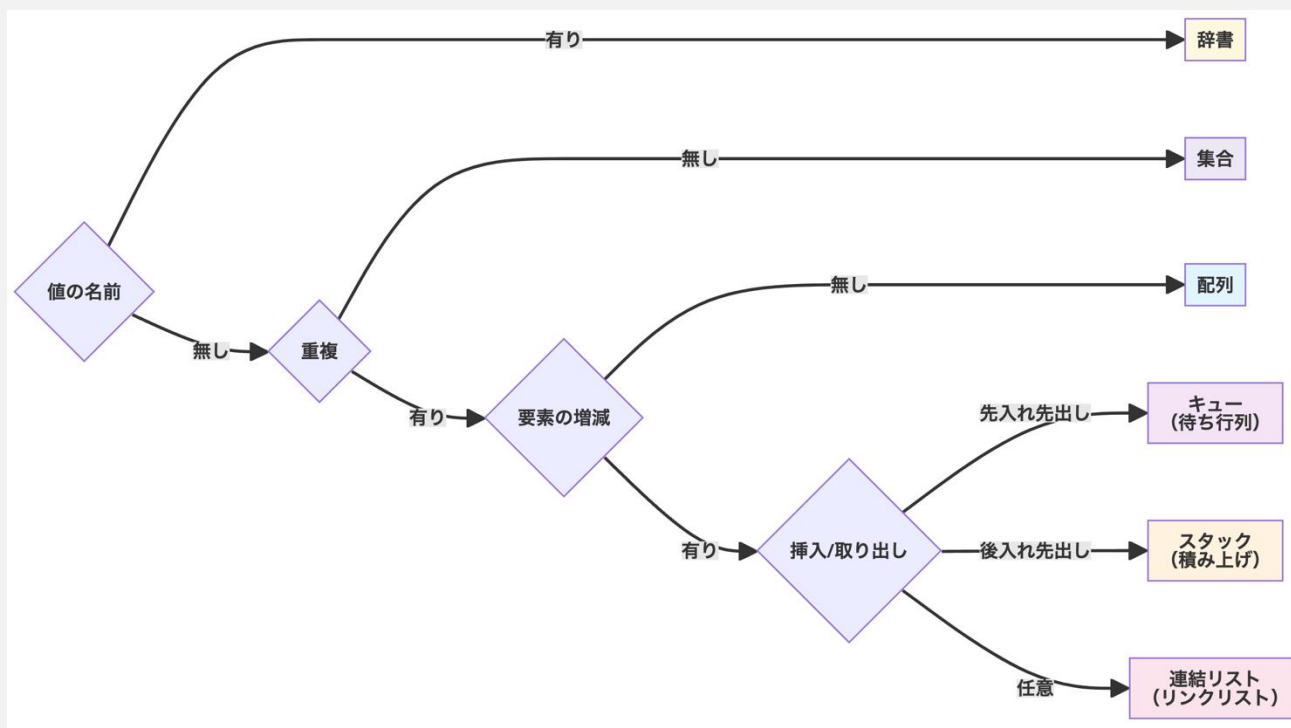
比喻）一つの根から枝分かれをして，末端には葉（情報）がある

→条件分岐（Yes/No）を繰り返すだけで，素早く必要な情報を探し出せる

例1) **決定木**：起点から分岐を続けて，結論に至る

例2) **組織図**：最高責任者から分岐を続けて，各部署に到達する

例3) **ファイルシステム**：ファイルは一つのフォルダに必ず属する



2025年度 コンピュータシステム 第15回： 演習（PERT, 決定表）

1. PERTの自動計算

- 記述しやすいET, LTの計算方法
- ES, EF, LF, LS, TF, FF, CPの記述方法

2. 決定表の自動計算

- 決定表, 条件分岐, 決定木の関係

グラフが複雑化・大規模化すると計算能率が悪くなる

- 規則的に計算する方法が存在
- プログラミング言語などで実装すれば自動的に計算することも可能

「所要計算表」を用いることで単純な規則でET・LTを導出可能
→プログラムを作成して、計算機に自動計算させることが可能

作業記号	作業(i,j)	所要日数 $D_{i,j}$
A	(1,2)	3
B	(1,3)	2
C	(2,4)	4
D	(3,4)	5
E	(2,5)	1
F	(4,6)	6
G	(4,5)	3
H	(5,6)	2

LTj								
j	1	2	3	4	5	6	i	ETi
							1	
							2	
							3	
							4	
							5	
							6	

①ある作業(i,j)のiを縦方向に探索し、②jを横方向に探索し、その交点座標にあたる欄内に所要時間を入力する

作業記号	作業(i,j)	所要日数 $D_{i,j}$
A	(1,2)	3
B	(1,3)	2
C	(2,4)	4
D	(3,4)	5
E	(2,5)	1
F	(4,6)	6
G	(4,5)	3
H	(5,6)	2

LTj								
j	1	2	3	4	5	6	i	ETi
②		3					1	
							2	
							3	
							4	
							5	
							6	

```
関数 線形探索(配列 探索対象, 整数 探索値){  
    整数 長さ = 配列の要素数を取得する関数（探索対象）  
    整数 繰返し数 = 0  
    (長さ) 回だけ繰返す{  
        もし（ 配列[繰返し数] が 探したい値 ） と等しいとき{  
            整数 探索値の場所 == 繰返し数  
            プログラムを終了  
        }  
    }  
    繰返しが終了したら、探したい値はないと判断する  
}
```



<https://claude.ai/public/artifacts/d07b7208-44e6-4d47-8f54-079161fe30eb>

※PC・タブレット推奨，スマホはレイアウトが崩れる可能性が高いです

探索の開始位置を配列の先頭とする

探索の終了位置を配列の末尾とする

（開始位置 $\leq$ 終了位置）の間、以下を繰り返す{

探索範囲の中央を計算する。ただし、中央が少数なら切り捨てる。

中央の位置にある値と探している値を比べる

もし（中央の値 == 探している値）ならば、{

探している値の位置を中央の値として動作終了

}

もし（中央の値 < 探している値）ならば{

終了位置を中央の一つ後ろに変更する

}

もし（中央の値 > 探している値）ならば{

開始位置を中央の一つ後ろに変更する

}

}

繰り返しが終了したら、探したい値は存在しないと判断する

⚡ 二分探索アルゴリズム視覚化

探索する値:
7 ランダム選択

配列の値 (カンマ区切り):
3,1,7,9,2,5,10,15,8,12,4,6,11,14,13 配列更新 ランダム生成

開始位置 終了位置 中央位置 探索範囲 範囲外

[0] [1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14]
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
開始 終了

探索範囲: [0] ~ [14] | 中央位置: [7] | 中央の値: 8 | 探索値: 7

中央値 8 > 7: 左半分を探索

探索開始 ← 1つ戻す 1つ進める → リセット

現在の状態: ステップ 1: 中央値 8 が探索値より大きいため、左半分を探索します。

二分探索の疑似コード

Made with Claude

Artifacts are user-generated and may contain unverified or potentially unsafe content. Report Customize Artifact

<https://claude.ai/public/artifacts/4ac68507-ef42-4c2e-9a05-9af4fa555d5e>

※PC・タブレット推奨，スマホはレイアウトが崩れる可能性が高いです

①ある作業(i,j)のiを縦方向に探索し、②jを横方向に探索し、その交点座標にあたる欄内に所要時間を入力する

作業記号	作業(i,j)	所要日数 $D_{i,j}$
A	(1,2)	3
B	(1,3)	2
C	(2,4)	4
D	(3,4)	5
E	(2,5)	1
F	(4,6)	6
G	(4,5)	3
H	(5,6)	2

LTj								
j	1	2	3	4	5	6	i	ETi
②		3					1	
							2	
							3	
							4	
							5	
							6	

①ある作業(i,j)のiを縦方向に探索し，②jを横方向に探索し，
その交点座標にあたる欄内に所要時間を入力する

作業記号	作業(i,j)	所要日数 $D_{i,j}$
A	(1,2)	3
B	(1,3)	2
C	(2,4)	4
D	(3,4)	5
E	(2,5)	1
F	(4,6)	6
G	(4,5)	3
H	(5,6)	2

LTj								
j	1	2	3	4	5	6	i	ETi
		3	2				1	
							2	
							3	
							4	
							5	
							6	

①ある作業(i,j)のiを縦方向に探索し，②jを横方向に探索し，その交点座標にあたる欄内に所要時間を入力する

作業記号	作業(i,j)	所要日数 $D_{i,j}$
A	(1,2)	3
B	(1,3)	2
C	(2,4)	4
D	(3,4)	5
E	(2,5)	1
F	(4,6)	6
G	(4,5)	3
H	(5,6)	2

LTj								
j	1	2	3	4	5	6	i	ETi
		3	2				1	
				4			2	
							3	
							4	
							5	
							6	

①ある作業(i,j)のiを縦方向に探索し，②jを横方向に探索し，その交点座標にあたる欄内に所要時間を入力する

作業記号	作業(i,j)	所要日数 $D_{i,j}$
A	(1,2)	3
B	(1,3)	2
C	(2,4)	4
D	(3,4)	5
E	(2,5)	1
F	(4,6)	6
G	(4,5)	3
H	(5,6)	2

LTj								
j	1	2	3	4	5	6	i	ETi
		3	2				1	
				4			2	
				5			3	
							4	
							5	
							6	

①ある作業(i,j)のiを縦方向に探索し、②jを横方向に探索し、その交点座標にあたる欄内に所要時間を入力する

作業記号	作業(i,j)	所要日数 $D_{i,j}$
A	(1,2)	3
B	(1,3)	2
C	(2,4)	4
D	(3,4)	5
E	(2,5)	1
F	(4,6)	6
G	(4,5)	3
H	(5,6)	2

LTj								
j	1	2	3	4	5	6	i	ETi
		3	2				1	
				4	1		2	
				5			3	
							4	
							5	
							6	

①ある作業(i,j)のiを縦方向に探索し、②jを横方向に探索し、その交点座標にあたる欄内に所要時間を入力する

作業記号	作業(i,j)	所要日数 $D_{i,j}$
A	(1,2)	3
B	(1,3)	2
C	(2,4)	4
D	(3,4)	5
E	(2,5)	1
F	(4,6)	6
G	(4,5)	3
H	(5,6)	2

LTj								
j	1	2	3	4	5	6	i	ETi
		3	2				1	
				4	1		2	
				5			3	
						6	4	
							5	
							6	

①ある作業(i,j)のiを縦方向に探索し、②jを横方向に探索し、その交点座標にあたる欄内に所要時間を入力する

作業記号	作業(i,j)	所要日数 $D_{i,j}$
A	(1,2)	3
B	(1,3)	2
C	(2,4)	4
D	(3,4)	5
E	(2,5)	1
F	(4,6)	6
G	(4,5)	3
H	(5,6)	2

LTj								
j	1	2	3	4	5	6	i	ETi
		3	2				1	
				4	1		2	
				5			3	
					3	6	4	
							5	
							6	

①ある作業(i,j)のiを縦方向に探索し、②jを横方向に探索し、その交点座標にあたる欄内に所要時間を入力する

作業記号	作業(i,j)	所要日数 $D_{i,j}$
A	(1,2)	3
B	(1,3)	2
C	(2,4)	4
D	(3,4)	5
E	(2,5)	1
F	(4,6)	6
G	(4,5)	3
H	(5,6)	2

LTj								
j	1	2	3	4	5	6	i	ETi
		3	2				1	
				4	1		2	
				5			3	
					3	6	4	
						2	5	
							6	

1. $ET_0=0$

LTj								
j	1	2	3	4	5	6	i	ETi
		3	2				1	0
				4	1		2	
				5			3	
					3	6	4	
						2	5	
							6	

1. $ET_0=0$

2. ET_i は以下の手順で導出

「 i 」と同じ値を持つ「 j 」の列を縦方向に探索

例) ET_2 なら, $j=2$ の列を探す

値のある箇所を見つけて, その値とその真横にある ET_i を加算

例) ET_2 なら, 「 $3+0=3$ 」

LTj								
j	1	2	3	4	5	6	i	ETi
		3	2				1	0
				4	1		2	3
				5			3	
					3	6	4	
						2	5	
							6	

- 1. $ET_0=0$
- 2. ET_i は以下の手順で導出
 - 「i」と同じ値を持つ「j」の列を縦方向に探索
 - 例) ET_3 なら, $j=3$ の列を探す
 - 値のある箇所を見つけて, その値とその真横にある ET_i を加算
 - 例) ET_3 なら, 「 $2+0=2$ 」

LTj								
j	1	2	3	4	5	6	i	ETi
		3	2				1	0
				4	1		2	3
				5			3	2
					3	6	4	
						2	5	
							6	

1. $ET_0=0$

2. ET_i は以下の手順で導出

「 i 」と同じ値を持つ「 j 」の列を縦方向に探索

例) ET_2 なら, $j=2$ の列を探す

値のある箇所を見つけて, その値とその真横にある ET_i を加算

複数の値がある場合は, その中で最大のものを採用

例) ET_4 なら, 「 $4+3=7$ 」か, 「 $5+2=7$ 」の大きい方を採用 (同じなので7にする)

LTj								
j	1	2	3	4	5	6	i	ETi
		3	2				1	0
				4	1		2	3
				5			3	2
					3	6	4	7
						2	5	
							6	

1. $ET_0=0$

2. ET_i は以下の手順で導出

「 i 」と同じ値を持つ「 j 」の列を縦方向に探索

例) ET_2 なら, $j=2$ の列を探す

値のある箇所を見つけて, その値とその真横にある ET_i を加算

複数の値がある場合は, その中で最大のものを採用

例) ET_5 なら, 「 $1+3=4$ 」か, 「 $3+7=10$ 」の大きい方を採用

LTj								
j	1	2	3	4	5	6	i	ETi
		3	2				1	0
				4	1		2	3
				5			3	2
					3	6	4	7
						2	5	10
							6	

1. $ET_0=0$

2. ET_i は以下の手順で導出

「 i 」と同じ値を持つ「 j 」の列を縦方向に探索

例) ET_2 なら, $j=2$ の列を探す

値のある箇所を見つけて, その値とその真横にある ET_i を加算

複数の値がある場合は, その中で最大のものを採用

例) ET_6 なら, 「 $6+7=13$ 」か, 「 $2+10=12$ 」の大きい方を採用

LTj								
j	1	2	3	4	5	6	i	ETi
		3	2				1	0
				4	1		2	3
				5			3	2
					3	6	4	7
						2	5	10
							6	13

1. 最大の「j」について, $LT_j = ET_j$ と代入

LTj						13		
j	1	2	3	4	5	6	i	ETi
		3	2				1	0
				4	1		2	3
				5			3	2
					3	6	4	7
						2	5	10
							6	13

1. 最大の「j」について, $LT_j = ET_j$ と代入
2. その他 LT_j は以下の手順で導出
 - ・ 「j」と同じ値を持つ「i」の列を横方向に探索
 - ・ 値のある箇所にて, その真上にある LT_j からその値を減算

LTj					11	13		
j	1	2	3	4	5	6	i	ETi
		3	2				1	0
				4	1		2	3
				5			3	2
					3	6	4	7
						2	5	10
							6	13

1. 最大の「j」について, $LT_j = ET_j$ と代入
2. その他 LT_j は以下の手順で導出
 - ・「j」と同じ値を持つ「i」の列を横方向に探索
 - ・値のある箇所にて, その真上にある LT_j からその値を減算

LTj					11	13		
j	1	2	3	4	5	6	i	ETi
		3	2				1	0
				4	1		2	3
				5			3	2
					3	6	4	7
						2	5	10
							6	13

- 1. 最大の「j」について、 $LT_j = ET_j$ と代入
- 2. その他 LT_j は以下の手順で導出
 - ・「j」と同じ値を持つ「i」の列を横方向に探索
 - ・値のある箇所にて、その真上にある LT_j からその値を減算
 - ・複数の値がある場合は、その中で最小のものを採用

LTj				7	11	13		
j	1	2	3	4	5	6	i	ETi
		3	2				1	0
				4	1		2	3
				5			3	2
					3	6	4	7
						2	5	10
							6	13

1. 最大の「j」について, $LT_j = ET_j$ と代入
2. その他 LT_j は以下の手順で導出
 - 「j」と同じ値を持つ「i」の列を横方向に探索
 - 値のある箇所にて, その真上にある LT_j からその値を減算
 - 複数の値がある場合は, その中で最小のものを採用

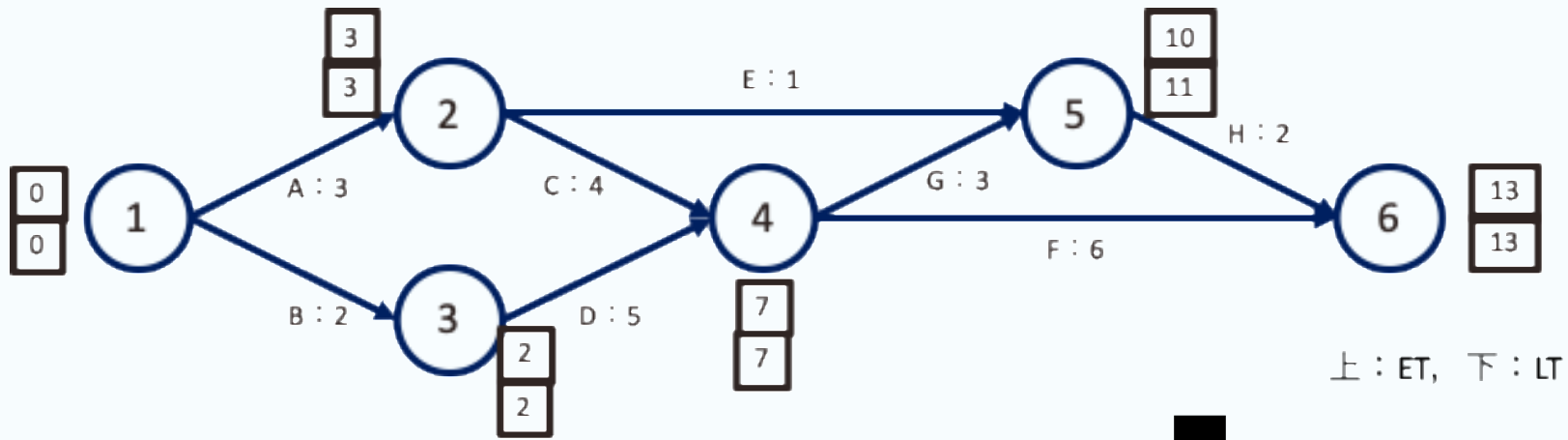
LTj			2	7	11	13		
j	1	2	3	4	5	6	i	ETi
		3	2				1	0
				4	1		2	3
				5			3	2
					3	6	4	7
						2	5	10
							6	13

1. 最大の「j」について、 $LT_j = ET_j$ と代入
2. その他 LT_j は以下の手順で導出
 - ・「j」と同じ値を持つ「i」の列を横方向に探索
 - ・値のある箇所にて、その真上にある LT_j からその値を減算
 - ・複数の値がある場合は、その中で最小のものを採用

LTj		3	2	7	11	13		
j	1	2	3	4	5	6	i	ETi
		3	2				1	0
				4	1		2	3
				5			3	2
					3	6	4	7
						2	5	10
							6	13

1. 最大の「j」について, $LT_j = ET_j$ と代入
2. その他 LT_j は以下の手順で導出
 - ・ 「j」と同じ値を持つ「i」の列を横方向に探索
 - ・ 値のある箇所にて, その真上にある LT_j からその値を減算
 - ・ 複数の値がある場合は, その中で最小のものを採用

LTj	0	3	2	7	11	13		
j	1	2	3	4	5	6	i	ETi
		3	2				1	0
				4	1		2	3
				5			3	2
					3	6	4	7
						2	5	10
							6	13



LTj	0	3	2	7	11	13		
j	1	2	3	4	5	6	i	ETi
		3	2				1	0
				4	1		2	3
				5			3	2
					3	6	4	7
						2	5	10
							6	13



i	ET _i	LT _i
1	0	0
2	3	3
3	2	2
4	7	7
5	10	11
6	13	13

数式と判定条件をそのままプログラムとして利用可能

- $ES_{ij} = ET_i$
- $EF_{ij} = ES_{ij} + D_{ij}$
- $LF_{ij} = LT_j$
- $LS_{ij} = LF_{ij} - D_{ij}$
- $TF_{ij} = LF_{ij} - EF_{ij}$
- $FF_{ij} = ES_{jk} - EF_{ij}$
- もし ($TF_{ij}=0$) ならば CP_{ij} は「*」, そうでないなら CP_{ij} は空欄 (条件分岐)

			作業	(i,j)	D_{ij}	ES_{ij}	EF_{ij}	LS_{ij}	LF_{ij}	TF_{ij}	FF_{ij}	CP_{ij}
結合点(i)	ET_i	LT_i	A	(1,2)	3	0	3	0	3	0	0	*
1	0	0	B	(1,3)	2	0	2	0	2	0	0	*
2	3	3	C	(2,4)	4	3	7	3	7	0	0	*
3	2	2	D	(3,4)	5	2	7	2	7	0	0	*
4	7	7	E	(2,5)	1	3	4	10	11	7	6	
5	10	11	F	(4,6)	6	7	13	7	13	0	0	*
6	13	13	G	(4,5)	3	7	10	8	11	1	0	
			H	(5,6)	2	10	12	11	13	1	1	

1. PERTの自動計算

- 記述しやすいET, LTの計算方法
- ES, EF, LF, LS, TF, FF, CPの記述方法

2. 決定表の自動計算

- 決定表, 条件分岐, 決定木の関係

条件と結果の対応関係を一覧できる表

条件1	○	○	×	×
条件2	○	×	○	×
結果1	○			
結果2		○		○
結果3			○	

条件1 and 条件2 → 結果1
条件1 and not (条件2) → 結果2
not(条件1) and 条件2 → 結果3
not(条件1) and not(条件2) → 結果2

条件と結果の対応関係を一覧できる表

条件記述部	条件1	○	○	×	×	条件部
	条件2	○	×	○	×	
動作記述部	結果1	○				動作部
	結果2		○		○	
	結果3			○		
記述部		指定部				

表現方法

- 国家規格（JIS X 0125:1986）として定められた厳密な記法が存在するが、
本授業では簡単のため概念的な説明のみに留める
- 同一の内容をプログラミング言語（**条件分岐**）やフローチャート（**ツリー**）で表現することもできる

条件1	○	○	×	×
条件2	○	×	○	×
結果1	○			
結果2		○		○
結果3			○	

もし（条件1 AND 条件2）なら {
結果1

}

もし（条件1 AND NOT条件2）なら {
結果2

}

もし（NOT条件1 AND 条件2）なら {
結果3

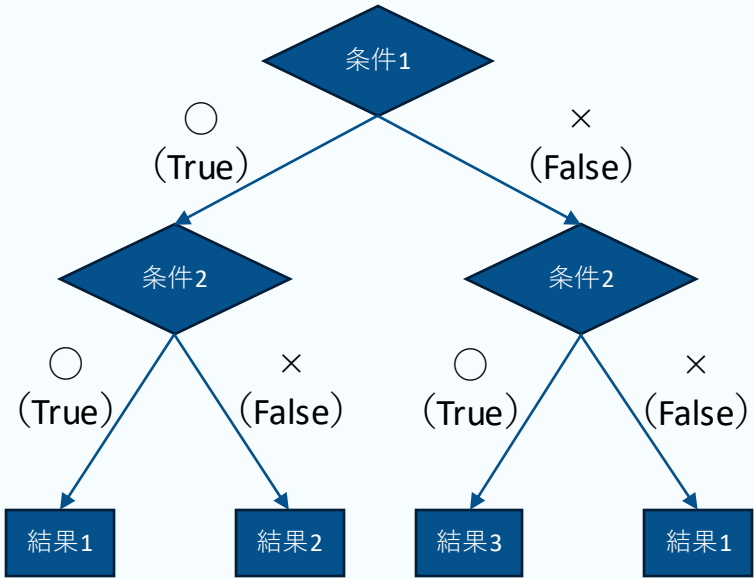
}

もし（NOT条件1 AND NOT条件2）なら {
結果4

}

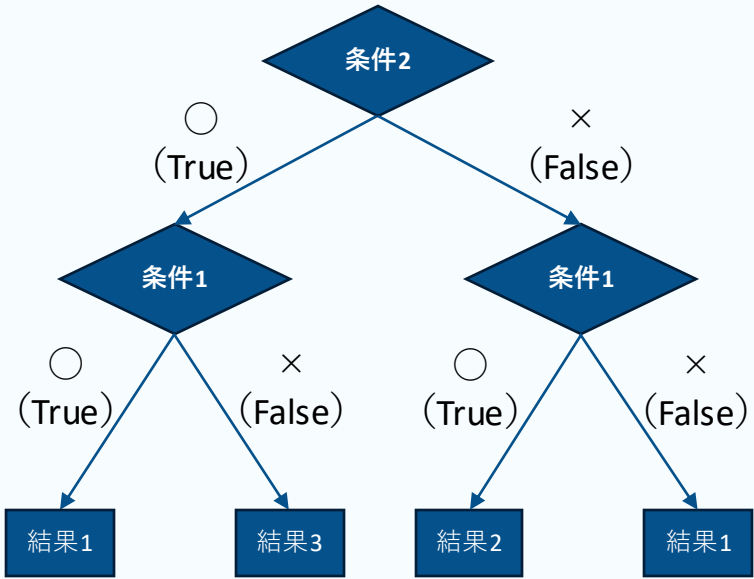
疑似コードで書いた条件分岐
（プログラミング言語）

条件1	○	○	×	×
条件2	○	×	○	×
結果1	○			
結果2		○		○
結果3			○	



フローチャート
(木構造)

条件1	○	○	×	×
条件2	○	×	○	×
結果1	○			
結果2		○		○
結果3			○	



フローチャート
（木構造）